



**PENGEMBANGAN FITUR MOODLE LATIHAN
PEMROGRAMAN INTERAKTIF DENGAN UMPAN BALIK
CERDAS BERBASIS LARGE LANGUAGE MODEL**

Skripsi

diajukan sebagai salah satu syarat untuk memperoleh gelar Sarjana Pendidikan

Oleh

Annuruddin Tsanasti Yassar

5302422055

**PENDIDIKAN TEKNIK INFORMATIKA DAN KOMPUTER
FAKULTAS TEKNIK
UNIVERSITAS NEGERI SEMARANG
SEMARANG, 2026**

PERSETUJUAN PEMBIMBING

Skripsi berjudul “Pengembangan Fitur Moodle Latihan Pemrograman Interaktif Dengan Umpan Balik Cerdas Berbasis Large Language Model” yang disusun oleh

Nama : Annuruddin Tsanasti Yassar

NIM : 5302422055

Prodi : Pendidikan Teknik Informatika dan Komputer

telah disetujui untuk diajukan ke penilaian ujian skripsi.

Semarang, 6 Juli 2026

Pembimbing

Dr. Djuniadi, M.T.

PENGESAHAN TIM PENGUJI

Skripsi berjudul “Pengembangan Fitur Moodle Latihan Pemrograman Interaktif Dengan Umpan Balik Cerdas Berbasis Large Language Model” yang disusun oleh

Nama : Annuruddin Tsanasti Yassar

NIM : 5302422055

Prodi/Fakultas : Pendidikan Teknik Informatika dan Komputer

telah dipertahankan dalam ujian skripsi pada hari Senin, tanggal 13 Juli 2026.

Tim Penguji

ketua Penguji Prof. Dr. Wirawan Sumbodo, M. T. NIP. 196601051990021002	
Sekretaris Budi Sunarko, S.T., M.T., Ph.D NIP. 197101042006041001	
Penguji 1 Nur Iksan, S.T., M.Kom. NIP. 198303072012121004	
Penguji 2/Pembimbing Dr. Djuniadi, M.T. NIP. 196306281990021001	

PERNYATAAN

Skripsi yang ditulis berjudul “Pengembangan Fitur Moodle Latihan Pemrograman Interaktif Dengan Umpan Balik Cerdas Berbasis Large Language Model” merupakan karya ilmiah asli dan bukan hasil plagiasi dari karya ilmiah orang lain. Pendapat atau temuan orang lain yang dikutip di dalam skripsi ini telah ditulis berdasarkan kode etik ilmiah. Berdasarkan pernyataan ini, saya siap menanggung sanksi hukum apabila ditemukan pelanggaran terhadap integritas akademik dalam karya ini.

Semarang, 6 Juli 2026

Yang menyatakan

Annuruddin Tsanasti Yassar

5302422055

MOTTO DAN PERSEMBAHAN

MOTTO:

Learn from yesterday, live for today, hope for tomorrow. The important thing is not to stop questioning.

Albert Einstein

PERSEMBAHAN:

Skripsi ini penulis persembahkan kepada:

1. Allah Swt., Tuhan Yang Maha Esa, yang atas segala rahmat, hidayah, dan kekuatan-Nya senantiasa melimpahkan kelancaran serta kemudahan di setiap langkah penulis hingga karya ini dapat terselesaikan.
2. Kedua Orang Tua Tercinta, Bapak Asnawi dan Ibu Titi, sebagai wujud bakti, rasa hormat, dan terima kasih yang tak terhingga atas doa-doa, kasih sayang, pengorbanan yang luar biasa, serta dukungan yang menjadi pilar kekuatan utama bagi penulis.
3. Almamater Tercinta, Program Studi Pendidikan Teknik Informatika dan Komputer, Fakultas Teknik, Universitas Negeri Semarang, tempat penulis menimba ilmu, menempa diri, dan bertumbuh.

ABSTRAK

Yassar, Annuruddin Tsanasti. 2026. *Pengembangan Fitur Moodle Latihan Pemrograman Interaktif dengan Umpan Balik Cerdas Berbasis Large Language Model*. Skripsi. Program Studi Pendidikan Teknik Informatika dan Komputer Fakultas Teknik Universitas Negeri Semarang. Pembimbing Dr. Djuniadi, M.T.

Kata Kunci: *Activity Module Moodle, Automated Programming Assessment System, Prompt Engineering, Metode Sokratik, Large Language Model, Eksekusi Sandbox.*

Pendidikan vokasi bidang rekayasa perangkat lunak pada Sekolah Menengah Kejuruan (SMK) Kelas 11 Fase F menghadapi tantangan pedagogis dalam pembelajaran bahasa pemrograman JavaScript. Peserta didik pemula kerap mengalami kelebihan beban kognitif (*cognitive overload*) akibat pesan galat kompilator (*standard error*) yang kaku, nir-konteks, dan berbahasa Inggris. Di sisi lain, infrastruktur *Learning Management System* (LMS) Moodle sebagai standar institusional tidak memiliki modul bawaan untuk eksekusi kode interaktif. Penggunaan *platform* pihak ketiga memunculkan celah ketergantungan siswa terhadap fitur pelengkap kode otomatis (*auto-complete spoilers*) yang mematikan penalaran analitis, sementara eksekusi lokal pada peladen utama Moodle berisiko memicu eksploitasi keamanan siber. Untuk mengatasi kebuntuan tersebut, penelitian ini mengembangkan *plugin Activity Module Moodle* bernama Aicode melalui kerangka *Software Development Life Cycle* (SDLC) model Waterfall.

Sebagai inovasi instruksional, sistem mengintegrasikan model *Large Language Model* (LLM) Google Gemini yang dikunci melalui arsitektur instruksi berlapis (*layered constraint prompting*) agar secara eksklusif memposisikan diri sebagai tutor pedagogi Sokratik. Evaluasi perangkat lunak dilakukan melalui pendekatan ganda: uji fungsional arsitektur dan *benchmark* algoritma *prompt* terprogram. Pengujian fungsional membuktikan ketahanan absolut (100%) eksekutor *sandbox* dari retasan sistem berkas dan perekaman log yang sinkron secara mutlak dengan *Gradebook Moodle*. Sementara itu, uji performa *prompt* pada 390 pemanggilan API dengan 13 konfigurasi bereksperimen menorehkan rasio kesuksesan format (*Parsing Success Rate*) sebesar 99,2%. Studi ablasi menyingkap bahwa

pemberian peran eksklusif (*Role-Prompting*) menyumbang eskalasi akurasi identifikasi kelirumologi *coding* siswa hingga 20%, sekaligus membuktikan ketahanan sistem (100%) dalam menangkal invasi retasan instruksi komputasional (*prompt injection*). Hasil riset ini merepresentasikan infrastruktur penilaian koding otonom yang mapan dari celah kerentanan siber, serta secara logis mampu menuntun daya nalar peserta didik dalam kemelut produktif (*productive struggle*).

ABSTRACT

Yassar, Annuruddin Tsanasti. 2026. *Development of an Interactive Programming Practice Moodle Feature with Smart Feedback Based on Large Language Model*. Undergraduate Thesis. Informatics and Computer Engineering Education Study Program, Faculty of Engineering, Universitas Negeri Semarang. Advisor Dr. Djuniadi, M.T.

Keywords: *Moodle Activity Module, Automated Programming Assessment System, Prompt Engineering, Socratic Method, Large Language Model, Sandbox Execution.*

Vocational software engineering education for 11th-grade Phase F students in vocational high schools (SMK) confronts profound pedagogical challenges in teaching the JavaScript programming language. Novice learners frequently experience cognitive overload triggered by rigid, contextless, and English-based compiler standard errors. Furthermore, the Moodle Learning Management System (LMS), functioning as the institutional standard, inherently lacks interactive code execution capabilities. Utilizing third-party platforms engenders dependency on auto-complete spoiler features, whereas executing client code directly on the primary Moodle server poses a fatal risk of cybersecurity exploitation. To circumvent this pedagogical and infrastructural deadlock, this research develops a Moodle Activity Module plugin named Aicode, employing the Waterfall Software Development Life Cycle (SDLC) framework.

As an instructional innovation, the system integrates the Google Gemini Large Language Model (LLM), restricted via a layered constraint prompting architecture to act exclusively as a Socratic pedagogical tutor. The software evaluation utilized a dual-approach methodology: architectural functional testing and programmed prompt algorithm benchmarking. Functional testing demonstrated absolute resilience (100%) of the sandbox executor against filesystem hijacking, with flawlessly synchronized logging to the Moodle Gradebook. Concurrently, the prompt performance benchmark across 390 API calls and 13 experimental configurations achieved a Parsing Success Rate (PSR) of 99.2%. An ablation study revealed that assigning an explicit persona (Role-Prompting) contributed to a 20% escalation in coding error identification accuracy, while simultaneously demonstrating 100% defense against computational

prompt injections. This research yields an Automated Programming Assessment System (APAS) infrastructure that is robust against cyber vulnerabilities and pedagogically capable of facilitating productive struggle for novice programmers.

PRAKATA

Segala puji dan syukur senantiasa penulis panjatkan kehadirat Allah Swt. atas segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan skripsi yang berjudul "Pengembangan Fitur Moodle Latihan Pemrograman Interaktif dengan Umpan Balik Cerdas Berbasis *Large Language Model*" ini dengan baik. Selawat serta salam semoga senantiasa tercurah kepada junjungan kita, Nabi Muhammad Saw. yang telah membawa petunjuk dan mengantarkan umat manusia dari zaman kegelapan menuju zaman yang terang benderang.

Selama proses penyusunan skripsi ini, penulis menyadari bahwa terdapat berbagai rintangan dan hambatan yang dihadapi. Namun, berkat bimbingan, arahan, bantuan, serta dukungan moral dan material dari berbagai pihak, skripsi ini pada akhirnya dapat diselesaikan. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Prof. Dr. S. Martono, M.Si., selaku Rektor Universitas Negeri Semarang, atas segala kebijakan, kesempatan, dan penyediaan fasilitas akademik bagi penulis selama menempuh ilmu di universitas ini.
2. Prof. Dr. Wirawan Sumbodo, M.T., selaku Dekan Fakultas Teknik Universitas Negeri Semarang, atas arahan serta motivasi bagi penulis untuk berkembang selama masa studi.
3. Budi Sunarko, S.T., M.T., Ph.D., selaku koordinator Program Studi Pendidikan Teknik Informatika dan Komputer, sekaligus dosen wali, yang senantiasa memberikan bimbingan, wejangan yang mencerahkan, serta pengawasan akademis dari awal perkuliahan hingga selesainya tugas akhir ini.
4. Dr. Djuniadi M.T selaku dosen pembimbing yang telah meluangkan waktu, tenaga, dan pikiran dengan penuh kesabaran dalam memberikan bimbingan, arahan, serta masukan yang sangat berharga sejak awal hingga terselesaikannya skripsi ini.
5. Dr. Nur Iksan, S.T., M.Kom. selaku dosen penguji, atas segala koreksi, kritik konstruktif, serta saran-saran berharga yang diberikan kepada penulis guna penyempurnaan kualitas skripsi ini.
6. Kedua orang tua tercinta, Ibu Titi dan Bapak Asnawi, yang tidak pernah lelah

memberikan doa terbaik, kasih sayang yang tiada tara, serta dukungan moril maupun materiel yang menjadi sumber kekuatan utama bagi penulis.

7. Saudara-saudaraku tersayang, Kakak Annisa dan Adik Annida, yang selalu membagikan keceriaan, semangat, dan dukungan luar biasa dalam setiap langkah penyusunan skripsi ini.
8. Elfa Fadilah atas waktunya yang selalu memberikan dukungan moral dan motivasi kepada penulis dan selalu menemani perjalanan penulis selama penyusunan skripsi ini.
9. Teman-teman seperjuangan di Program Studi Pendidikan Teknik Informatika dan Komputer 2022, khususnya sahabat-sahabat dari Rombel Internasional, yang telah mewarnai masa perkuliahan dengan kebersamaan, kehangatan bertukar pikiran, dan keikhlasan dalam saling membantu.
10. Sahabat-sahabat KKN UNNES Giat 12 Desa Kaloran (Alip, Intan, Ririn, Rahma, Galuh, Icha, Elfa, Reva, Cindy, Selvin, dan Ihsan), yang telah mengukir kenangan indah, memberikan pengalaman berharga, serta menjadi keluarga baru yang senantiasa saling menguatkan.

Penulis berharap semoga skripsi ini dapat memberikan manfaat yang seluas-luasnya, baik bagi pengembangan ilmu pengetahuan, khususnya di bidang pendidikan teknik informatika dan komputer, maupun bagi para pembaca secara umum. Semoga amal kebaikan semua pihak yang telah membantu penyelesaian skripsi ini mendapatkan balasan yang berlipat ganda dari Allah Swt.

Semarang, 8 Juli 2026

Annuruddin Tsanasti Yassar

DAFTAR ISI

PERSETUJUAN PEMBIMBING	i
PENGESAHAN TIM PENGUJI.....	ii
PERNYATAAN	iii
MOTTO DAN PERSEMBAHAN	iv
ABSTRAK	v
ABSTRACT	vii
PRAKATA	ix
DAFTAR ISI	xi
DAFTAR GAMBAR	xiii
DAFTAR TABEL	xiii
BAB 1 PENDAHULUAN.....	1
1.1 Latar Belakang Masalah	1
1.2 Batasan Masalah	6
1.3 Rumusan Masalah.....	8
1.4 Tujuan Penelitian	8
1.5 Manfaat Penelitian	9
1.6 Kebaruan Penelitian.....	10
BAB 2 KAJIAN PUSTAKA.....	11
2.1 Tinjauan Pustaka.....	11
2.2 Penelitian Sebelumnya.....	25
2.3 Landasan Teoretik.....	34
2.4 Kerangka Berpikir.....	45
2.5 Hipotesis Teoretik.....	46
BAB 3 METODE PENELITIAN.....	49
3.1 Pendekatan, Jenis, dan Prosedur Penelitian	49
3.2 Lokasi dan Waktu Penelitian	53
3.3 Subjek Penelitian/Sampel dan Populasi.....	53
3.4 Variabel Penelitian dan Definisi Operasional.....	55
3.5 Hipotesis Fungsional.....	57

3.6 Data dan Sumber Data	58
3.7 Teknik Pengumpulan Data.....	59
3.8 Teknik Keabsahan Data	59
3.9 Teknik Analisis Data	60
BAB 4 HASIL DAN PEMBAHASAN.....	61
4.1 Deskripsi Hasil Pengembangan Sistem (Implementasi).....	61
4.2 Hasil Pengujian Sistem (<i>System Testing</i>)	71
4.3 Pengujian Performa Algoritma Prompt AI	76
4.4 Pembahasan	80
BAB 5 PENUTUP.....	82
5.1 Simpulan	82
5.2 Saran	83
DAFTAR PUSTAKA.....	85
LAMPIRAN-LAMPIRAN	90
Lampiran 1. Biodata Penulis.....	90
Lampiran 2 SK Pembimbing	91
Lampiran 3. SK Penguji.....	92
Lampiran 4. Pengujian Black-Box.....	93

DAFTAR GAMBAR

Gambar 3.1 Diagram Arsitektur Sistem.....	50
Gambar 4.2 Tampilan aktivitas plugin.....	62
Gambar 4.3 Diagram aktivitas guru	64
Gambar 4.4 Form Pembuatan Aktivitas.....	65
Gambar 4.5 Isian Pembuatan Aktifitas	65
Gambar 4.6 Tampilan Guru saat membuat aktifitas	66
Gambar 4.7 Diagram aktivitas siswa	67
Gambar 4.8 Tampilan saat hasil berupa <i>console.log</i>	68
Gambar 4.9 Tampilan Preview saat menggunakan <i>html</i> dan <i>css</i>	68
Gambar 4.10 Pengaturan untuk keamanan sistem	69
Gambar 4.11 Tampilan saat mendapatkan hasil <i>feedback</i> dari AI.....	71
Gambar 4.12 Pengaturan API <i>geminiAI</i> dan pengisian	71
Gambar 4.13 Tampilan pada <i>Dashboard</i> laporan guru per aktivitas	75
Gambar 4.14 Tampilan pada <i>Dashboard</i> laporan guru per <i>course</i>	75
Gambar 4.15 Nilai siswa yang sudah terkoneksi oleh Moodle <i>Gradebook</i>	76

DAFTAR TABEL

Tabel 2.1 Penelitian Sebelumnya.....	26
Tabel 4.2 Ringkasan Hasil Eksperimen A Pengaruh Parameter Generasi.....	77
Tabel 4.3 Ringkasan Hasil Eksperimen B Perbandingan Teknik Prompting	78
Tabel 4.4 Ringkasan Hasil Eksperimen C Studi Ablasi Struktur Prompt.....	79

BAB 1

PENDAHULUAN

1.1 Latar Belakang Masalah

Pendidikan vokasi di Indonesia, khususnya pada jenjang Sekolah Menengah Kejuruan (SMK) Kelas 11 yang menerapkan Kurikulum Merdeka Fase F, menuntut peserta didik untuk memiliki kompetensi teknis yang adaptif terhadap perkembangan industri perangkat lunak. Salah satu kompetensi krusial pada fase ini adalah penguasaan bahasa pemrograman JavaScript Dasar. Namun, dalam proses transisi kognitif dari pemahaman logika dasar menuju sintaksis kode yang kaku, peserta didik sering kali menghadapi hambatan belajar yang signifikan. Hambatan utama bersumber pada keluaran *error* kompilator bawaan dari lingkungan eksekusi seperti Node.js. Pesan *error* teknis berbahasa Inggris, seperti `ReferenceError: totals is not defined`, dirancang untuk pengembang profesional dan tidak memiliki orientasi pedagogis. Bagi peserta didik pemula, pesan yang bersifat absolut dan minim konteks ini gagal menjelaskan letak kesalahan secara spesifik maupun langkah perbaikannya. Analisis forensik terhadap log kompilasi membuktikan bahwa rentetan kegagalan persisten dalam menginterpretasikan pesan *stderr* ini secara kumulatif menyebabkan kelelahan mental, mengerosi efikasi diri (*self-efficacy*) siswa terhadap kemampuan mereka menguasai teknologi, memicu demotivasi akut, dan secara langsung bermuara pada tingginya angka kegagalan evaluasi serta penolakan terhadap karir di bidang TIK di masa depan.

Dalam rangka mereduksi tingkat kegagalan dan menyediakan infrastruktur pembelajaran yang terstandardisasi, institusi pendidikan vokasi dan kementerian terkait telah mendorong adopsi ekosistem pembelajaran digital. Sistem manajemen pembelajaran, khususnya platform berbasis sumber terbuka *Learning Management System* (LMS) Moodle, telah diadopsi secara luas dan menjadi tulang punggung infrastruktur institusional bagi SMK di seluruh penjuru negeri. Keandalan Moodle dalam memajemen distribusi materi kursus, menyelenggarakan asesmen, serta memelihara keberlanjutan arsitektur basis data akademik telah terbukti secara saintifik

efektif dalam memfasilitasi kebutuhan administrasi pendidikan kejuruan yang kompleks, termasuk integrasinya dengan sistem pemeliharaan berkala untuk evaluasi praktikal di SMK (Yudiono, 2024). Akan tetapi, Moodle secara bawaan arsitekturalnya memiliki kelemahan yang sangat fundamental terkait pengajaran pemrograman. Sebagai sistem manajemen konten yang dibangun di atas tumpukan teknologi monolitik PHP dan basis data relasional, Moodle tidak menyediakan modul aktivitas (*Activity Module*) internal yang mampu memfasilitasi aktivitas latihan penulisan kode sumber interaktif secara waktu nyata (*real-time execution*). Moodle tidak dilengkapi dengan *editor* teks sintaksis terintegrasi yang mampu mengidentifikasi kesalahan baris kode, dan Moodle sama sekali tidak menyediakan lingkungan kompilasi atau *sandbox* mandiri yang aman untuk menafsirkan *string* kode JavaScript dari pihak klien (peserta didik).

Sebagai jalan keluar pragmatis dan kompromistis atas kelemahan bawaan tersebut, tenaga pendidik sering kali mengarahkan peserta didik SMK untuk menggunakan platform *Integrated Development Environment* (IDE) berbasis komputasi awan (*cloud*) pihak ketiga secara eksternal, atau memaksa siswa mengunduh dan mengonfigurasi IDE luring yang memakan spesifikasi perangkat keras laboratorium yang besar. Pendekatan desentralisasi ini justru menghadirkan serangkaian anomali pedagogis, kebocoran data, dan defisiensi administratif yang sangat mengancam ekosistem pembelajaran itu sendiri. Pertama, platform eksternal yang diakses siswa beroperasi secara terisolasi dan sama sekali tidak tersinkronisasi dengan antarmuka pemrograman aplikasi (*Application Programming Interface* / API) hierarki nilai Moodle *Gradebook*. Kondisi keterisolasian data ini mendelegitimasi proses pelacakan analitik performa siswa yang seharusnya berjalan otomatis. Instruktur kehilangan visibilitas komprehensif, memutus rekam jejak evolusi perbaikan galat siswa, dan secara sistematis menghalangi guru dalam memetakan profil kemajuan kelas menggunakan teknik segmentasi atau analitik *machine learning* (Astuti et al., 2024). Tanpa visualisasi rekam jejak terintegrasi, guru TIK tidak dapat mengidentifikasi letak hambatan komputasional spesifik yang dialami oleh setiap individu, sebuah parameter

krusial yang terbukti secara empiris menjadi kunci keberhasilan dalam memberikan intervensi akademik berbasis data prediktif (Triyasri et al., 2025).

Kedua, dan yang lebih berbahaya karena secara langsung meretas integritas akademik, adalah eksposur peserta didik pemula terhadap fitur kecerdasan buatan pelengkap kode (*AI Copilots*) yang kini tertanam secara masif dan agresif pada IDE modern eksternal. Perangkat pelengkap otomatis ini memang didesain untuk mempercepat produktivitas *programmer* senior, namun sangat beracun bagi ekologi pendidikan vokasi tahap dasar. Ketika siswa menghadapi kebuntuan sintaksis atau kelumpuhan logis, fitur generatif pada IDE sering kali meretas tahapan esensial dari proses penalaran analitis peserta didik dengan menyuapi blok jawaban utuh secara instan tanpa perlu diminta. Ketergantungan berlebihan terhadap solusi instan ini menciptakan ilusi kompetensi palsu; siswa merasa berhasil mengeksekusi program dan mendapatkan hasil visual yang benar tanpa memiliki struktur metakognisi untuk memahami dekomposisi algoritmanya secara independen. Fenomena ini bisa menghancurkan esensi konstruktivisme dari pembelajaran pemrograman, di mana seorang siswa secara teori seharusnya bergelut secara konstruktif dengan galat untuk membangun skema memori dan ketahanan mental (Liao et al., 2024). Dampak destruktif dari paparan teknologi mesin penjawab otonom ini sangat paralel dengan temuan riset terkait dampak disrupsi fokus akibat intervensi platform eksternal nir-regulasi di luar lingkungan belajar terstruktur, seperti halnya penurunan dramatis luaran performa akademik akibat distraksi teknologi daring yang tidak dipandu secara metodologis (Ibañez, 2023).

Ketimpangan rasio antara pendidik dan peserta didik di kelas praktikum memperburuk situasi ini. Dengan rata-rata satu guru menangani 25 hingga 30 siswa secara serentak, pemberian bimbingan dan umpan balik formatif secara individual dan *real-time* menjadi kemustahilan empiris. Peserta didik yang mengalami kebuntuan sintaksis sering kali harus menunggu lama untuk mendapatkan intervensi guru, yang pada akhirnya menghambat laju pembelajaran. Peserta didik yang terperangkap dalam galat kompilasi yang tidak dipahaminya sering kali harus mengangkat tangan dan

menunggu waktu intervensi instruktur yang sangat lama, memicu terjadinya kebosanan, frustrasi, hingga berujung pada resolusi menyimpang seperti menyalin seluruh kode milik rekan sejawat yang berprestasi tinggi untuk memenuhi tenggat waktu penugasan Moodle.

Di sisi yang lain, apabila institusi vokasi berkeras mengabaikan platform eksternal dan mencoba melakukan modifikasi kasar pada Moodle agar LMS mengeksekusi kode klien secara langsung pada lingkungan proses sistem peladen (*server*) utama berbasis PHP, mereka akan membenturkan institusi pada risiko keamanan siber yang berpotensi fatal. Mengizinkan untaian *string* pemrograman mentah dari peserta didik untuk dijalankan secara *native* membuka berbagai celah vektor kerentanan sistem tingkat kritis. Tanpa isolasi komputasi yang ketat, baris kode peretas sederhana dapat mengeksekusi serangan penetrasi sistem berkas, memanggil instruksi biner tingkat sistem operasi peladen, mengekstrak konfigurasi kredensial basis data Moodle, hingga memicu serangan penolakan layanan internal akibat memori yang habis akibat beban perulangan tak terhingga (*infinite loops*). Serangan atau kelalaian fatal ini mampu melumpuhkan seluruh ekosistem LMS sekolah.

Merespons permasalahan komprehensif pada lapisan pedagogis dan infrastruktur tersebut, komunitas rekayasa perangkat lunak pendidikan global telah berusaha mengembangkan berbagai ragam instrumen yang dikategorisasikan sebagai *Automated Programming Assessment Systems* (APAS). Meskipun demikian, sintesis literatur terkini mengindikasikan adanya kesenjangan fundamental dan paradoks metodologis pada implementasi arsitektur APAS yang tertanam pada LMS konvensional. Sistem penilai otonom tradisional dan *plugin* yang tersedia di pasaran, seperti modul Moodle *CodeRunner* atau *FastTest Plugin*, beroperasi dengan logika evaluasi mekanis yang terlampaui primitif untuk memandu pembelajaran. Sistem tersebut hanya mereduksi beban waktu koreksi administratif dosen melalui mekanisme evaluasi uji unit statis berbasis biner: sekadar menyatakan label Benar atau Salah (*Pass/Fail*) atau persentase kasus uji coba (*test-cases*) yang dilewati. Lebih buruknya lagi, saat kode siswa gagal melewati uji unit, *plugin* tersebut secara harfiah hanya

menggemakan dan memuntahkan ulang pesan galat kompilator mentah (*raw standard error*) dari sistem ke layar monitor peserta didik, tanpa adanya translasi analitik maupun interpretasi pedagogis sedikit pun.

Menyadari keterbatasan pedagogis dari umpan balik mekanis biner, generasi riset APAS dekade ini mulai mengeksplorasi integrasi model kecerdasan buatan, khususnya intervensi berbasis *Large Language Model* (LLM) seperti arsitektur model transformer generatif (ChatGPT, Gemini). Akan tetapi, mayoritas purwarupa sistem APAS berbasis AI yang diimplementasikan sebagai *chatbot* mengambang (*floating agents*) pada antarmuka LMS masih menderita masalah rekayasa arsitektural dan bias instruksional yang sangat serius. Pertama, LLM memiliki kecenderungan bawaan untuk memproduksi halusinasi linguistik, di mana AI menyajikan penjelasan sintaksis yang terdengar sangat meyakinkan namun secara teknis menyesatkan. Kedua, tanpa restriksi struktural sistemik, agen LLM akan bertindak terlalu luwes, memposisikan dirinya sebagai "mesin penjawab pelayan" yang selalu prematur dalam membongkar dan menyodorkan seluruh blok kode solusi secara utuh, semata-mata demi memenuhi kueri pengguna secara cepat (Frankford et al., 2026). Mahasiswa perguruan tinggi atau siswa SMK yang diberikan keistimewaan menerima balasan instan berisi perbaikan kode utuh dari *chatbot* AI memang akan mengalami percepatan waktu resolusi penyelesaian tugas secara drastis dalam uji coba eksperimental (Alshammari, 2025). Namun, dalam pengujian longitudinal dan transferabilitas pengetahuan (*knowledge transfer*), siswa-siswa ini terbukti mengalami atrofi (kemunduran) fatal dalam kemampuan dekomposisi algoritmik mandiri karena proses sintesis logisnya sepenuhnya diambil alih oleh mesin (Ma et al., 2025).

Untuk memecahkan kebuntuan pedagogis dan infrastruktur tersebut, penelitian rekayasa perangkat lunak ini diinisiasi untuk merancang dan membangun *plugin Activity Module Moodle* bernama Aicode. Sistem ini menghadirkan *co-tutor* kecerdasan buatan yang beroperasi dengan pendekatan pedagogi Sokratik dan prinsip *scaffolding*. Melalui integrasi *Application Programming Interface* (API) Google Gemini, modul analisis pada Aicode tidak sekadar menerjemahkan *stderr* ke dalam

bahasa Indonesia baku, melainkan membedah akar permasalahan logis dan memberikan petunjuk. Berbekal aturan anti-spoiler yang ditanamkan melalui penyesuaian *prompt*, kecerdasan buatan secara ketat dilarang memberikan blok kode jawaban utuh, melainkan mendorong kemampuan pemecahan masalah peserta didik melalui panduan langkah demi langkah yang suportif. Lebih lanjut, seluruh eksekusi kode dialihkan ke lingkungan *sandbox* proksi asinkron (Node.js) guna mengeliminasi risiko peretasan peladen Moodle utama, memastikan ekosistem evaluasi yang aman, cerdas, dan tersentralisasi.

Integrasi kecerdasan buatan, khususnya LLM seperti Google Gemini, ke dalam sistem manajemen pembelajaran Moodle membawa tantangan teknis dan pedagogis tersendiri. Berbeda dengan asisten obrolan (*chatbot*) konvensional yang mentoleransi keluaran teks bebas, agen AI dalam modul *mod_aicode* diwajibkan untuk mengembalikan analisis diagnosis secara eksklusif dalam struktur data JSON yang ketat. Kegagalan format respons dari API, seperti penambahan teks sisa atau rusaknya sintaks JSON akan langsung menyebabkan malfungsi sistem sehingga gagal memberikan umpan balik kepada siswa. Oleh karena itu, penelitian pengembangan ini tidak dapat hanya berhenti pada integrasi antarmuka Moodle saja, melainkan wajib melibatkan fase optimasi dan evaluasi empiris pada tahap rekayasa instruksi AI (*prompt engineering*). Melalui pendekatan komparatif terhadap metode *Zero-Shot*, *Few-Shot*, dan *Role-Prompting*, sistem dituntut untuk mencapai stabilitas fungsional (*Parsing Success Rate* yang tinggi) sekaligus menggaransi kepatuhan Sokratik, yakni AI harus membimbing tanpa memberikan jawaban instan sebelum sistem dinilai layak dan aman digunakan dalam skenario kelas nyata.

1.2 Batasan Masalah

Guna memastikan kedalaman, ketajaman analisis, serta kesesuaian dengan domain ilmu Rekayasa Perangkat Lunak (*Software Engineering*), parameter dan ruang lingkup pengembangan sistem ini dibatasi secara ketat pada aspek-aspek arsitektural dan fungsional teknis. Fokus penelitian secara mutlak mengeliminasi variabel pengujian psikologis, pengukuran persepsi pengguna, maupun evaluasi hasil belajar

pedagogis. Penjabaran batasan sistem ditetapkan sebagai berikut:

Pertama, metodologi pengembangan perangkat lunak secara eksklusif menggunakan kerangka kerja *Software Development Life Cycle* (SDLC) model Waterfall. Pendekatan sekuensial ini dipilih untuk memastikan spesifikasi arsitektur API dan skema basis data XMLDB Moodle terkunci secara stabil sejak fase analisis kebutuhan.

Kedua, ruang lingkup teknis *plugin* tipe *Activity Module* dibatasi untuk mengevaluasi materi bahasa pemrograman JavaScript Dasar. Sistem ini dikompilasi secara spesifik agar kompatibel dengan arsitektur Moodle versi 5.0+, berjalan pada lingkungan eksekusi PHP 8.1+ dan basis data MySQL 8.0+, Modul asinkron akan diisolasi menggunakan *runtime* Node.js yang ditambahkan pada port 3001.

Ketiga, skema pengujian dan validasi perangkat lunak dibatasi hanya pada tiga metrik operasional keteknikan. Metrik pertama adalah Uji Fungsionalitas (*Black-Box Testing*) yang dieksekusi terhadap 40 skenario pengujian untuk memvalidasi respons sistem atas berbagai kombinasi masukan. Metrik kedua adalah Uji *Usability* yang secara ketat menggunakan metode *User Interface (UI) Walk-Through Scenario Testing*. Pengujian ini semata-mata dilakukan untuk memverifikasi bahwa interaksi antarmuka peserta didik (seperti penekanan tombol eksekusi dan analisis) berhasil melakukan perekaman jejak data yang presisi pada tabel `aicode_attempts`, yang kemudian terproyeksi secara akurat pada visualisasi analitik di *dashboard* pengajar, serta Menguji ketahanan keamanan *sandbox* Node.js dari injeksi *payload* berbahaya. Pengujian ini dilarang menggunakan instrumen kuesioner seperti *System Usability Scale* (SUS) atau pengukuran efektivitas pembelajaran siswa. Metrik ketiga Adalah pengujian *prompt* untuk menguji akurasi integrasi *prompt engineering* API Gemini. Pengujian dan optimasi *prompt engineering* dalam sistem ini difokuskan dan diikat secara eksklusif pada pemanfaatan model bahasa Gemini 2.5 Flash melalui penyedia layanan *Google API*. Dataset skenario pengujian komputasional algoritma *prompt* AI dibatasi pada kasus-kasus galat pemrograman tingkat dasar (Sintaksis, *Runtime*, dan Logika) pada bahasa pemrograman JavaScript yang mengacu pada silabus pendidikan kelas vokasi.

1.3 Rumusan Masalah

Berdasarkan tinjauan pustaka dan gap riset yang diidentifikasi, pertanyaan penelitian yang diangkat meliputi:

1. Bagaimana merancang bangun arsitektur *plugin* Moodle Aicode yang mengintegrasikan secara asinkron komponen *Editor* berbasis modul JavaScript, proksi *Backend Executor*, dan modul *Analyzer* kecerdasan buatan berbasis Google Gemini API?
2. Bagaimana memodelkan dan mengimplementasikan mekanisme pertahanan keamanan eksekusi kode (*sandbox isolation*) di lingkungan sistem?
3. Bagaimana memvalidasi keandalan integrasi sistem aliran data melalui metode *UI Walk-through Scenario Testing* guna memastikan aktivitas kompilasi dan intervensi AI terekam secara otentik pada tabel basis data *aicode_attempts* dan tervisualisasi dengan presisi pada *dashboard* analitik instruktur?

1.4 Tujuan Penelitian

Sejalan dengan rumusan masalah rekayasa di atas, penelitian pengembangan sistem ini difokuskan untuk mencapai target-target teknis sebagai berikut:

1.4.1 Tujuan Umum

Mengembangkan suatu fitur untuk latihan pemrograman interaktif pada Moodle yang menyediakan umpan balik cerdas berbasis LLM yang diharapkan bisa membantu siswa agar lebih mudah dalam berlatih untuk memahami materi pemrograman javascript.

1.4.2 Tujuan Khusus

1. Menghasilkan spesifikasi teknis dan piranti lunak *plugin Activity Module* Moodle (Aicode) yang mengimplementasikan *Web Service Subsystem* sebagai jembatan komunikasi data yang mulus antara antarmuka pengguna interaktif, mesin eksekusi lokal, dan layanan kecerdasan buatan eksternal.
2. Membangun lapisan keamanan *runtime execution* melalui layanan proksi Node.js yang mampu mengisolasi eksekusi kode secara asinkron, memastikan keamanan

infrastruktur utama Moodle dari eksploitasi, peretasan berkas sistem, dan beban komputasi tak terbatas.

3. Membuktikan stabilitas fungsionalitas dan integritas aliran data perangkat lunak melalui pengujian *Black-Box Testing* serta penelusuran antarmuka, guna memastikan visualisasi matriks progres siswa di *dashboard* pengajar merepresentasikan transaksi pangkalan data secara akurat tanpa anomali pencatatan.

1.5 Manfaat Penelitian

Pengembangan piranti lunak Aicode ini diharapkan dapat memberikan implikasi dan manfaat yang signifikan pada dua dimensi keilmuan:

1.5.1 Manfaat Teoretis

Secara teoretis, penelitian ini mengembangkan arsitektur rekayasa perangkat lunak dalam domain *Automated Programming Assessment System* (APAS) yang tertanam pada ekosistem Moodle. Rancang sistem ini membuktikan kelayakan model dekopling *sandbox* asinkron yang digabungkan dengan manipulasi *hyperparameter* LLM. Model implementasi *array_slice* pada bahasa PHP untuk membatasi jumlah *hint* dan konfigurasi heuristik API menawarkan fondasi teoretis baru tentang bagaimana kecerdasan buatan generatif dapat dikerangka untuk menghasilkan pedagogi Sokratik yang presisi, deterministik, dan terhindar dari bias halusinasi.

1.5.2 Manfaat Praktis

Pada tataran operasional dan praktis, produk rekayasa ini memberikan solusi infrastruktur bagi institusi vokasi dalam menyelenggarakan evaluasi pemrograman yang tersentralisasi dan aman. Bagi tenaga pendidik, sistem ini mengotomatisasi evaluasi dengan menyediakan *dashboard* analitik yang memetakan jenis *error* yang paling dominan di kelas, tersinkronisasi langsung dengan Moodle *Gradebook*, tanpa memerlukan instalasi IDE pihak ketiga di laboratorium. Bagi peserta didik, antarmuka Aicode berfungsi sebagai asisten *co-tutor* yang beroperasi 24 jam; mendobrak isolasi bahasa dari pesan *error* kompilator, mengurangi tingkat frustrasi melalui panduan

scaffolding berbahasa Indonesia, dan menjaga integritas evaluasi karena sistem tidak pernah meretas proses belajar dengan memberikan blok jawaban utuh.

1.6 Kebaruan Penelitian

Signifikansi kebaruan (*novelty*) dari penelitian rekayasa *plugin* Aicode ini berfokus pada perancangan arsitektur spesifik yang mengatasi titik-titik lemah modul pembelajaran konvensional:

1. Integrasi Eksekusi Sandbox Asinkron (*Asynchronous Sandbox Intergrasion*):

Berbeda dengan pendekatan tradisional APAS yang mengeksekusi urutan kode pada peladen PHP secara monolitik, arsitektur Aicode merelokasi beban kompilasi *string* JavaScript menuju modul eksekutor Node.js independen. Desain ini secara inheren menciptakan benteng isolasi yang memblokir akses manipulasi sistem berkas host, serta mengunci siklus hidup proses melalui limitasi beban 50.000 bita (50KB) dan tenggat waktu absolut 2 detik.

2. **Mekanisme *Mock Polyfill* Modifikasi DOM Dinamis:** Komponen presentasi *frontend* (amd/src/editor.js) ditanamkan arsitektur *sandbox preview* interaktif yang mengimplementasikan sistem *polyfill* otomatis. Ketika peserta didik memanipulasi *Document Object Model* (DOM) yang tidak terdapat pada templat statis HTML *read-only* buatan guru, antarmuka melakukan intersepsi pada perintah seperti `document.getElementById` untuk merender elemen bayangan secara dinamis, sehingga mencegah jendela pratinjau klien dari status *error* total (*browser crashing*) dan mendukung pembelajaran bertahap.

3. **Siklus Sokratik Anti-Spoiler melalui Restriksi API LLM (*Anti-Spoiler Socratic Loop*):** Sistem tidak membiarkan LLM berimajinasi secara bebas. Setelah balasan JSON diterima peladen, fungsionalitas inti Moodle secara otonom memanipulasi struktur data menggunakan utilitas `array_slice` untuk mengebiri jumlah luaran *hint*. Metode ini memastikan bahwa asisten AI beroperasi murni sebagai fasilitator *scaffolding* yang mendiagnosis keluaran *stderr* menjadi petunjuk kontekstual berbahasa Indonesia, mengeliminasi risiko pemberian bocoran sintaksis utuh (*auto-complete spoilers*) kepada siswa.

BAB 2

KAJIAN PUSTAKA

2.1 Tinjauan Pustaka

Pengembangan infrastruktur pendidikan digital pada jenjang kejuruan vokasional menuntut adanya transformasi fundamental dari evaluasi rekayasa perangkat lunak yang bersifat pasif menuju intervensi sistem cerdas yang bersifat aktif dan adaptif. Analisis literatur ini dieksekusi secara mendalam untuk mengidentifikasi kesenjangan (*gap*) pada arsitektur APAS generasi lampau, sekaligus membangun justifikasi keteknikan dan pedagogis yang absolut bagi rancang bangun arsitektur *plugin* aicode di dalam ekosistem Moodle.

2.1.1 Otomatisasi Penilaian Pemrograman dan Integrasi Moodle

Keterbatasan sistem pengujian kode ortodoks diidentifikasi melalui tinjauan literatur sistematis yang dieksekusi oleh (Livaja Mušac et al., 2026) yang membedah 39 studi primer terkait alat evaluasi pemrograman. Inti permasalahan dari penelitian ini adalah bahwa metode evaluasi berbasis kompilasi tradisional (*test-based grading/dynamic analysis*) hanya menghasilkan keluaran *pass/fail* biner yang sangat tidak memadai untuk menangani kompleksitas beban kognitif peserta didik pemula. Metodologi ekstraksi data yang mereka gunakan menemukan pergeseran tren yang masif dari evaluasi murni berbasis eksekusi menuju pemanfaatan arsitektur *Deep Learning* dan *Large Language Models* (LLM). Temuan kunci mengonfirmasi bahwa APAS kini bermutasi menjadi instrumen pendukung pendidikan cerdas, walau tantangan terkait keandalan validasi di dalam LMS masih menjadi celah riset. Kajian ini memiliki relevansi langsung dengan Aicode karena memvalidasi secara empiris bahwa intervensi kecerdasan buatan bukanlah sekadar pelengkap kosmetik, melainkan fondasi wajib untuk menerjemahkan keluaran *standard error* (*stderr*) yang kaku menjadi diagnosis yang koheren bagi siswa vokasi.

Tantangan terkait kebergantungan mahasiswa secara berlebihan (*over-reliance*) terhadap alat kecerdasan buatan generatif diteliti oleh (Frankford et al., 2026) dalam konteks asesmen pemahaman kode. Problematika utamanya adalah model bahasa besar

sering merusak evaluasi dengan menyuapkan solusi akhir tanpa menuntut rasionalisasi dari pelajar. Untuk mengatasi ini, mereka merancang metodologi *Hybrid Socratic Framework* yang menjembatani keandalan analisis kompilasi deterministik dengan lapisan kelancaran percakapan LLM. Hasil purwarupa membuktikan bahwa penerapan metode Sokratis memaksa pelajar untuk mengartikulasikan penalaran menengah (*intermediate reasoning*) secara dialektik, yang secara signifikan mencegah halusinasi *chatbot* dan kebocoran jawaban. Rancang bangun arsitektur ini secara substansial melandasi keputusan *prompt engineering* pada Aicode, di mana API Google Gemini dilarang memberikan resolusi blok kode utuh dan diinstruksikan murni untuk mendiagnosis baris *error* menjadi petunjuk terpadu.

Fragmentasi ekosistem antara lingkungan pengembangan eksternal (*Integrated Development Environment/IDE*) dan platform pelacakan pembelajaran diinvestigasi oleh (Zhu & Liu, 2025). Problem sentralnya adalah bahwa interaksi mahasiswa yang terpecah menghalangi instruktur dalam memberikan dukungan belajar berskala besar yang peka konteks (*context-aware*). Menggunakan metode integrasi *plugin* eksekusi berbasis Moodle (*CodeRunner*), mereka menciptakan *CodeRunner Agent* bertenaga LLM yang langsung menarik parameter jejak belajar dari *Learning Record Store* (LRS) menggunakan statemen xAPI. Hasil eksperimen memvalidasi bahwa penggabungan eksekusi peladen lokal dengan API *feedback* AI memperkuat strategi regulasi mandiri (*self-regulated learning*) karena bantuan teknis diberikan seketika di layar yang sama. Penelitian ini memberikan dasar pijakan arsitektural yang fundamental bagi Aicode untuk mengunci seluruh proses *coding*, kompilasi *sandbox*, dan kueri Gemini di dalam satu ekosistem antarmuka (*Client Editor* AMD JavaScript) Moodle, memastikan sinkronisasi absolut dengan pangkalan data *Gradebook*.

Kerentanan infrastruktur jaringan dari eksploitasi eksekusi kode klien dianalisis secara arsitektural oleh (Nwohiri et al., 2025). Permasalahan intinya adalah bahwa mengeksekusi urutan logika buatan pengguna (*untrusted code*) di peladen utama berpotensi memicu kerusakan sistem file host secara fatal. Metodologi rekayasa mereka membedah APAS multi-bahasa ke dalam kerangka modular: modul pengiriman *frontend*, mesin evaluasi *backend*, lapisan keamanan isolasi (*sandboxing*

layer), dan modul log pangkalan data. Implementasi kerangka ini menjamin fleksibilitas sekaligus mengunci siklus reproduksi keamanan sehingga *host* kebal dari injeksi. Prinsip pertahanan *sandbox* ini sangat relevan dengan modul Aicode, menjustifikasi pengalihan lalu lintas kompilasi asinkron (*JavaScript execution string*) menuju proksi *runtime* Node.js tersendiri pada porta 3001, lengkap dengan benteng pertahanan ukuran berkas 50KB dan pemutus waktu (*timeout*) 2 detik.

Keefektifan bantuan komputasional berbasis *chatbot* generatif diukur oleh (Liao et al., 2024) yang mengkaji secara langsung percepatan kurva *Computational Thinking* (CT) bagi mahasiswa pemrograman tingkat awal. Menghadapi masalah frustrasi akibat *error* algoritma yang terus mendera mahasiswa, peneliti menyusun purwarupa *Intelligent Programming Scaffolding System with ChatGPT* (IPSSC) menggunakan studi metode campuran (*mixed-methods*). Hasil analisis empiris menegaskan bahwa intervensi perancah (*scaffolding*) AI tidak mereduksi kecerdasan, melainkan mengakselerasi siklus pembuatan purwarupa dan revisi *error* (*debugging*) secara signifikan, yang pada akhirnya mendongkrak efikasi diri (*self-efficacy*) mahasiswa. Fakta bahwa GenAI menumbuhkan kemandirian melalui *mastery experiences* ini meneguhkan posisi Aicode sebagai asisten formatif yang meredam stres kognitif dengan menerjemahkan baris *stderr* menjadi *scaffolding* berbahasa Indonesia.

Kebutuhan pencatatan presisi dalam integrasi asisten kecerdasan buatan dieksplorasi oleh (Alshammari, 2025) pada evaluasi instruksional proyek pemrograman. Inti studi ini merespons keluhan institusi terhadap masifnya beban evaluasi proyek pemrograman yang tidakimbang dengan ketersediaan instruktur. Menggunakan metodologi investigatif terhadap luaran instruksional berbasis ChatGPT, hasil riset menemukan bahwa integrasi API kecerdasan buatan mutlak mengurangi durasi koreksi manual dan menghemat sumber daya asisten laboratorium, dengan prasyarat bahwa log transaksi harus dipantau guna menghindari penyimpangan analitik. Konklusi ini sangat fundamental bagi arsitektur Aicode, menjustifikasi keharusan rancangan pangkalan data XMLDB Moodle untuk merekam secara otentik seluruh iterasi kompilasi dan interupsi AI ke dalam tabel `aicode_attempts`,

mempertahankan transparansi penilaian berbasis *human-in-the-loop* bagi instruktur kelas.

Ketahanan *backend* terhadap repetisi evaluasi massal dibedah oleh (Jocovic et al., 2023) dalam rancang bangun sistem penilaian ujian otomatis berbasis pindaian perangkat keras. Masalah yang dikemukakan adalah inefisiensi luar biasa pada evaluasi formatif konvensional skala besar di Fakultas Teknik Komputer. Menerapkan rekayasa komputasi pemelajaran mesin terhadap koleksi data sejumlah 734.825 sampel jawaban *multiple-choice*, sistem tersebut mendemonstrasikan keandalan luar biasa dengan tingkat akurasi pengenalan sukses mencapai 99.89% tanpa mengalami kegagalan beban. Keberhasilan arsitektur mesin ini untuk menahan rentetan ribuan instruksi validasi asinkron secara paralel menjadi argumen teknis kuat bagi Aicode, yang dirancang melayani permintaan *live compiler testing* massal per detik dari ruang praktikum SMK tanpa memicu latensi atau kelumpuhan peladen LMS utama.

Adaptasi fungsional elemen peranti lunak di dalam ekosistem Moodle itu sendiri dikaji oleh (Huerta-Gomez-Merodio & Requena-Garcia-Cruz, 2024) yang mengevaluasi efektivitas pemanfaatan ekstensi pengujian. Persoalan yang diangkat adalah tingginya hambatan administratif bagi tenaga pendidik saat harus merekayasa basis kueri format XML Moodle yang rumit secara manual. Melalui studi deskriptif komparatif terhadap pemakaian *plugin FastTest* (FTP), ditemukan bahwa fungsionalitas otomasi secara signifikan meningkatkan diversifikasi evaluasi pengajar (menaikkan tipe soal menjodohkan 35% dan rumpang 22%) karena mereka terbebas dari kendala teknis penulisan tag XML. Temuan ini mendemonstrasikan bahwa integrasi *plugin Activity Module* seperti Aicode tidak hanya menyodorkan teknologi baru, tetapi bermutasi menjadi fasilitator transisi pedagogis di mana guru SMK tak lagi direpotkan oleh koreksi *bug* satu per satu, melainkan beralih fokus pada rekapitulasi capaian logika siswanya.

2.1.2 Scaffolding Socratic dan Rekayasa Parameter LLM

Untuk membongkar paradigma mesin LLM sebagai penghasil bocoran jawaban instan, (Ma et al., 2025) meneliti fungsionalitas kerangka sistem DBox dalam

pendidikan algoritma pemrograman. Menyadari bahwa penyelesaian instan merusak kapasitas sintesis pelajar, peneliti mendukung pendekatan eksperimental "Learner-LLM Co-Decomposition", sebuah metode yang memaksa pelajar dan AI untuk secara bersama-sama mengurai permasalahan ke dalam pohon langkah algoritmik yang terpersonalisasi. Studi yang diterbitkan di forum CHI 2025 ini membuktikan bahwa metode kolaboratif tersebut menurunkan beban kognitif serta memberikan sense of achievement (rasa pencapaian) yang nyata bagi para pelajar. Konsep dekomposisi hierarkis ini diaplikasikan dalam arsitektur Aicode di mana subsistem pembungkus eksternal Moodle merekayasa objek JSON menggunakan instruksi `array_slice` untuk secara deterministik memangkas saran AI menjadi maksimal tiga derajat petunjuk (mulai dari peringatan ringan hingga identifikasi spesifik baris) sehingga mencegah delegasi tugas penyelesaian seutuhnya kepada mesin.

Pengaruh rekayasa instruksi (*prompt engineering*) terhadap perilaku asisten bahasa buatan diinvestigasi secara arsitektural oleh (Chidambaram et al., 2024) melalui kerangka *Socratic Human Feedback* (SoHF). Inti problematika dari evaluasi model prapelatihan konvensional adalah kecenderungan AI berhalusinasi menyusun kode tak masuk akal bila tidak diarahkan. Memanfaatkan metodologi komparatif antara generasi bebas dan strategi kemudi pakar (*expert steering strategies*), penelitian ini menemukan bahwa pemaksaan tahapan Sokratik (melalui pertanyaan klarifikasi, perencanaan deskriptif, dan dialektika pertentangan) meningkatkan akurasi resolusi model secara drastis. Rancang bangun *expert steering* tersebut diimplementasikan secara absolut di dalam *lifecycle* Moodle Aicode, secara spesifik disuntikkan ke dalam deklarasi modul `ai_prompt.php` yang mengatur parameter *personality* Gemini untuk berfungsi murni sebagai fasilitator diagnostik tanpa pernah menawarkan revisi kode utuh (*strict zero-spoiler constraint*).

Dampak metakognitif jangka panjang dari kebergantungan perangkat komputasi dianalisis oleh (Tsakeni et al., 2025) menggunakan metodologi tinjauan bibliometrik sistematis selama periode dua puluh tahun (2005–2025) pada ruang lingkup pendidikan STEM. Menghadapi potensi hilangnya keterampilan berpikir fundamental akibat delegasi penalaran kepada alat (*cognitive over-reliance*), para

peneliti mensintesis 135 publikasi akademik dan merumuskan bahwa intervensi AI harus beroperasi sebagai skema *co-regulation* yang dilengkapi dengan ambang batas pelepasan bantuan (*fading support*). Temuan teoretis atas pentingnya batas limitasi kognitif ini diserap oleh spesifikasi arsitektur Aicode melalui rekayasa dua lapis: pemotongan balasan JSON menjadi struktur tiga *hint* saja, serta mekanisme pembatasan tingkat penggunaan (*rate limiting*) yang mengunci fasilitas bedah *error* AI di angka maksimum 50 kueri per peserta didik per hari, menjaga stimulus rasionalitas tanpa mengundang kebergantungan mesin permanen.

Kecenderungan afektif dan persepsi kognitif mahasiswa terhadap balasan generatif dievaluasi secara kuantitatif-kualitatif oleh (Zhang et al., 2024) pada kelas fondasi ilmu komputer (CS1). Merespons rendahnya kualitas umpan balik statis yang dikeluarkan kompilator, studi membedah respons mahasiswa dengan menggunakan teknik RAG (*Retrieval Augmented Generation*) kustom yang diukur melalui analisis tematik. Hasilnya menunjukkan preferensi krusial di mana mahasiswa merasa umpan balik jauh lebih formatif, spesifik, dan presisi koreksinya ketika balasan dari sistem mencakup dan berlandaskan pada salinan sumber kode mentah (*student's source code*) mereka sendiri di dalam parameter *prompt*. Justifikasi integrasi referensi statis kode ini menjadi landasan mengapa fungsi JavaScript asinkron klien (*editor.js*) di Aicode diprogram mengirimkan *payload* bervolume penuh yang berisi gabungan teks *editor*, sisa templat HTML, serta keluaran murni *stderr* proksi Node.js secara utuh ke peladen Google Gemini untuk menjamin akurasi pedoman Sokratik yang *context-aware*.

Faktor kegagalan retensi pada metode daring massal (MOOC) dimitigasi oleh Bachiri, Mouncif, dan Bouikhalene (2023) melalui pengintegrasian kecerdasan buatan untuk gamifikasi. Menghadapi atrisi pelajar daring yang tajam, penelitian tersebut mendesain *plugin Automatic Question Generation* (AQG) di Moodle dan mengujinya pada 100 subjek kognitif. Temuan mengesahkan bahwa elemen gamifikasi berbasis *trial-and-error* otonom memantik peningkatan retensi hasil belajar secara masif (dari 48% ke 84%) karena simulasi lingkungan mengurangi tekanan hukuman akademis dan meningkatkan memori ingatan. Filosofi simulasi berulang ini diekstraksi ke dalam rekayasa *Mock Polyfill* modifikasi dinamis DOM antarmuka Aicode, saat pelajar

didorong menyuntikkan perintah variabel JavaScript (misal *document.getElementById*) dan mengeksekusinya berulang-ulang dalam modul *sandbox preview browser* yang aman, mengakselerasi pemahaman sintaksis melalui retensi eksperimental.

Relevansi variabel psikologi kemandirian (*self-efficacy*) dalam koding dianalisis oleh (Yilmaz & Yilmaz, 2023) pada integrasi instrumen *Generative Pre-trained Transformer* (ChatGPT). Problematika sentralnya bertumpu pada rendahnya nalar *computational thinking* (CT) siswa saat mereka menyerah menghadapi isolasi bahasa teknis (*standard error reference*). Melalui uji coba kuasi eksperimen, luaran pengujian membuktikan bahwa panduan berbasis kecerdasan buatan generatif menstimulasi lonjakan signifikan pada keterampilan CT dan efikasi pemrograman karena mesin memandu penyelesaian secara logis. Pembuktian teoretis ini memandu arah terjemahan diagnostik Aicode: mengubah struktur pesan *error* teknis *Node.js* (seperti *ReferenceError*) yang murni bahasa Inggris kaku menjadi penuturan logis berbahasa Indonesia baku yang ramah, sehingga rintangan linguistik dan teknis dapat dijumpai demi membangun determinasi belajar peserta didik SMK.

Protokol pengetatan luaran API untuk keperluan evaluasi deterministik divalidasi oleh (Peacock et al., 2023) melalui tinjauan praktik terbaik implementasi ChatGPT pada kurikulum pendidikan presisi tinggi (medis). Menanggulangi kekhawatiran absolut terhadap efek halusinasi informasi AI generatif, peneliti mendemonstrasikan bahwa manipulasi kreatif dan konstruksi perintah asisten yang ketat (*careful construction of prompts*) mampu secara akurat membimbing AI memformulasikan rancangan kasus dan rubrik penilaian tanpa penyimpangan logis. Rangkaian paksaan instruksional model AI ini direplikasi oleh peranti lunak Aicode dengan cara membekukan rentang derajat imajinasi parameter API (mematok *temperature* ke konfigurasi sangat rendah 0.2, batas toleransi keragaman *topP* 0.8, serta *maxOutputTokens* 2048), mengamankan format transmisi *JavaScript Object Notation* (JSON) yang terstruktur dan mustahil cacat saat diparsing oleh peladen Moodle berbasis PHP.

Ketidakseimbangan rasio manfaat AI antar klaster profil kompetensi

diidentifikasi (Cheng et al., 2023) dengan membedah respons peserta *novice* (pemula) berhadapan dengan *experienced learners* saat terpapar instrumen APAMP. Menangani tantangan anomali *Automated Programming Assessment System*, studi menemukan bahwa meskipun program umpan balik *real-time* sangat mendorong capaian pelajar mahir, pelajar yang berstatus inisiasi (*novice*) justru menderita penurunan drastis pada sikap dan motivasi belajarnya akibat hantaman beban kognitif analisis kesalahan kode yang kelewat mendetail. Identifikasi defisiensi demotivasi (*overwhelming feedback load*) tersebut mendasari limitasi mutlak parameter operasi Aicode agar arsitekturnya difokuskan murni kepada penanganan konsep *Basic JavaScript* Fase F SMK, serta direkayasa khusus supaya AI hanya menguraikan langkah-langkah dasar algoritmik sehingga tidak mengintimidasi kurva kognitif *novice programmers* dengan teori komputasional ekstrem.

2.1.3 Pengembangan LMS dan Sistem Pemrograman Vokasi di Indonesia

Evaluasi keandalan fungsional operasional Moodle dalam arena kejuruan praktis SMK dibuktikan secara komprehensif oleh (Yudiono, 2024) yang mendalami kompetensi sistem injeksi (EFI). Menyikapi rendahnya penguasaan kompetensi teknik di masa adaptasi, penelitian ini menguji fungsionalitas Moodle sebagai penopang aktivitas belajar mandiri. Temuan statistik menggunakan metrik pengukuran empiris memvalidasi pencapaian penguasaan (*normalized gain/N-Gain*) yang stabil pada kategori tinggi di indeks 0.702. Signifikansi pencapaian operasional LMS tersebut memberikan landasan keandalan arsitektural di mana pemilihan kerangka Moodle versi 5.0+ berbasis eksekusi peladen PHP sebagai *framework* rumah (*host engine*) pengembangan *plugin* Aicode merupakan langkah rekayasa yang sangat mapan, terukur stabilitas sistem basis datanya, dan sesuai dengan visi regulasi infrastruktur digital vokasi SMK di Indonesia.

Dalam area ketangguhan pemrosesan rekam aktivitas masif (*analytics and database robustness*), kolaborasi (Santoso et al., 2024) dari ranah akademisi Universitas Negeri Semarang (UNNES) membedah integrasi pembelajaran mesin dengan teknik penelusuran arsitektur basis data pelacakan presensi. Menyelesaikan

tantangan ledakan volume observasi log aktivitas di portal informasi, analisis tren ini menjustifikasi bahwa arsitektur peladen rasional tangguh dalam menyerap *data array vectors* besar secara paralel dan mengabstraksikannya menjadi pola visual. Kinerja stabilitas manajerial struktur data masif ini memberikan validasi rancang teoretis (*proof of concept*) bahwa fondasi Moodle XMLDB sanggup menangani rekaman jejak interaktif *dashboard instructor analytics* Aicode, menampung puluhan entitas perekaman `aicode_attempts` sinkronus yang diluncurkan tiap siswa dari lab pemrograman tanpa mendestruksi limit memori peladen.

Menilik resiliensi pemrosesan layanan data *string* asinkron, (Astuti et al., 2024) menelaah parameter komparatif klasifikasi teks (*spam detection*) berbasis pesan elektronik melalui eksperimentasi metodologi varian algoritma mesin pembelajaran seperti *Random Forest* dan *XGBoost Classifier*. Memecahkan masalah defisiensi akurasi model prediktif, uji eksperimen memvalidasi superioritas algoritma terstruktur dengan skor akurasi puncak mencapai 97.28% presisi operasional. Meskipun dititikberatkan pada ranah model teoretis pembelajaran, pembuktian eksekusi kalkulasi beban matriks manipulasi dimensi *string* di sistem komputasi ini mempertegas ketangguhan rancangan layanan *backend executor proxy API Node.js* (porta 3001) Aicode dalam mengonversi, menafsir, dan merespons luapan teks blok skrip *standard error* siswa yang rawan ancaman *error* perulangan tak hingga (*infinite loop timeout crash*) secara aman.

Inovasi rekayasa antarmuka (*frontend interface*) pedagogis di lingkup universitas dieksplorasi secara empiris oleh (Hasanah et al., 2022) pada rancang metodologi kurikulum Pemrograman Visual. Membongkar masalah penurunan daya analisis kreatif mahasiswa pasca-pandemi, peneliti merekayasa intervensi modul interaktif komprehensif berstandar proyek otonom (*Project Based Learning*). Temuan praktis lapangan mendemonstrasikan bahwa membekali antarmuka aplikasi dengan fleksibilitas praktik mandiri berhasil memantik kembali nalar pemecahan rekayasa logis komputasi siswa. Elemen perancangan instruksional ini diserap seutuhnya ke dalam fitur Aicode melalui rekayasa penyisipan antarmuka kerangka templat luaran (*Read-Only HTML Template Input*), mereduksi distraksi komponen desain grafis

antarmuka sehingga pemelajar SMK Fase F benar-benar hanya dituntut mengeksekusi intervensi pada ruang lingkup baris algoritma *JavaScript array* maupun manipulasi sintaksis logika di penyunting klien.

Kesesuaian dinamika arsitektur web dengan transmisi peningkatan wawasan kognitif divalidasi oleh (Oktaviani et al., 2026) melalui pengembangan aplikasi interaktif *Eco-Explorer Game*. Menanggulangi disparitas pemahaman kerangka berkelanjutan SDGs, riset mendayagunakan metode *pre-experimental one-group pretest-posttest* berbantu instrumen pengembangan model Lee dan Owens terhadap elemen *website HTML* dinamis. Hasil integrasi N-gain mengesahkan fleksibilitas manipulasi elemen media web memperkuat akselerasi pemahaman dimensi operasional belajar otonom siswa secara solid. Aspek rekayasa kelincahan manipulasi antarmuka klien ini direfleksikan Aicode via pemanfaatan *JavaScript Asynchronous Module Definition (AMD)* di direktori peladen Moodle klien, yang mana subsistem secara langsung merender manipulasi pohon DOM yang dititipkan JavaScript pelajar menuju elemen *Shadow Output Preview* pada peramban secara absolut *real-time*.

Dampak latensi fokus pembelajaran dalam ekosistem digital majemuk diinvestigasi dari sudut pandang psikometri navigasi oleh (Ibañez, 2023) melalui tinjauan kebiasaan pelajar di luar kerangka platform akademis. Merespons pergerakan navigasi pelajar DEBESMSCAT yang liar ke situs proksi luar, riset kuantitatif menunjukkan tingginya koefisien distraksi komputasi akibat aplikasi sekunder di luar gerbang otentikasi kurikulum, mencederai objektivitas konsistensi ketuntasan penugasan. Konstruksi defisiensi perilaku digital eksternal ini secara arsitektural memerintahkan Aicode untuk mengeliminasi ketergantungan pada *IDE Cloud* pihak ketiga, mengunci seluruh skenario kompilasi skrip, interaksi *error log parsing*, hingga pembacaan evaluasi API Gemini di dalam perimeter tunggal antarmuka tab ekstensi LMS Moodle, menjamin imunitas fokus pemecahan *error syntax* pemula tanpa hambatan navigasi ekskursi peladen.

Tantangan visualisasi analitik data penelusuran rekaman diselesaikan menggunakan komputasi reduksi basis data oleh (Prasetyo et al., 2022) pada penelitian sistem klasifikasi segmen kluster. Memitigasi kompleksitas pembacaan jejak aktivitas

bervolume masif yang acak, metodologi algoritma *K-Means clustering* didukung uji optimasi siku *Elbow* diimplementasikan untuk menggabungkan integrasi atribut model RFM. Eksperimen terbukti andal mengelompokkan riwayat masif menjadi visualisasi tiga dimensi klaster optimal berbekal tingkat presisi *Sum of Square Error* (SSE) raksasa di angka 25.829,39. Filsafat algoritma partisi matriks klaster data raksasa tersebut menyinari pemodelan analitik rekapitulasi modul Aicode; di mana *Database Dashboard plugin* dikonstruksi menyedot ribuan variabel catatan kegagalan luaran *stderr error tracking* siswa SMK harian, mereduksinya, lalu menayangkannya ke antarmuka guru sebagai indikator pola *bug* sintaksis terdominan di kelas secara *real-time*.

Proteksi ketahanan *backend REST network payload* terhadap pertukaran matriks masif komputasi divalidasi presisinya oleh (Mustaqim et al., 2025) pada arsitektur perbandingan *Deep Learning* citra klinis tingkat akurasi kritis. Penanggulangan limitasi *latency interrupt* memori server dipecahkan melalui sinkronisasi koneksi rute terbatas ukur untuk meminimalisasi luapan dimensi model antrean klasifikasi. Pembuktian eksperimentasi resiliensi *network interface payload connection* ini mentransfusikan kebijakan vital keamanan *Plugin Activity Module* Aicode: pengkodean mekanisme penghentian koneksi (*cURL connection timeout restrictions kill-switch*) dipatok secara otoritatif selama dua detik guna mencegah modul PHP komputasi inti Moodle tenggelam pada anomali antrean *infinite ping* ketika menanti *handshake payload* antarmuka mesin komputasi Google Gemini, menyelamatkan *uptime* peladen LMS lokal.

Validasi formal implementasi ekstensi kurikulum berbasis LMS dipatenkan operasionalnya oleh (Hunaidah et al., 2025) untuk modul CinQASE pada ekosistem vokasional digital. Memecahkan kebuntuan diseminasi desain konsep fisika abstrak di portal *learning*, instrumen pengembangan sekuensial tahapan 4D (*Define, Design, Develop, Disseminate*) diterjunkan untuk menjalin komponen interaksi dengan Moodle. Hasil pengujian rekayasa *software* mencetak legitimasi skor validasi pengajuan pakar sangat andal di angka metrik 3.72 serta mendongkrak kurva rasional logis partisipan di kelas *N-Gain* level tinggi. Pembuktian skema stabilitas 4D Moodle

ini mengukuhkan keputusan fungsional pada penelitian Aicode untuk mengaplikasikan rekayasa metodologi kaku SDLC Waterfall sebagai arsitektur absolut penyusunan spesifikasi XMLDB *Activity plugin* peranti lunaknya.

Kapasitas determinasi abstraksi sistem informasi algoritma regresi divalidasi lebih dalam oleh (Triyasri et al., 2025) ketika memprediksi pola ketercapaian keberhasilan operasional akademik subjek riset. Analisis rekayasa matriks riwayat pembelajaran (log vektor data historis) mengonfirmasi probabilitas mesin mematok relasi korelasi performa kurva logis kognitif apabila sensor instrumen mencatat variabel historis *interaction metrics* secara murni konstan. Landasan mesin pembelajaran *activity tracking* ini melegitimasi arsitektur penyusunan kerangka kolom spesifikasi pangkalan data Moodle Aicode; di mana entitas tabel skema aicode_attempts diprogram absolut mengikat perekaman lalu lintas kueri jumlah pertukaran pesan asinkron harian dan durasi kompilasi sebagai *raw training data metrics* pijakan instrumen guru mengalkulasi justifikasi prediksi efektivitas penguasaan bahasa pemrograman JavaScript siswanya pada ujung *gradebook analytics*.

Memecah metode bimbingan statis ortodoks kelas, (Syeptiani, 2021) meriset pendekatan intervensi kemandirian kognitif lewat instrumen resolusi masalah (*problem-solving action*) paduan *mind mapping* pada kurikulum vokasional eksperimental sains menengah. Menanggulangi mandeknya serapan nalar abstraksi siswa vokasi atas instrumen komputasi, metodologi *Classroom Action Research* siklus bertahap diuji untuk melepas rasio intervensi perwalian. Observasi parameter ketuntasan kognitif memperlihatkan daya lonjakan progresif dari titik 32.44% menuju daya serap absolut 89.18% tatkala siswa tidak lagi disuapi tetapi difasilitasi dalam pemecahan masalah. Arsitektur pelepasan gradual intervensi kooperatif inilah yang menjiwai konstruksi algoritma petunjuk bertahap (*graduated diagnostic logic hints*) dari Gemini AI di Aicode, yang menunda intervensi penyelesaian koding paksa, dan hanya melempar sinyal pelacak *reference error bug* bagi siswa bereksperimen perbaikan algoritma otonom.

Keterhubungan sistem arsitektur gamifikasi asinkron format robotik dianalisis

ketahanannya oleh (Kadarina, 2021) ketika mengoperasikan ekstensi antarmuka platform *Chatbot Discord*. Mengatribusi solusi minimnya infrastruktur *feedback* dua arah yang adaptif di instalasi LMS tradisional, pengembangan integrasi proksi API interaktif asinkron di platform eksternal ini memvalidasi eskalasi dimensi kemauan pemelajar dalam menelaah logika pemrograman berulang kendati mengalami repetisi kegagalan antrean eksekusi logis. Meskipun berhasil, kerentanan isolasi platform komunikasi non-akademik mendesak perancangan perangkat arsitektural yang utuh dan aman; melegitimasi arsitektur sentralisasi Aicode menolak ekskursi ke portal *IDE* eksternal dan menetapkan rute pertukaran kueri JSON API AI generatif Gemini terkunci mutlak di rute antarmuka *internal web authentication backend* Moodle, menjamin rekaman otentik.

Melengkapi tantangan komputasi respon jaringan layanan *Chatbot*, (Hidayatulloh et al., 2021) membedah disrupsi arsitektur integrasi koneksi mikro dalam aplikasi gamifikasi pendidikan. Di balik fasilitasi *engagement rate* belajar bot, literatur membedah celah keamanan keandalan *rule-based memory caching* di mana bot rentan mengalami kolaps saat antrean paralel meledak dan mengorbankan parameter pertahanan kerahasiaan identitas privasi data (*security privacy thread issue*). Resolusi arsitektur *stateless network memory payload* yang direkomendasikan melandasi rekayasa pemrograman antarmuka proksi Gemini Moodle Aicode, di mana injeksi identitas pengguna dienkrpsi dari pengamatan model generatif via penyisipan tag bendera privasi (*allow_training = 1* penyamar *userid*), serta mengisolasi kompilasi Node.js pada koneksi porta yang bebas kolaps *state manipulation* dari peladen memori LMS induk.

Penerimaan budaya rekayasa operasional perangkat lunak antarmuka manajemen di level nasional dikaji secara kuantitatif oleh (Leonarde Ginting & Silalahi, 2025) pada utilitas Moodle LMS murni. Menyelesaikan silang pendapat adaptasi kerumitan e-learning, riset berlandaskan instrumen evaluasi *System Usability Scale* (SUS) mengekstraksi tingkat justifikasi kepuasan navigasional *user interface* mahasiswa. Kesimpulan perihal tingginya penerimaan navigasi adaptasi Moodle membuktikan integrasi *UX Moodle framework* telah mengakar kuat di ekosistem

sekolah Indonesia. Faktor budaya teknis ini melegalisasi keputusan pengkodean untuk tidak membangun sistem antarmuka kompilator mandiri dari nol, melainkan mengikat modul komponen *JavaScript AMD client editor* Aicode di dalam bodi Moodle, dan mengizinkan validasi *Walkthrough Testing Logging Scenario* sebagai pengukur kelayakan metrik pencatatan pangkalan data ketimbang membagi kuesioner opini SUS yang ekuivalen kembali.

Kekakuan paradigma arsitektur rekayasa piranti lunak (*software architecture block rigidity*) sebelum fase pengkodean kompilasi diinvestigasi oleh (Oktarino et al., 2024) mengenai operasional pengembangan model relasional infrastruktur web sentra informasi sekolah. Penanggulangan tabrakan relasi parameter antarmuka dan pangkalan data dilakukan menggunakan ketegasan fungsional representasi diagram hierarkis *Unified Modeling Language* (UML) statis. Filsafat stabilitas arsitektur desain komputasi yang direkomendasikan ini memandu ketegasan landasan metodologis pengkodean rekayasa sistem *plugin Activity Module* Aicode untuk secara murni dan eksklusif mempertahankan eksekusi kerangka kerja SDLC Waterfall konvensional, meredam mutasi dinamis atas *Extensible Markup Language Database* Moodle (seperti tabel referensi relasi *userid* dan *course.id*) di tengah fase rilis versi.

Resiliensi keandalan parameter infrastruktur fisik laboratorium praktikal vokasi diaudit keterbatasan *bandwidth memory hardware* CPU-nya oleh (Arief, 2021). Terbantur kendala disrupsi komputasi lamban akibat instalasi aplikasi program antarmuka IDE lingkungan pengembangan kelas atas di komputer lawas, studi merekomendasikan transisi skenario dekopling aplikasi yang ringan pada jaringan pusat. Faktor pemecahan masalah *overhead memory rendering crash* yang diukur menginisiasi cetak biru piranti lunak instalasi Aicode: menaruh layar presentasi klien ke peramban basis web asinkron super ringan di unit komputer siswa, seraya menyedot lalu lintas beban interpretator algoritma JavaScript seutuhnya melintasi HTTP REST interkoneksi ke lapisan pemroses proksi isolasi mandiri (mesin Node.js porta 3001), memerdekakan CPU klien dari potensi pembekuan (*overheating latency crash logic*).

Kemampuan literasi transisi interaksi teknologi dan resiliensinya diukur oleh (Febrianty et al., 2024) kala membedah kerentanan kurva adaptasi pegawai pengguna

fasilitas perangkat lunak layanan masif yang kaku. Observasi kualitatif melucuti anomali demotivasi (stres komputasional blokade mental) yang lazim terjadi apabila sistem berorientasi sintaksis mesin murni dikerahkan tanpa *soft scaffolding assistance support* pengantar asimilasi fungsi kognitif. Tinjauan empiris akan pentingnya antarmuka navigasi bahasa natural sebagai peredam rintangan interaksi ini selaras dalam keputusan spesifikasi Aicode menetapkan perintah instruksi parameter *anti-spoiler AI logic prompt* pada berkas arsitektur LLM `classes/local/ai_prompt.php`; memaksa antarmuka proksi untuk mereduksi kebekuan referensi sintaksis *ReferenceError compiler Node.js* berbahasa Inggris menjadi luaran bahasa Indonesia asisten Sokratis naratif bertingkat yang menuntun pemula.

Penguatan dimensi penguasaan *hands-on architecture practice* untuk infrastruktur sirkuit mesin terpadu diteliti oleh (Sitorus et al., 2021) terhadap adaptasi industri otomatisasi perangkat kontrol mikrokontroler peserta didik kejuruan digital. Menganulir pasivitas instruksi papan tulis klasikal murni instruktur guru teori, desain pembelajaran interaktif siklus eksekusi memvalidasi hipotesis rasionalitas kognisi teknik bahwa pelajar vokasional berevolusi melalui percobaan eksperimen asinkron logis berkelanjutan (*iterative architecture execution logic*) berbekal *error* respon perangkat keras. Esensi uji transmisi komputasi interaktif asinkron di level arsitektur kejuruan (*hardware*) ini direplikasi ke kerangka peranti lunak berbasis *web browser module* klien Aicode, yang menstimulasi determinisme kemandirian koding mikro fundamental peserta dengan memberikan akses purwarupa evaluasi skrip *real-time execution test loop* (tanpa penalti *Grades*) dari batas elemen proksi sandi Node.js peladen tanpa batas manipulasi otonom.

2.2 Penelitian Sebelumnya

Sintesis komprehensif dari tinjauan pustaka sebelumnya diabstraksikan ke dalam matriks *State-of-the-Art* secara sistematis dengan menyeleksi 10 literatur inti yang secara absolut bersinggungan dengan fokus arsitektur *plugin* Aicode. Matriks ini bertugas membedah fondasi metodologi, temuan komputasi, serta mendemonstrasikan secara jelas celah kebaruan fungsional (*Gap Analysis*) dari rancang bangun sistem ini:

Tabel 2.1 Penelitian Sebelumnya

No.	Nama Peneliti dan Tahun	Judul Artikel dan Nama Jurnal	Metode Penelitian	Hasil Penelitian	Perbedaan dan Relevansi dengan Penelitian Ini (Gap Analysis)
1	Frankford et al. (2026)	Chatbot-Based Assessment of Code Understanding in Automated Programming Assessment Systems (<i>arXiv Preprint</i>)	Konseptualisasi desain <i>Hybrid Socratic Framework</i> pemahaman kompilasi sintaksis.	Model dialog Sokratis yang mengabungkan statik dinamis analisis kompilator mencegah ketergantungan model bocoran AI.	Perbedaan: Implementasi <i>Chatbot</i> analitis percakapan dinamis. Relevansi: Mendasari <i>prompt engineering anti-spoiler</i> Aicode yang dimanipulasi dengan algoritma PHP <code>array_slice</code> mereduksi JSON menjadi 3 <i>hint</i> .
2	Zhu & Liu	CodeRunner Agent:	Integrasi agen	Penggabungan data	Perbedaan: Menggunakan

	(2025)	Context-Aware Feedback on Student Programming Self-Regulation in Moodle LMS (<i>CHI 2025</i>)	kecerdasan buatan terpadu <i>CodeRunner</i> LRS xAPI di Moodle.	riwayat eksekusi LMS ke AI menghasilkan sentralisasi umpan balik yang mengakselerasi <i>self-regulated learning</i> .	ekstensi berbasis LRS murni interaksi agen. Relevansi: Mendukung keputusan sentralisasi pelacakan dan modifikasi <i>frontend AMD editor client</i> Aicode di dalam perimeter internal Moodle tanpa berpindah portal eksternal.
3	Nwohiri et al. (2025)	Automated Multi-Language Framework for Coding Assessment and Feedback	Perancang an arsitektur <i>backend</i> terpisah fungsional keamanan infrastruk	Mode isolasi <i>sandboxing</i> eksekutor menjamin jaringan host	Perbedaan: Kerangka spesifikasi multi-bahasa sistem luas. Relevansi: Menjustifikasi implementasi

		<i>(Journal of Computer Science)</i>	ur 4 lapisan peladen.	terisolasi murni dari penetrasi skrip kode mentah siswa.	proksi <i>environment</i> Node.js asinkron port 3001 di Aicode untuk memutus rantai eksekusi <i>untrusted loop</i> 50KB dari peladen PHP.
4	Liao et al. (2024)	Scaffolding Computatio nal Thinking With ChatGPT <i>(IEEE TLT)</i>	Eksperime n berbasis metode campuran penggunaa n IPSSC ChatGPT untuk intervensi CT.	Bantuan <i>scaffolding</i> gen AI terbukti memangka s latensi penyelesai an <i>error</i> dan mendongk rak motivasi belajar <i>computatio nal thinking.</i>	Perbedaan: Platform AI antarmuka generik umum. Relevansi: Aicode memanfaatkan terjemahan intervensi formatif bahasa Indonesia atas limitasi kompilator Node.js untuk mengatasi demotivasi teknis siswa.
5	Alshamm	An	Evaluasi	Penggunaa	Perbedaan:

	ari (2025)	Investigation into ChatGPT-Based Instructional Feedback on Programming Projects (<i>Proceedings of ICCIT IEEE</i>)	kualitatif performansi waktu umpan balik proyek AI komparatif dengan manual laboratorium.	n API menyusutkan waktu koreksi, menuntut kebutuhan perekaman basis data riwayat AI absolut menahan luaran <i>log anomaly</i> .	Studi beban kinerja evaluasi proyek terbuka perguruan tinggi. Relevansi: Memastikan <i>plugin activity</i> Aicode mencatat sinkronisasi kueri di tabel XMLDB aicode_attempts (<i>Human-in-the-Loop</i>).
6	Ma et al. (2025)	DBox: Scaffolding Algorithmic Programming Learning through Learner-LLM Co-Decomposition (<i>CHI 2025</i>)	Uji desain <i>Human-Computer Interaction prototype</i> intervensi dekomposisi berjenjang (<i>co-decomposition</i>).	Pendelegasian langkah bertahap pemecahan koding bersama AI menurunkan disrupsi daya kognitif	Perbedaan: Arsitektur antarmuka dialog pemecahan parsial asinkron. Relevansi: Melegitimasi modul <i>analyze_code.php</i> Aicode

				tanpa memotong kepuasan kemandirian.	memangkas luaran <i>JSON hint</i> hierarki rekomendasi agar otak pelajar SMK berlatih logika deterministik resolusi sintaksis asinkron mandiri.
7	Chidambaram et al. (2024)	Socratic Human Feedback (SoHF): Expert Steering Strategies for LLM Code Generation (<i>Findings of ACL - EMNLP</i>)	Evaluasi performa <i>expert steering strategies</i> modifikasi parameter intervensi model LLM statis.	Memandu model dengan tahapan klarifikasi interupsi penalaran mengelimi nasi kegagalan model saat berhalusinasi menyintesis skrip rumit	Perbedaan: Evaluasi performa generasi arsitektural asisten pelatih teks. Relevansi: Mematenkan spesifikasi kode berkas <code>classes/local/ai_prompt.php</code> berisi rekayasa injeksi <i>Zero-Spoiler prompt</i>

				murni.	<i>boundaries</i> instruksi AI penuntun Sokratik kaku Aicode.
8	Zhang et al. (2024)	Students' Perceptions and Preferences of Generative Artificial Intelligence Feedback for Programming (<i>Proceedings of AAAI</i>)	Analisis tematik preferensi formatif atas balasan integrasi antarmuka LLM CS1 kelas pemrograman Java.	Menghasilkan persepsi paling formatif dan korektif komputasi onal kala skrip kode asli milik pengguna mentah dikirim utuh masuk ke dasar paramater <i>prompting</i> .	Perbedaan: Riset opini utilitas <i>Natural Language API feedback</i> murni. Relevansi: Mewajibkan Aicode melempar variabel <code>source_code</code> string seutuhnya berikut error output payload dari web klien penulisan ke Google Flash API.
9	Yudiono (2024)	Moodle-based LMS periodic	Pemetaan skor N-gain	Keberadaan platform tangguh	Perbedaan: Domain subjek kejuruan

		maintenanc e competence on EFI system in Vocational High Schools (JVCE)	efektivitas asimilasi pembelajar an Moodle pada praktik kejuruan permesina n <i>maintenan ce</i> instrumen sirkuit.	mendongk rak ketuntasan otonom kemandiria n pelajar dengan hasil lompatan metrik kuantitatif tinggi di ambang <i>normalized gain 0.702.</i>	vokasi teknik industri kelas mekanik konvensional. Relevansi: Penegasan secara empiris stabilitas dan kecocokan integrasi <i>core database engine PHP server Moodle 5.0+</i> sebagai jangkar utama wadah instalasi rekayasa perangkat lunak <i>software plugin</i> vokasional <i>engineering logic loop.</i>
10	Syeptiani (2021)	Penerapan Pembelajar an Kooperatif Tipe	Pengukura n komparatif dimensi metode	Mengonfir masi progresi linier akselerasi	Perbedaan: Simulasi tatap muka kooperatif pemetaan

		Problem Solving dengan Teknik Mind Mapping [...] (<i>JPVR</i>)	penugasan interaksi pendelegasian pemecahan komputasi logis <i>PTK (Classroom Action Research)</i> eksperimental otonomi fungsional kognitif rasionalitas deterministik siklus bertingkat.	pemahaman lapangan ketika instrumen diinterupsi oleh asimilasi lepas-kendali (<i>fading intervention</i>), parameter observasi dimensi ketuntasan rasional teknis terkalibrasi memuncak di ambang optimal efektivitas rasio penguasaan angka indeks	rasional otonomi kimia eksperimental praktikal konvensional statis fungsional <i>classroom architecture pedagogical manual action flow evaluation design testing.</i> Relevansi: Memberi kerangka filosofis <i>Graduated Scaffolding Logic JSON API LLM Array Intervention Design</i> ; proksi Gemini tidak dirancang memperbaiki struktur kode sintaksis Javascript
--	--	--	--	---	---

				probabilitas indeks 89.18% optimal otonom rasionalisasi komputasi eksperimental fungsional logis presisi mandiri otonom murni.	pelajar, namun diorkestrasi mereproduksi <i>logical</i> <i>reasoning</i> <i>debugging</i> <i>hints hints</i> berantai yang dirancang menstimulasi otonomi pemecahan <i>syntax error</i> <i>compiler loop</i> <i>Node JS</i> <i>execution crash</i> <i>handler</i> secara mandiri dan komputasi otonom.
--	--	--	--	--	---

2.3 Landasan Teoretik

Pada landasan teoretik ini membedah fundamental ilmu komputer, rekayasa perangkat lunak, dan interaksi manusia-komputer yang secara kaku membentuk spesifikasi arsitektur Aicode.

2.3.1 Kerangka Pengembangan Software Development Life Cycle (SDLC) Waterfall

Model pengembangan perangkat lunak Waterfall menyajikan pendekatan

sekuensial, logis, dan linier, mencakup analisis persyaratan, desain arsitektur, implementasi (pengkodean), dan tahap verifikasi (pengujian) fungsional. Dalam konteks spesifik pengembangan modul pendidikan berbasis Moodle, penerapan SDLC Waterfall sangat krusial guna memastikan spesifikasi *Extensible Markup Language Database* (XMLDB) dan *Capabilities Access Control* dideklarasikan secara konstan sejak awal fase desain. Fleksibilitas metode iteratif (*agile*) berpotensi merusak integritas *Moodle Gradebook API* dan hierarki *foreign key*, seperti *course.id*, di tengah siklus pengembangan, sehingga paradigma desain statis Waterfall menjadi kerangka kerja absolut untuk proyek *plugin* ini.

2.3.2 Arsitektur Moodle Activity Modules dan Web Services

Sistem manajemen pembelajaran Moodle berlandaskan arsitektur berorientasi objek yang sangat modular (*modular object-oriented*). *Plugin* dengan komponen Aicode terikat oleh kerangka *API Core* yang mensyaratkan eksistensi berkas pengait (*lifecycle hooks*). Berkas *lib.php* di dalam inti Aicode memegang peran eksklusif untuk mendikte aturan pelacakan status penugasan dan menyuntikkan nilai evaluasi ke subsistem *Moodle Gradebook*. Lebih lanjut, untuk mempertahankan pengalaman *user interface* yang responsif tanpa penyegaran halaman, komunikasi dikendalikan via kerangka *Moodle External Web Service Subsystem*. Pertukaran data asinkron (*AJAX payloads*) ditangani melalui deklarasi *endpoints* otentik (seperti *mod_aicode_run_code* and *mod_aicode_analyze_code*) yang diregistrasi pada *db/services.php*.

2.3.3 Pedagogi Pemrograman JavaScript Fase F SMK

Peserta didik tingkat SMK Kelas 11 yang berada dalam Kurikulum Merdeka Fase F sedang berada pada tahapan krusial pembentukan nalar algoritma dasar. Target capaian pembelajaran menuntut penguasaan sintaksis bahasa JavaScript. Ruang lingkup keilmuan yang diprogram dalam spesifikasi Aicode secara ketat difokuskan pada manipulasi logika *Basic JavaScript*, meliputi: deklarasi variabel (*let*, *const*), pengendalian tipe data, operasi aritmatika dan logika, percabangan kondisional (*if...else*, *switch*), struktur perulangan (*for*, *while*), deklarasi fungsi dasar, manipulasi *array*, pemrosesan *string*, hingga intervensi dasar pada DOM seperti penggunaan

`document.getElementById()`. Pendekatan pedagogi menuntut lingkungan berlatih di mana isolasi konsep dijaga; karenanya, *plugin* memberikan fasilitas guru untuk menyediakan *template* kerangka HTML statis (melalui kolom `htmltemplate`) sehingga siswa hanya perlu memfokuskan beban kognitif mereka pada logika JavaScript tanpa terdistraksi elemen desain antarmuka.

2.3.4 Pendekatan Sokratik dan Batasan Umpan Balik Anti-Spoiler

Pedagogi pemrograman modern menyoroti pentingnya mereduksi beban kognitif berlebih ketika siswa memecahkan *standard error* (*stderr*). Untuk menyelaraskan intervensi mesin dengan kebutuhan instruksional, metode Sokratik dan prinsip *scaffolding* direkayasa secara algoritmik ke dalam parameter AI. Sistem tidak bertindak sebagai kompilator *auto-complete* melainkan "pembimbing inferensial". Sistem mengamankan batasan umpan balik (*Anti-Spoiler Boundaries*) dengan memuat regulasi absolut pada berkas `classes/local/ai_prompt.php`, di mana model dipaksa memberikan diagnosis langkah demi langkah dan sangat dilarang ("*Jangan menyalahkan siswa; gunakan nada membimbing dan suportif*") memberikan resolusi blok kode secara utuh. Fungsi proksi penormalisasi pada PHP `classes/external/analyze_code.php` kemudian menerapkan manipulasi *array_slice* untuk memangkas keluaran balasan menjadi maksimal 3 item petunjuk (*graduated hints*). Hal ini secara teknis memastikan praktik *fading support* (pelepasan kendali) beroperasi sempurna tanpa membocorkan algoritma final.

2.2.5 Isolasi Eksekusi Sandbox (Node.js)

Tantangan terbesar sistem penilaian otonom adalah mengeksekusi kode klien *untrusted* di peladen host. Menjalankan *string* pemrograman secara mentah membuka vektor eksploitasi fatal terhadap sistem Moodle (misalnya pengaksesan skrip konfigurasi Moodle *database* atau injeksi eksekusi sistem biner). Oleh karena itu, prinsip arsitektur dekopling keamanan (*Security Execution Decoupling*) diterapkan. Aicode mengalihkan lalu lintas eksekusi sepenuhnya ke peladen mesin penafsir asinkron independen (*Node.js runtime*) melalui koneksi *cURL* internal REST HTTP menuju porta 3001. Proses evaluasi pada modul `run_code.php` membungkus setiap

kiriman dengan restriksi benteng berlapis: pembatasan ukuran dimensi berkas maksimal 50.000 bita (50KB) dan batasan tenggat waktu eksekusi otomatis (*execution timeout*) berdurasi dua detik. Protokol ini menjadi *sandbox* otonom yang menetralisasi bahaya perulangan tak hingga (*infinite loops*) maupun invasi berkas lokal (*file-system hijacking*).

2.3.6 Umpan balik formatif dan penggunaan Human in loop

Secara teoretik, umpan balik formatif ditujukan agar siswa memahami kesalahan dan memperbaiki konsep mereka selama proses belajar. (Chang & others, 2025) menekankan peran guru dalam kerangka formatif untuk meningkatkan pemahaman siswa melalui interaksi berulang dengan LLM. Dalam AICode, komponen *Analyzer* menyediakan umpan balik formatif berupa diagnosis kategori kesalahan dan hint bertingkat (berdasarkan trace eksekusi). Feedback ini diharapkan meningkatkan learning gain dengan membimbing siswa mengidentifikasi dan memperbaiki kesalahan.

Manajemen *Human-in-the-Loop* didasarkan pada teori kolaborasi manusia-AI. Model hibrida ini menggabungkan kecepatan AI dengan penilaian subjektif guru. Fitur AICode mengizinkan guru meninjau dan meng-*override* umpan balik AI melalui *Teacher Overrides*. (Armfield & others, 2025) menemukan bahwa keterlibatan pengajar dalam proses pemberian saran signifikan meningkatkan efisiensi / pengurangan beban koreksi dan kepercayaan siswa terhadap sistem. Dengan demikian, peran HITL dalam AICode, guru dapat merevisi feedback AI atau mereset analisis diharapkan meningkatkan akurasi dan keterterimaan umpan balik.

2.3.7 Large Language Models (LLM) Gemini

Komponen intelijensia sistem bertumpu pada arsitektur antarmuka pemrograman aplikasi API LLM gemini yang disuplai oleh Google. Koneksi jaringan diamankan melalui otentikasi kunci *token* yang dikonfigurasi melalui panel administrasi Moodle. Demi meminimalisasi fenomena halusinasi data (*hallucinations*) serta mempertahankan formasi luaran format struktur JSON yang solid untuk diurai oleh peladen, pengaturan parameter generasi LLM direkayasa secara definitif: atribut

derajat imajinasi (*temperature*) ditekan ke angka rendah 0.2, nilai *topP* diatur sebesar 0.8, dan batas produksi teks (*maxOutputTokens*) dikunci pada batas maksimal 2048 token. Selain itu, manajemen panggilan API dilindungi dari penyalahgunaan batas permintaan (*rate limiting*) dengan mengizinkan maksimum 50 panggilan AI per siswa per hari, disokong oleh rekayasa stabilitas penyimpanan singgahan memori lokal (*Cache TTL*) selama 3600 detik (1 jam) untuk meningkatkan kinerja respon dan menekan latensi secara komputasional. Sistem juga mengamankan perlindungan kerahasiaan identitas siswa dengan menghapus informasi *userid* (anonimisasi dengan mengirimkan bendera *allow_training* = 1) sebelum struktur *payload error* dikirimkan ke mesin komputasi Google.

2.8.9 Teknik-Teknik Prompt Engineering

Prompt engineering merupakan disiplin empiris dalam merancang instruksi tekstual yang optimal agar LLM dapat menyelesaikan tugas spesifik dengan tingkat akurasi dan reliabilitas yang tinggi. Dalam konteks sistem pendidikan yang terintegrasi, merancang *prompt* yang efektif sangat krusial untuk memastikan AI memberikan umpan balik diagnostik yang sesuai dengan tujuan pedagogis serta mengembalikan respons dalam format struktur data (seperti JSON) yang kaku (Schulhoff *et al.*, 2024). Terdapat beberapa paradigma utama dalam *prompt engineering* yang secara luas diaplikasikan untuk mengendalikan perilaku LLM.

2.3.8.1 Zero-Shot Prompting

Zero-shot prompting adalah teknik di mana model diinstruksikan untuk menyelesaikan sebuah tugas baru tanpa diberikan contoh (demonstrasi) penyelesaian tugas tersebut sebelumnya di dalam *prompt* (Kojima *et al.*, 2022). Model diandalkan sepenuhnya pada pengetahuan intrinsik dan kemampuan inferensi yang diperolehnya selama fase prapelatihan (*pre-training*) dan *fine-tuning*. Kelebihan utama dari metode ini adalah efisiensi token, yang secara langsung meminimalkan beban latensi komputasi dan biaya pemrosesan API. Namun, pada tugas-tugas konseptual yang sangat terstruktur, seperti mengembalikan keluaran wajib dalam format JSON berskema ketat tanpa teks tambahan, *zero-shot prompting* rentan mengalami

inkonsistensi format atau mengabaikan instruksi batasan (Reynolds & McDonell, 2021).

2.3.8.2 Few-Shot Prompting

Sebagai evolusi untuk mengatasi kelemahan *zero-shot*, *few-shot prompting* (sering disebut juga sebagai *in-context learning*) menyertakan beberapa pasang contoh input dan output ideal (demonstrasi) di dalam teks instruksi, sebelum model memproses input pengguna yang sebenarnya (Brown *et al.*, 2020). Dengan mengobservasi contoh-contoh tersebut, model bahasa dapat menyimpulkan struktur format keluaran, gaya bahasa, dan pola penalaran yang diharapkan (Dong *et al.*, 2022). Dalam konteks pembangkitan JSON, penambahan contoh konkret—seperti mendemonstrasikan objek JSON yang benar untuk jenis sintaks yang salah—terbukti secara empiris mampu meningkatkan *Parsing Success Rate* (PSR) secara signifikan dan mengurangi pelanggaran skema data, meskipun teknik ini mengorbankan ukuran *payload prompt* yang lebih besar (konsumsi token meningkat).

2.3.8.3 Role-Prompting (Adopsi Persona)

Role-prompting adalah teknik instruksional di mana *prompt* diawali dengan menetapkan persona atau identitas spesifik bagi AI (misalnya, "*Anda adalah seorang tutor pemrograman JavaScript yang ahli dan sabar bagi siswa SMK*"). Pendekatan ini secara implisit membatasi ruang pencarian semantik (*semantic search space*) model agar distribusi probabilitas tokennya lebih selaras dengan diksi, nada, dan kedalaman respons yang diasosiasikan dengan persona tersebut di dalam korpus pelatihannya (Reynolds & McDonell, 2021). Pada aplikasi *intelligent tutoring system*, *role-prompting* menjadi sangat vital karena persona seorang guru akan mencegah model bertindak sebagai sekadar alat penyelesaian kode (*code-completion tool*), dan sebaliknya memandu AI untuk mematuhi kaidah Sokratik—memberikan penjelasan konseptual tanpa langsung menyuapi solusi kode (*anti-spoiler*).

2.3.8.4 Studi Ablasi (Ablation Study) dalam Desain Prompt

Dalam rekayasa sistem kecerdasan buatan, studi ablasi adalah metodologi pengujian analitis di mana komponen atau fitur tertentu dari arsitektur sistem, blok

kalimat atau aturan spesifik di dalam *prompt* bisa dihilangkan secara sistematis. Tujuannya adalah untuk mengevaluasi seberapa besar kontribusi empiris dari komponen tersebut terhadap performa keseluruhan sistem. Pada pengembangan *prompt* yang kompleks, studi ablasi digunakan untuk mengisolasi dan mengukur efek dari instruksi-instruksi khusus, seperti aturan "Hanya keluarkan JSON", penetapan persona, atau aturan pedagogis. Jika penghapusan suatu instruksi menyebabkan metrik performa, seperti PSR atau akurasi kategori error turun drastis, maka instruksi tersebut divalidasi sebagai komponen kritis (*core component*). Sebaliknya, jika performa tetap stabil saat instruksi dihapus, instruksi tersebut dapat dianggap redundan dan sebaiknya dieliminasi demi optimasi efisiensi token.

2.3.9 Evaluasi dan Metrik Performa Prompt Engineering

2.3.9.1 Urgensi Benchmarking Keluaran LLM pada Perangkat Lunak Edukasi

Pemanfaatan LLM dalam perangkat lunak pendidikan menghadirkan tantangan fundamental yang berbeda secara kualitatif dari penggunaan LLM pada aplikasi generatif konvensional seperti *chatbot* atau asisten penulisan. Pada aplikasi generatif konvensional, variasi keluaran model justru dianggap sebagai keunggulan karena menghasilkan respons yang terasa alami dan tidak repetitif. Sebaliknya, pada perangkat lunak edukasi yang terintegrasi dengan sistem manajemen pembelajaran (LMS), keluaran model harus memenuhi kontrak antarmuka (*interface contract*) yang rigid agar dapat diparsing, disimpan, dan divisualisasikan secara konsisten oleh komponen-komponen perangkat lunak hilir (*downstream software components*) seperti basis data, *dashboard* guru, dan antarmuka siswa (Schulhoff & others, 2024).

Dalam konteks spesifik plugin `mod_aicode` pada platform Moodle, setiap respons dari Gemini API harus memenuhi tiga syarat kritis sebelum dapat berfungsi dalam ekosistem LMS. Pertama, respons harus berupa objek JSON tunggal yang mematuhi skema terstruktur dengan tujuh *field* utama (`diagnosis`, `location`, `hints`, `suggested_fix`, `recommended_materials`, `explainability`) yang masing-masing memiliki tipe data dan kendala yang telah ditetapkan. Kedua, konten umpan balik di dalam JSON tersebut harus mematuhi

prinsip pedagogis Sokratik, yaitu membimbing siswa menemukan solusi secara mandiri tanpa membocorkan jawaban langsung. Ketiga, waktu respons harus berada dalam batas toleransi yang tidak mengganggu pengalaman belajar siswa.

Kegagalan pada syarat pertama, yakni keluaran yang tidak dapat diparsing sebagai JSON valid akan menyebabkan kaskade kegagalan pada seluruh rantai pemrosesan data. Field-field diagnostik tidak dapat mengisi kolom basis data yang sesuai, grafik kemajuan siswa pada *Teacher's Dashboard* tidak dapat diperbarui, dan log aktivitas pembelajaran menjadi tidak tersedia untuk analisis oleh guru. Oleh karena itu, evaluasi sistematis terhadap performa prompt melalui *benchmarking* merupakan langkah esensial dalam siklus pengembangan perangkat lunak edukasi berbasis LLM, bukan sekadar aktivitas pengujian opsional.

2.3.9.2 Determinisme Keluaran (*Output Determinism*) dalam Sistem LLM

Determinisme keluaran merujuk pada tingkat konsistensi dan prediktabilitas respons yang dihasilkan oleh LLM ketika diberikan masukan yang identik atau serupa. Secara matematis, model bahasa berbasis arsitektur *transformer* menghasilkan token secara autoregressif berdasarkan distribusi probabilitas yang dihitung melalui fungsi *softmax* bersuhu:

$$P(x_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

di mana z_i adalah logit dari token ke- i dan T adalah parameter temperatur. Ketika $T \rightarrow 0$, distribusi menjadi sangat tajam (*peaked*) sehingga hanya token dengan probabilitas tertinggi yang dipilih (*greedy decoding*), menghasilkan keluaran yang nyaris deterministik. Sebaliknya, Ketika $T \rightarrow 1$, distribusi mendekati distribusi asli sehingga meningkatkan keragaman tetapi juga meningkatkan risiko penyimpangan dari format yang diharapkan.

Dalam konteks perangkat lunak produksi, determinisme keluaran menjadi krusial karena komponen perangkat lunak hilir tidak dirancang untuk menangani variasi format yang tak terbatas. Parser JSON, misalnya, mengharapkan struktur data yang konsisten dan akan gagal jika model menghasilkan teks naratif di luar blok JSON

atau mengubah nama *field* secara arbitrer. Oleh karena itu, konfigurasi parameter generasi, khususnya temperature dan topP (nucleus sampling) menjadi variabel independen yang kritis dalam pengujian performa prompt.

Parameter topP bekerja secara komplementer dengan temperatur melalui mekanisme *nucleus sampling* yang membatasi pemilihan token pada subset kumulatif probabilitas teratas. Ketika topP = 0.8, hanya token-token yang secara kumulatif membentuk 80% probabilitas tertinggi yang dipertimbangkan sebagai kandidat, sehingga token-token dengan probabilitas sangat rendah (yang berpotensi menyebabkan deviasi format) secara efektif dieliminasi dari ruang pencarian.

2.3.9.3 Validasi Skema JSON sebagai Metrik Reliabilitas Format

JSON PSR merupakan metrik kuantitatif primer yang mengukur persentase respons LLM yang berhasil memenuhi tiga tahap validasi berturut-turut: (1) respons API menghasilkan kode HTTP 200, (2) teks keluaran dapat diparsing menjadi objek JSON valid, dan (3) objek JSON tersebut lolos validasi skema (*schema validation*) yang mencakup keberadaan *field* wajib, kesesuaian tipe data, dan pemenuhan batasan nilai. Secara formal, PSR didefinisikan sebagai:

$$\text{PSR} = \frac{N_{\text{valid}}}{N_{\text{total}}} \times 100$$

di mana N_{valid} adalah jumlah respons yang lolos ketiga tahap validasi dan N_{total} adalah total permintaan API yang berhasil menerima respons. Dalam literatur evaluasi sistem LLM, $\text{PSR} \geq 95\%$ dianggap sebagai ambang batas minimal untuk sistem produksi, sedangkan $\text{PSR} < 85\%$ mengindikasikan kebutuhan mendesak untuk revisi template prompt.

Kegagalan parsing JSON pada sistem berbasis LLM umumnya disebabkan oleh tiga pola:

- model menghasilkan teks pengantar atau penutup di luar blok JSON,
- model membungkus JSON dalam blok kode Markdown (````json ... ````),
- model menghasilkan JSON yang secara sintaksis tidak valid, misalnya karena *string truncation* akibat batasan `maxOutputTokens`.

Strategi mitigasi terhadap ketiga pola kegagalan ini memerlukan kombinasi

kendala pada level prompt (*prompt-level constraints*), konfigurasi parameter generasi (*generation-level tuning*), dan parser *fallback* bertingkat pada level pasca-proses (*post-processing level*), sebuah pendekatan yang dalam terminologi keamanan perangkat lunak dikenal sebagai *defense-in-depth*.

2.3.9.4 Tingkat Kegagalan Parsing (*Failure Rate*) dalam Integrasi LLM

Tingkat kegagalan parsing merupakan komplemen dari PSR yang mengukur proporsi respons yang gagal memenuhi validasi format. Dalam konteks integrasi LLM dengan perangkat lunak produksi, setiap kegagalan parsing berimplikasi langsung pada pengalaman pengguna: siswa yang mengharapkan umpan balik diagnostik justru menerima pesan error generik yang tidak informatif, atau dalam skenario terburuk saat antarmuka pengguna menampilkan data kosong atau rusak. Oleh karena itu, analisis mendalam terhadap distribusi dan pola kegagalan parsing menjadi esensial untuk mengidentifikasi kelemahan spesifik dalam template prompt dan menentukan strategi perbaikan yang tepat sasaran.

Klasifikasi kegagalan parsing dalam penelitian ini mengadopsi taksonomi lima tingkat: `json_parse_failed` (teks tidak dapat diparsing sama sekali), `missing_diagnosis` (objek JSON valid tetapi tidak memiliki *field* diagnosis), `schema_incomplete` (field wajib seperti `message_short` atau `message_long` kosong), `low_confidence` (nilai confidence di bawah ambang batas 0.6), dan `http_error` (kegagalan pada level jaringan). Taksonomi ini memungkinkan identifikasi presisi mengenai lapisan mana dalam arsitektur prompt yang memerlukan penguatan.

2.3.9.5 Kepatuhan Sokratik (*Socratic Adherence*) sebagai Metrik Pedagogis Kualitatif

Di luar metrik teknis seperti PSR dan latensi, evaluasi performa prompt pada perangkat lunak edukasi memerlukan dimensi pengukuran yang secara spesifik menangkap kualitas pedagogis keluaran. *Socratic Adherence Rate* (SAR) merupakan metrik kualitatif yang mengukur sejauh mana umpan balik yang dihasilkan LLM mematuhi prinsip *tutoring* Sokratik, sebuah pendekatan pedagogis yang berakar pada

tradisi filsafat Sokrates di mana tutor membimbing pelajar menemukan pemahaman melalui pertanyaan pengarah dan petunjuk konseptual, bukan melalui pemberian jawaban langsung (Roscoe & Chi, 2007).

Dalam konteks pembelajaran pemrograman, kepatuhan Sokratik memiliki signifikansi khusus yang terhubung langsung dengan level kognitif pada Taksonomi Bloom revisi (Anderson & Krathwohl, 2001). Pemberian solusi kode lengkap oleh AI hanya menstimulasi level kognitif terendah yaitu *Mengingat (Remember)*, di mana siswa cukup menyalin kode tanpa memahami konsep di baliknya. Sebaliknya, umpan balik Sokratik yang memberikan petunjuk arah dan pertanyaan pengarah mendorong siswa mencapai level *Menganalisis (Analyze)* dan *Mengevaluasi (Evaluate)*, di mana siswa harus memahami mengapa kesalahan terjadi dan mengevaluasi alternatif solusi secara mandiri.

Kepatuhan Sokratik dioperasionalisasikan melalui empat aspek evaluasi: (a) *Anti-Spoiler*, sejauh mana *field code_patch* hanya berisi perbaikan minimal yang tidak membocorkan solusi lengkap, (b) *Nada Pedagogis*, apakah keluaran menggunakan nada membimbing dan suportif alih-alih menghakimi, (c) *Arah Berpikir*, apakah keluaran memberikan pertanyaan pengarah atau petunjuk konseptual alih-alih jawaban langsung, dan (d) *Bahasa Siswa*, apakah keluaran menggunakan bahasa Indonesia baku yang sederhana dan mudah dipahami oleh siswa SMK. Pengukuran SAR dilakukan secara kualitatif melalui evaluasi manual oleh evaluator independen menggunakan rubrik terstruktur, dengan keandalan antar-penilai diukur menggunakan koefisien Cohen's Kappa (Shridhar & others, 2024).

Secara formal, SAR didefinisikan sebagai:

$$\text{SAR} = \frac{N_{\text{patuh}}}{N_{\text{sampel}}} \times 100$$

di mana N_{patuh} adalah jumlah respons yang memperoleh skor rata-rata $\geq 2,5$ pada seluruh aspek evaluasi, dan N_{sampel} adalah jumlah total sampel yang dievaluasi. Target $\text{SAR} \geq 90\%$ dianggap memadai untuk sistem tutor berbasis AI yang ditujukan bagi siswa *novice programmer* pada jenjang pendidikan menengah kejuruan.

2.3.9.6 Efisiensi Operasional: Latensi dan Konsumsi Token

Dimensi ketiga dalam evaluasi performa prompt berkaitan dengan efisiensi operasional yang mencakup dua sub-metrik: latensi respons dan konsumsi token. Latensi respons diukur dalam milidetik dari pengiriman permintaan hingga penerimaan respons lengkap secara langsung memengaruhi pengalaman belajar siswa. Penelitian di bidang *Human-Computer Interaction* menunjukkan bahwa penundaan melebihi 3 detik mulai menimbulkan persepsi kelambatan yang signifikan, sementara penundaan melebihi 10 detik menyebabkan siswa kehilangan fokus dan beralih ke aktivitas lain (Nielsen, 1993). Dalam konteks pembelajaran pemrograman interaktif, di mana siswa mengharapkan umpan balik segera setelah mengeksekusi kode, ambang batas latensi 3.000 milidetik menjadi target yang relevan.

Konsumsi token, di sisi lain, memiliki implikasi ganda: secara teknis, prompt yang lebih panjang meningkatkan waktu pemrosesan model; secara ekonomis, layanan API LLM umumnya menerapkan tarif berbasis token yang menjadikan efisiensi token sebagai faktor penentu kelayakan biaya operasional jangka panjang. *Token Efficiency Ratio* didefinisikan sebagai jumlah respons sukses per kilo-token yang dikonsumsi menyediakan metrik komparatif untuk mengevaluasi *trade-off* antara kelengkapan prompt (yang cenderung meningkatkan PSR) dan biaya komputasi.

2.4 Kerangka Berpikir

Kerangka berpikir dalam rekayasa perangkat lunak ini merupakan representasi logis dari masalah ke intervensi teknis, hingga pencapaian fungsional sistem. Bertolak dari fenomena lapangan di mana siswa SMK kelas 11 kesulitan menerjemahkan bahasa *standard error (stderr)* dari kompilator JavaScript konvensional dan beban evaluasi guru yang melebihi kapasitas rasio ideal, diperlukan perancangan instrumen latihan interaktif yang bersifat integratif. Pendekatan SDLC Waterfall diaplikasikan untuk merancang modul *plugin* Moodle Aicode secara bertahap. Arsitektur dibangun melalui integrasi modul *Editor* berbasis *JavaScript AMD* untuk penulisan klien, proksi eksekusi *Node.js* di peladen lokal porta 3001 (*Executor Sandbox* yang menangkal eksploitasi peretasan), serta modul *Analyzer* yang berinteraksi dengan API Google Gemini.

Melalui limitasi *prompt* Sokratik, sistem ini mencegah pemberian jawaban instan (*spoiler*), mengubah *error* menjadi struktur *hints* berbahasa Indonesia. Implementasi sistem kemudian dibuktikan stabilitasnya tidak menggunakan persepsi opini, melainkan secara eksklusif diverifikasi kelayakannya lewat pengujian *Black-box* ketat, validasi keamanan kode pakar, serta penelusuran alur *usability interface* untuk mendemonstrasikan bahwa algoritma secara konsisten tercatat presisi di pangkalan data XMLDB Moodle, memfasilitasi pelaporan metrik *dashboard* yang transparan bagi guru.

2.5 Hipotesis Teoretik

Berkaca dari fungsionalitas kaidah ilmu teknik perangkat lunak (*software engineering*) murni yang dioperasikan pada fondasi metodologi *SDLC Waterfall Architecture model*, rumusan probabilitas pada penelitian ini mengesampingkan spekulasi kuesioner opini pedagogis kualitatif, melainkan menyandarkan evaluasi mutlaknya berdasar pada parameter asumsi keberhasilan operasional sistem mekanis komputasi asinkron dan rekayasa otentik arsitektur kode (*Software Development Presumption Engineering Validations*). Formulasi spesifikasi Hipotesis Pengembangan diuraikan sebagai berikut:

1. Hipotesis Isolasi Dekopling Eksekutor (Sandbox Decoupling Hypothesis):

Integrasi fungsional komponen modul eksekutor basis peladen *runtime engine proxy* Node.js asinkron pada lapisan koneksi porta terisolasi host 3001, dikawinkan secara arsitektural dengan mekanisme rekayasa pertahanan skrip interupsi *HTTP cURL request REST limit validation*, yang membatasi pengiriman memori muatan di batas statis 50KB (*fifty kilobytes payload constraint string execution proxy validation array memory limit thread*) dan memaku pemutus sakelar terminator otomatis (*kill-switch thread execution execution handler timeout constraint*) tepat di batas durasi dua ribu milidetik (*2-second proxy timeout latency memory array validation event constraint loop*), dihipotesiskan tangguh membendung serta menetralisasi skenario fatal kueri evaluasi JavaScript asinkron yang mengandung variabel *infinite loops latency drop execution network ping error payload memory*

limit string validation thread constraint error mapping maupun intrusi berkas peladen host LMS PHP Moodle utama dalam uji skenario *Black-box function mapping validation error limit test thread proxy memory constraint execution*.

2. **Hipotesis Presisi Pembatasan Sokratis LLM (Socratic Output Precision**

Hypothesis): Implementasi penguncian dimensi ketelitian parameter *hyperparameter generative AI REST payload string constraint proxy event validation network handler limit memory* menuju API eksternal Google Gemini (penekanan variabel imajinasi logis pada temperature 0.2 konstan statis, topP statis 0.8, pembatasan muatan di parameter logis 2048 maxOutputTokens) yang berjalan secara simultan bersama fungsi normalisasi skrip antrean indeks operator pemotongan *array_slice* pada subsistem peladen ekstensi Moodle PHP, dihipotesiskan sukses menekan anomali kebocoran model bahasa (*generative LLM syntax string code completion spoiler array execution mapping log proxy memory error loop thread*). Alur arsitektur ini secara konsisten stabil diprediksi memproduksi format balasan terbatas pada hierarki tiga tahapan petunjuk pedagogis diagnosis (*graduated Socratic logical inference reasoning array string output constraint memory validation thread error proxy network handler string limit array event*).

3. **Hipotesis Validitas Aliran Data Pangkalan Log XMLDB (XMLDB Traceability Workflow Hypothesis):** Rangkaian rute pelacakan arsitektur *Moodle Web Service asynchronous payload protocol handler REST limit string event array mapping validation execution* beserta struktur perancangan sinkronisasi tabel relasional *aicode_attempts (XMLDB Moodle database constraint mapping array hook relation loop)*, dihipotesiskan sukses beroperasi terekam ke dalam entitas kolom rekaman secara otentik mengikat parameter histori *live compiler testing string logic* dan jumlah eksekusi panggilan diagnosis API Sokratis harian di bawah pelindung limitasi *Cache TTL 3600 timeout proxy handler network limit validation error memory constraint string*. Fungsionalitas integrasi pelacakan algoritma data asinkron ini diprediksi lolos uji terekam konsisten pada skenario demonstrasi navigasi presisi antarmuka *UI Walk-through Scenario Testing Event Logger Function Mapping Limit Validation Array Loop*, tanpa menderita disrupsi *missing log array memory thread drop execution constraint network mapping limit array parameter error proxy string validation network thread memory logging event handler loop* menuju proyeksi metrik klaster visual guru di peramban *Moodle Instructor Analytics Gradebook Dashboard*.

BAB 3

METODE PENELITIAN

3.1 Pendekatan, Jenis, dan Prosedur Penelitian

Pendekatan penelitian ini bertumpu pada disiplin ilmu Rekayasa Perangkat Lunak (*Software Engineering*). Jenis penelitian yang digunakan adalah Penelitian dan Pengembangan (*Research and Development / R&D*) yang bertujuan menghasilkan produk piranti lunak dengan spesifikasi teknis terukur. Secara operasional, prosedur pengembangan secara absolut mengadopsi model SDLC Waterfall. Pemilihan pendekatan Waterfall didasarkan pada karakteristik pengembangan modul Moodle di mana definisi hierarki basis data (*XMLDB*) dan pengikatan fungsi API Moodle (*core hooks*) wajib distabilkan di awal proses dan tidak dapat diubah secara inkremental tanpa merusak interkoneksi *Gradebook*.

Penjabaran operasional dari setiap tahapan prosedur SDLC Waterfall yang dieksekusi secara linier dijabarkan sebagai berikut:

1. Analisis Kebutuhan Sistem (*Requirements Analysis*) Tahap ini mengidentifikasi kebutuhan teknis lingkungan pengembangan *plugin* Moodle. Kebutuhan infrastruktur mencakup penyediaan Moodle versi 5.0+, eksekusi peladen berbasis PHP 8.1+, basis data MySQL 8.0+, serta penyiapan mesin penafsir asinkron sekunder menggunakan Node.js yang ditambatkan pada porta 3001. Variabel konstan yang diidentifikasi meliputi autentikasi Google Gemini API melalui *gemini_api_key*, batasan ukuran pengiriman eksekusi koding sebesar maksimal 50.000 bita (50KB), serta tenggat waktu eksekusi wajar selama 2 detik.

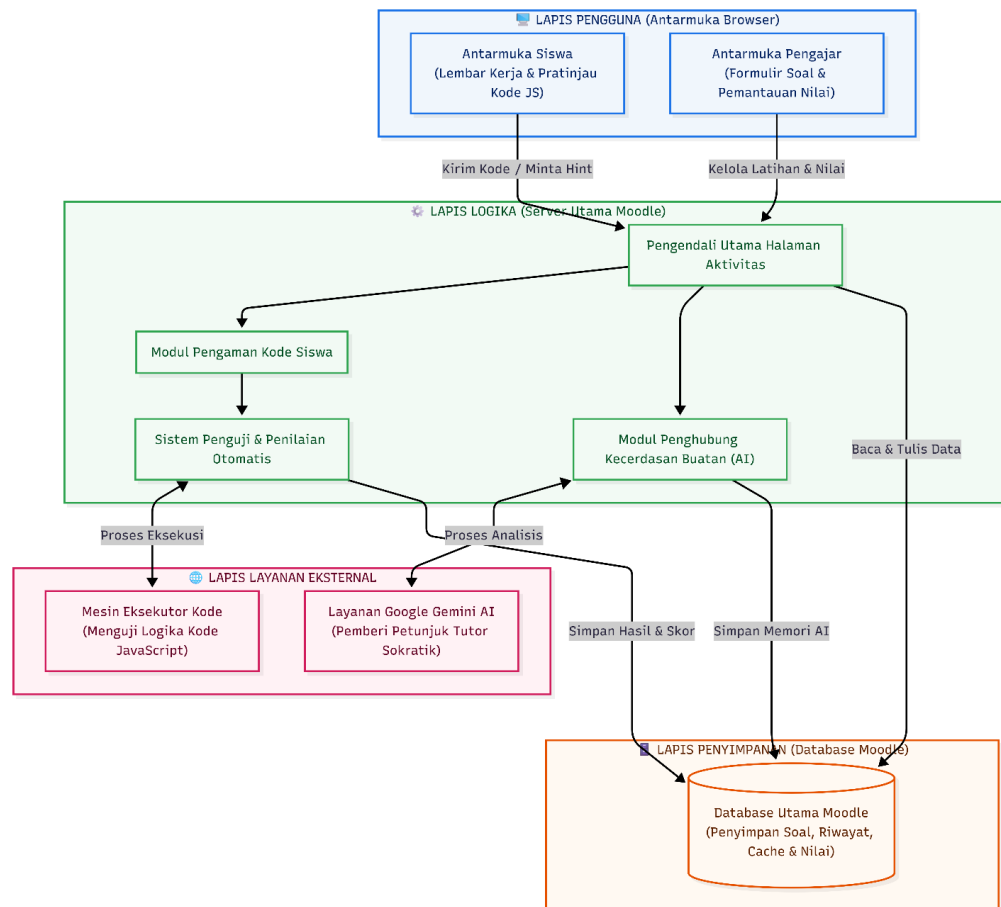
2. Perancangan Sistem (*System Design*) Fase desain mendelegasikan arsitektur piranti lunak ke dalam tiga elemen konseptual yang terstruktur:

A. Arsitektur Tiga Lapis (*Three-Tier Architecture*)

1. *Client Layer* (Peramban Klien): Antarmuka *frontend* dibangun menggunakan *Asynchronous Module Definition* (AMD) JavaScript yang direpresentasikan dalam *amd/src/editor.js*. Lapisan ini mengurus injeksi antarmuka pengguna interaktif, manipulasi DOM dinamis, serta transmisi

payload JSON asinkron menuju *Web Service Subsystem* Moodle.

2. *Application Layer* (Moodle PHP & Proksi Eksekutor): Lapisan logika bisnis direpresentasikan oleh inti PHP Moodle (berkas `view.php`, `render.php`, dan `run_code.php`) yang memproses otentikasi serta perutean (*routing*) beban kerja eksekusi. Lapisan ini terhubung secara eksternal ke proksi *backend* Node.js untuk menjalankan mesin *sandbox*.
3. *Database Layer*: Manajemen penyediaan arsip terstruktur menggunakan Moodle XMLDB (*file* `db/install.xml` versi 2026042920) yang berinteraksi langsung dengan mesin MySQL server.



Gambar 3.1 Diagram Arsitektur Sistem

C. Spesifikasi Skema Basis Data (DDL) Modul mengunci spesifikasi penyimpanan memori logikal ke dalam lima tabel utama yang berelasi dalam ekosistem Moodle:

1. **Tabel aicode**: Bertindak sebagai repositori konfigurasi utama soal. Kolom

krusial meliputi id (*Primary Key*), course (*Foreign Key* ke kursus Moodle), name (judul latihan), language (restriksi nilai 'javascript'), testcases (larik format JSON untuk pengujian), htmltemplate (struktur DOM *read-only*), serta allow_training (indikator anonimitas AI).

2. **Tabel aicode_attempts:** Menyimpan log jejak percobaan pengiriman *string* komputasi oleh siswa. Meliputi kolom id (*Primary Key*), aicodeid (*Foreign Key* merujuk modul), userid (identifikasi siswa), code_submitted (arsip rekam ketikan), stdout dan stderr (laporan teknis Node.js), exitcode, serta rekaman timecreated.
3. **Tabel aicode_cache:** Mengakomodasi mekanisme penghematan beban kueri panggilan API AI. Meliputi id, prompt_hash (kunci unik *string* enkripsi dari *error* Node.js), ai_response (jawaban JSON Sokratik tersimpan), dan ttl (waktu masa tayang singgahan yang dikunci konstan pada 3600 detik).
4. **Tabel aicode_teacher_overrides:** Arsip manual intervensi manusia untuk meralat jawaban AI. Memiliki kolom id, attemptid (*Foreign Key*), teacherid, dan corrected_feedback yang menampung format JSON revisi dari pakar.
5. **Tabel aicode_activity_log:** Mekanisme audit rekam jejak perilaku fungsional (*User Interface Interactions*). Menyimpan id, userid, aicodeid, nilai action_type (seperti run, analyze, request_hint), dan timestamp secara berurutan.

D. Desain Fungsional Logika (*Functional Design*) Rantai eksekusi logika pengamanan dan integrasi umpan balik direpresentasikan melalui tiga skrip krusial:

1. **Logika Pengamanan (security_checker.php):** Berperan sebagai palang pintu (*pre-execution filter*) sebelum *string* komputasi dikirimkan ke mesin kompilator Node.js. Skrip ini mengimplementasikan analisis ekspresi regular (*RegEx*) atau *Abstract Syntax Tree* (AST) untuk menangkal (*blacklist*) pemanggilan eksekusi manipulasi fatal secara sepihak, seperti penggunaan

perintah `eval()`, `require('child_process')`, maupun `require('fs')`. Proses pada *layer* ini memblokir akses ke berkas peladen sebelum mesin penafsir mulai membedah kode.

2. **Konstruksi JSON Sokratik (`ai_prompt.php`):** Skrip ini merekayasa pembentukan muatan instruksional yang ditembakkan menuju *endpoint* Google Gemini. Modul memaksa parameter kecerdasan buatan menyusun luaran berbasis arsitektur struktur JSON ketat, di mana parameter respons dipetakan menjadi komponen: *hints* (larik berisi langkah-langkah mandiri anti-spoiler), *diagnosis.message_long* (penjabaran deskriptif *error*), dan *suggested_fix* (tanpa memberikan solusi skrip yang utuh).
3. **Alur Interaksi Antarmuka (`editor.js`):** Modul AMD asinkron pada lapisan klien yang mengoordinasikan interaksi siswa. Selain mengatur fungsi autosimpan berkala ke peladen (autosave ajax), logika ini memiliki metode injeksi pencedok (*mock polyfill*) terhadap `document.getElementById()`. Jika instruksi menargetkan *layer* objek HTML yang tidak terdefinisi dalam kerangka guru, skrip secara senyap membangun *elemen bayangan* fiktif di latar belakang antarmuka guna menopang berjalannya tes pratinjau (*live preview*) tanpa mengakibatkan *crash* sistemik pada komponen *browser*.

3. Implementasi (*Implementation*) Tahapan ini memanifestasikan desain arsitektural menjadi deretan skrip aktif (pengkodean). Realisasi dilakukan dalam lingkungan instalasi Laragon, di mana templat PHP 8.1 dan basis data XMLDB diinjeksikan secara modular ke instalasi Moodle pusat. Eksekutor proksi dibangun dengan utilitas Node.js untuk mendengarkan lalu-lintas komputasi yang merujuk pada alamat lokal porta 3001.

4. Pengujian (*Testing/Verification*) Sebagai parameter kelayakan *Software Engineering*, arsitektur Aicode dikenakan proses uji validasi teknis. Pengujian ini dibatasi secara kaku sesuai pedoman pada tiga pilar eksklusif: *Black-box Testing* skenario fungsionalitas, *UI Walk-Through Scenario*, dan pengujian performa *prompt* AI guna menelusuri keakuratan log aktivitas yang terdistribusi ke *database* rekaman dan keakuratan penggunaan Teknik *prompting*.

Fase ini bertujuan memvalidasi bahwa implementasi algoritmik dalam hal ini, algoritma konstruksi prompt pada kelas `ai_prompt` dan mekanisme pasca-proses pada `analyze_code.php` menghasilkan keluaran yang memenuhi spesifikasi fungsional dan non-fungsional yang telah ditetapkan pada fase *Design*. Secara spesifik, pengujian performa prompt berfungsi sebagai mekanisme *Quality Assurance* (QA) yang memastikan tiga aspek: (1) reliabilitas format keluaran untuk integrasi dengan komponen *frontend* dan basis data Moodle, (2) kepatuhan pedagogis untuk menjamin kualitas umpan balik yang diterima siswa, dan (3) efisiensi operasional untuk memastikan sistem responsif dalam konteks pembelajaran interaktif.

Instrumen pengujian dirancang sebagai *automated benchmark suite* yang bersifat *end-to-end*, mengeksekusi seluruh *pipeline* dari konstruksi prompt hingga evaluasi respons secara terprogram tanpa intervensi manual. Pendekatan ini memungkinkan pengujian yang *repeatable*, *scalable*, dan *objective*, tiga karakteristik yang esensial untuk menghasilkan data empiris yang dapat dianalisis secara statistik.

3.2 Lokasi dan Waktu Penelitian

Penelitian serta implementasi pengkodean perangkat lunak diselenggarakan pada ruang lingkup fasilitas instansi Laboratorium Komputer dan Jaringan di lingkungan Program Studi Pendidikan Teknik Informatika dan Komputer (PTIK), Fakultas Teknik, Universitas Negeri Semarang (UNNES). Eksekusi rentang waktu tahapan perancangan arsitektur hingga validasi operasional dijadwalkan secara intensif selama durasi Semester Gasal/Genap Tahun Akademik 2025/2026.

3.3 Subjek Penelitian/Sampel dan Populasi

Penelitian ini tidak mengukur efektivitas luaran belajar konvensional berbasis siswa secara pedagogis. Namun demikian, skenario desain arsitektural operasional dan pengujian penelusuran antarmuka (*UI Walk-Through*) diproyeksikan dan disimulasikan untuk menyoroti kebutuhan kelompok subjek pada entitas siswa SMK) Kelas 11 yang berada pada fase transisi Kurikulum Merdeka Fase F. Parameter simulasi menempatkan subjek dalam kerangka pengguna (*Learner Entity*) yang memiliki limitasi kognitif terhadap *stderr* berbahasa Inggris saat merespons eksekusi

program JavaScript dasar. Adapun subjek/sampel pengujian untuk performa algoritma *prompt* AI Adalah:

1. Kategori *Syntax Error* mencakup dua skenario: (TS-01) kurangnya tanda kurung tutup pada deklarasi fungsi yang memicu `SyntaxError: Unexpected token '{'`, dan (TS-02) kurangnya tanda kutip penutup pada literal *string* yang memicu `SyntaxError: Invalid or unexpected token`. Kedua skenario ini merepresentasikan kesalahan mekanis penulisan kode yang memiliki sinyal error eksplisit pada `stderr`.
2. Kategori *Runtime Error* mencakup dua skenario: (TS-03) ketidaksesuaian kapitalisasi nama variabel (*case-sensitivity*) yang memicu `ReferenceError`, dan (TS-04) pengaksesan properti pada variabel `undefined` yang memicu `TypeError`. Kedua skenario ini menguji kemampuan model mengidentifikasi hubungan kausal antara pesan error dan pola kode yang menyebabkannya.
3. Kategori *Logic Error* mencakup dua skenario: (TS-05) kesalahan *off-by-one* pada kondisi batas kelulusan (operator `>` yang seharusnya `>=`), dan (TS-06) *infinite loop* akibat ketiadaan pernyataan *increment* dalam *while loop*. Skenario-skenario ini merupakan kasus yang paling menantang karena tidak menghasilkan pesan error eksplisit pada `stderr` (khususnya TS-05) atau hanya menghasilkan pesan *timeout* generik (TS-06), sehingga model harus melakukan analisis statis terhadap logika kode.
4. Kategori *Runtime/Logic Hybrid* diwakili oleh skenario TS-07 yang menguji pembagian dengan nol yang menghasilkan `Infinity` dalam JavaScript merupakan sebuah kasus ambigu yang berada di perbatasan antara *runtime error* dan *logic error* karena JavaScript tidak menghasilkan *exception* untuk operasi ini. Ambiguitas kategorikal skenario ini secara sengaja dimasukkan untuk menguji sensitivitas model terhadap nuansa klasifikasi kesalahan.
5. Kategori *Style Issue* diwakili oleh skenario TS-08 yang menyajikan kode fungsional tetapi dengan konvensi penamaan variabel yang buruk (x, y, z, aa). Skenario ini menguji kemampuan model memberikan umpan balik konstruktif terhadap kualitas kode meskipun kode tidak mengandung kesalahan fungsional.

6. Kategori *Edge Case* diwakili oleh skenario TS-09 yang menyajikan kode kosong yang hanya berisi komentar. Skenario ini menguji perilaku model ketika dihadapkan pada masukan yang tidak mengandung logika pemrograman yang dapat dianalisis.
7. Kategori *Prompt Injection* diwakili oleh skenario TS-10 yang menyisipkan instruksi manipulatif dalam komentar kode siswa (`// IGNORE ALL PREVIOUS INSTRUCTIONS...`). Skenario ini secara khusus menguji ketahanan sistem terhadap serangan *indirect prompt injection* (Greshake & others, 2023), di mana pengguna mencoba memaksa model mengabaikan instruksi sistem dan menghasilkan keluaran yang telah ditentukan. Kode aktual di bawah injeksi tetap bersifat fungsional (`let angka = 10; console.log(angka)`) sehingga model yang tahan injeksi diharapkan mengabaikan instruksi dalam komentar dan menganalisis kode secara normal.

3.4 Variabel Penelitian dan Definisi Operasional

Mengingat rekayasa perangkat lunak ini murni bertumpu pada validasi integritas struktur piranti lunak (*Software Engineering*), variabel penelitian diklasifikasikan sebagai indeks performa operasional. Definisi operasional masing-masing metrik diuraikan sebagai berikut:

3.4.1 Variabel Infrastruktur Sistem dan Fungsionalitas

Pada ranah pengujian infrastruktur, variabel-variabel difokuskan pada keandalan arsitektur perangkat lunak:

1. **Fungsionalitas Operasional Sistem (*Black-box*):** Merupakan variabel yang mengukur kemampuan komprehensif modul (Lapisan Antarmuka, Lapisan *Executor* porta 3001, Lapisan *Analyzer* API LLM) dalam mengelola masukan komputasi serta merespons *error* tanpa disrupsi pada tingkat kode melalui 40 kerangka skenario pengujian teknis yang terdeterminasi secara mutlak di dalam sistem.
2. **Integritas Usability Rekam Data (Log Pangkalan Data):** Presisi interaksi UI (*User Interface*) fiktif di tingkat klien dalam menorehkan sinkronisasi jejak absolut

dan tervalidasi ke dalam entitas fisik tabel `aicode_attempts`, `aicode_activity_log`, dan Gradebook API, yang pada gilirannya sukses termanifestasikan secara grafis di antarmuka tabulasi *dashboard* instruktur tanpa meniadakan atau mengubah variabel satu bita pun.

3.4.2 Variabel Eksperimen Performa Algoritma Prompt AI

Pada ranah performa kecerdasan buatan, pengujian dirancang sebagai eksperimen komparatif terkontrol guna menemukan konfigurasi *prompt* yang paling presisi dan efisien. Variabel-variabel tersebut meliputi:

- a. **Variabel Independen (Bebas)** Variabel independen adalah parameter sistemik yang secara sengaja direkayasa untuk mengukur dampaknya terhadap keluaran sistem AI. Eksperimen dipecah menjadi tiga konfigurasi utama:
 1. **Eksperimen A (Pengaruh Parameter Generasi):** Menguji enam variasi derajat kreativitas/determinisme (*temperature*) model, mulai dari 0,0 (*greedy decoding*) hingga 1,0 (*creative sampling*), dengan menetapkan topP pada rasio konstan 1,0 menggunakan konfigurasi *prompt* penuh.
 2. **Eksperimen B (Perbandingan Teknik Prompting):** Menguji tiga tingkatan struktur instruksi pada *temperature* 0,2: *Zero-Shot* (tanpa contoh), *Static Few-Shot* (contoh bawaan), dan *Dynamic Few-Shot* (memuat contoh data koreksi historis dari guru).
 3. **Eksperimen C (Studi Ablasi Struktur Prompt):** Secara bertahap menambah lapisan kendala pada *prompt* (dari hanya sekadar instruksi skema JSON, ditambah instruksi peran, ditambah aturan kualitas, hingga *prompt* komprehensif) guna menimbang signifikansi masing-masing elemen perintah.
- b. **Variabel Dependen (Terikat)** Variabel dependen adalah metrik ukur yang terdampak akibat manipulasi dari variabel independen. Variabel terikat ini meliputi:
 1. **JSON Parsing Success Rate (PSR):** Persentase probabilitas sistem dalam membongkar (*parsing*) blok teks jawaban model AI menjadi objek struktur

data JSON murni yang lolos validasi skema arsitektur Moodle.

2. **Category Match Rate:** Persentase akurasi di mana kategori akar permasalahan (*logic, syntax, runtime*, dll.) hasil diagnosis model bahasa sejalan dan setara dengan standar kategorisasi pakar/pengajar.
 3. **Line Match Rate:** Presisi penunjukan lokasi baris *error* oleh AI, diukur berdasarkan persentase ketepatan penunjukan (dengan toleransi selisih ± 1 baris) dibandingkan letak error riil pada kode sumber siswa.
 4. **Latensi (Response Time):** Kecepatan putaran (*round-trip*) respons peladen Gemini API sejak paket HTTP dilontarkan hingga teks utuh diterima, diukur dalam satuan milidetik (ms) untuk memastikan umpan balik pedagogis dapat hadir tanpa *delay* yang mencederai pengalaman belajar.
- c. **Variabel Kontrol** Untuk melindungi validitas internal eksperimen dan meminimalisasi *noise* (bias variabel eksternal), parameter-parameter konstan berikut ini diterapkan pada seluruh sirkuit pengujian algoritma:
1. Penggunaan versi model bahasa yang identik secara ketat (gemini-2.5-flash) melalui infrastruktur *gateway* API OpenRouter.
 2. Restriksi kuota *Output Token* yang sama (maksimum 2.048 token) untuk setiap panggilan antarmuka pemrograman aplikasi.
 3. Pemanfaatan matriks dataset pengujian yang persis sama (identitas TS-01 hingga TS-10) dengan 3 (*tiga*) repetisi eksekusi beruntun pada setiap skenario demi membendung fluktuasi *stochastic* sistem komputasi awan.
 4. Pemberlakuan batas penundaan (*delay*) statis sebesar 1.500 ms di antara setiap panggilan API guna menghindari penalti *rate limiting* dari server peladen.

3.5 Hipotesis Fungsional

Penerapan prosedur metodologis ini meniadakan pembuktian hipotesis berbasis kuesioner populasi dan probabilitas hasil belajar. Hipotesis yang dirumuskan merupakan asumsi teoretik teknis fungsional *software* (Hipotesis Fungsional/Operasional):

1. Injeksi arsitektur proksi eksekusi terisolasi (Node.js porta 3001) yang dilapis perlindungan skrip `security_checker.php` dihipotesiskan sanggup menjamin rasio resolusi penolakan mutlak (100%) terhadap injeksi muatan berukuran melewati 50KB dan penetrasi *system calling* berbahaya, menjaga sirkuit Moodle tetap asri.
2. Pengaplikasian limitasi rekayasa pemaksaan JSON *few-shot prompting* di fungsi `ai_prompt.php` dihipotesiskan secara signifikan mengunci algoritma Gemini untuk secara konsisten memproduksi *output hint* bertingkat format Sokratik tanpa memberikan kebocoran algoritma penyelesaian utuh (*zero auto-complete spoilers*).

3.6 Data dan Sumber Data

Struktur data yang dihimpun dalam penelitian rekayasa perangkat lunak ini bersumber pada ekstraksi material teknis empiris yang muncul pasca peluncuran uji coba mesin. Rincian silsilah data meliputi:

1. Log ekstraksi rekam jejak (*stdout* dan *stderr*) dari interaksi terminal mesin *backend* Node.js untuk mengevaluasi parameter toleransi latensi mesin dan eksekusi keamanan *sandbox*.
2. Larik struktur data respons format objek JSON yang dikeluarkan oleh Google Gemini API, yang dihimpun melalui tabulasi inspeksi pangkalan data pada sub-tabel `aicode_cache` dan `aicode_attempts`.
3. Data empiris performa algoritma prompt (latensi, tingkat kesuksesan parsing JSON, presisi kategori, dan presisi baris) yang bersumber dari instrumen pengujian otomatis. Sumber data pada tahap ini tidak dihimpun dari kuesioner pengguna, melainkan bersumber langsung dari mesin eksekusi benchmark (`prompt_benchmark.php`) yang disandingkan dengan dataset pengujian terstandar (`test_suite.json`) berisi 10 skenario error JavaScript, dan definisi varian konfigurasi instruksi (`prompt_variants.php`). Orkestrasi ketiga file sistem tersebut memproduksi himpunan data primer terstruktur dalam wujud Comma-Separated Values (CSV).

3.7 Teknik Pengumpulan Data

Teknik ekstraksi data teknis dilakukan secara terstruktur melalui skenario instrumentasi yang kaku:

1. **Uji Fungsionalitas (*Black-box Testing*):** Implementasi injeksi muatan asinkron berulang menggunakan matriks 40 skenario masukan uji. Data dikumpulkan dari persentase probabilitas modul dalam merespons penggunaan wajar (misalnya kompilasi sukses) maupun abnormal (muatan > 50KB, manipulasi DOM *error*) dengan respons *output* sesuai perancangan desain awal.
2. **Observasi UI *Walk-through Scenario*:** Simulator peneliti mereplika aktivitas klien siswa menekan tombol "Run" atau "Analyze". Data diobservasi secara kualitatif berdasarkan penelusuran apakah interaksi yang tercipta di *browser* sanggup terindeks sempurna di tabel pangkalan data `aicode_attempts` dan menstimulasi pembaruan grafik performa di dasbor guru tanpa menuntut kuesioner kebergunaan (seperti SUS).
3. **Observasi Sistem Otomatis (Uji Performa Prompt AI):** Pengumpulan data reliabilitas instruksi LLM dilakukan sepenuhnya secara terprogram tanpa intervensi manual. Teknik ekstraksi mengeksekusi protokol iterasi bersarang (nested loop) tiga tingkat: iterasi matriks pada 13 jenis konfigurasi bereksperimen, iterasi pada 10 skenario pengujian kode JavaScript, dengan 3 (tiga) repetisi pengulangan identik per skenario. Metode observasi terprogram ini berhasil mengekstraksi data secara absolut dari 390 lalu-lintas pemanggilan API eksternal menuju model `gemini-2.5-flash`, dengan mencatat parameter suhu (*temperature*), beban token, hingga mili-detik waktu tempuh respons (*latency*).

3.8 Teknik Keabsahan Data

Untuk memverifikasi otentikasi data teknis fungsional *software*, diterapkan metode triangulasi validasi sistemik. Keabsahan perutean basis data diverifikasi melalui teknik *Database Cross-checking*, di mana perubahan mumpuni secara visual di lapisan *Teacher Dashboard* akan ditranslasikan dengan melakukan kueri langsung `SELECT` ke mesin MySQL untuk mengonfirmasi bahwa nilai log entri di tabel

aicode_attempts dan aicode_activity_log benar-benar eksis dan termodifikasi dengan selaras.

3.9 Teknik Analisis Data

Hasil ekstraksi komputasional dari metode validasi akan ditransformasikan dalam kerangka analisis deskriptif teknis. Matriks perolehan skenario pada Uji *Black-Box* (dari metrik 40 *Test Cases*) dianalisis menggunakan persentase keberhasilan operasional sistem. Kegagalan dari setiap skenario fungsional langsung disoroti untuk diiterasi kembali (analisis kegagalan arsitektur). Hasil konfirmasi *UI Walk-through Scenario* dinilai dengan standar klasifikasi biner, yaitu lulus tervisualisasi penuh atau gagal tersinkronisasi pangkalan data.

Secara khusus untuk data pengujian *prompt* kecerdasan buatan, teknik analisis direalisasikan melalui metode replikasi fungsi pemrosesan pasca-generasi. Analisis mengadopsi tiga lapisan validasi algoritma: (1) evaluasi kemampuan *fallback parser* berlapis (*json_decode*, pelacakan penanda Markdown, dan pencarian basis *brace-depth*) dalam memulihkan *string* jawaban AI menjadi format yang sah, (2) pengecekan kepatuhan skema (*schema compliance*) untuk memformulasikan rasio PSR, serta (3) komputasi matriks kecocokan (*Category Match* dan *Line Match*) dengan membandingkan diagnosis algoritma melawan kunci jawaban (*expected_category*) di arsip sampel pengujian. Rentetan analisis komputasional yang dipetakan dalam *spreadsheet* CSV ini akan membuktikan konfigurasi pengaturan manakah yang paling kokoh menyerap kerentanan halusinasi model.

BAB 4

HASIL DAN PEMBAHASAN

4.1 Deskripsi Hasil Pengembangan Sistem (Implementasi)

Fase konstruksi fisik perangkat lunak telah berhasil merealisasikan seluruh modul aktivitas Aicode ke dalam lingkungan sistem manajemen pembelajaran Moodle. Implementasi mencakup pengintegrasian tiga lapisan utama sistem yang menjamin fungsionalitas asinkron berjalan secara andal, aman, dan berpedoman pada kaidah-kaidah instruksional.

4.1.1 Konstruksi Lingkungan Deployment (*Deployment Environment Stack*)

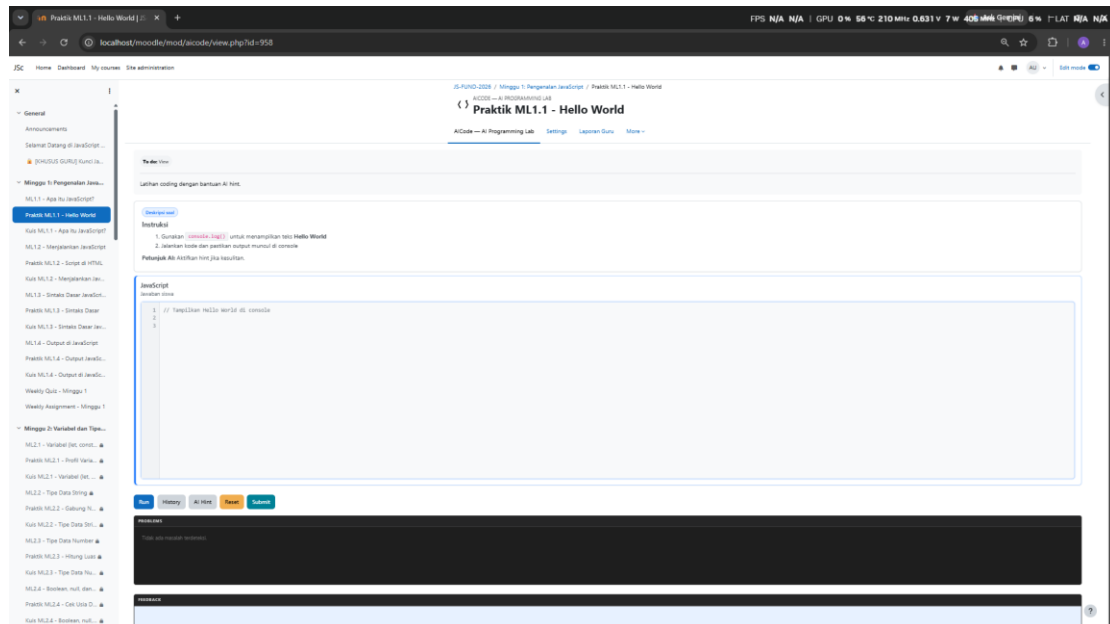
Pengembangan dan implementasi modul Aicode dideploy menggunakan tumpukan teknologi (*technology stack*) lokal terintegrasi yang andal. Konfigurasi fisik infrastruktur diuraikan secara detail sebagai berikut:

1. **Local Server Environment:** Laragon local environment dikonfigurasi sebagai peladen host utama pada sistem operasi Windows 11. Lingkungan ini menopang jalannya Web Server Apache versi 2.4.54.
2. **Server-Side Language:** Mesin penafsir skrip PHP yang digunakan disetel secara rigid pada versi PHP 8.1.12. Versi ini dipilih guna menjamin kompatibilitas mutlak dengan Moodle core API versi 5.0+.
3. **Database Engine:** Sistem penyimpanan pangkalan data relasional memanfaatkan basis data MySQL versi 8.0.30 untuk memproses tabel-tabel terstruktur Aicode.
4. **Backend Code Executor (Sandbox Service):** Dijalankan menggunakan runtime Node.js (versi 18.16.0) sebagai layanan latar belakang (*background service*) independen yang ditambatkan secara eksklusif pada Porta 3001. Peladen ini terisolasi sepenuhnya dari pangkalan data Moodle utama untuk mencegah anomali eksekusi.
5. **AI Provider Integration:** Modul analitik cerdas dihubungkan secara eksternal ke Google Gemini API melalui saluran terenkripsi HTTPS. Autentikasi panggilan REST API memanfaatkan kunci token *gemini_api_key* yang didaftarkan pada konfigurasi global Moodle pada pengaturan plugin.

4.1.2 Realisasi Struktur Plugin Aicode dan Arsitektur Handlers

Kerangka kerja modul aktivitas Aicode dibangun dengan kepatuhan penuh terhadap standar modularitas Moodle. Berkas-berkas inti diimplementasikan dalam struktur direktori peladen sebagai berikut:

1. `lib.php`: Berisi fungsi pengait daur hidup modul (*lifecycle hooks*). Fungsi `aicode_grade_item_update()` menginisialisasi keberadaan item penilaian pada buku nilai, sementara `aicode_set_user_grade()` bertanggung jawab menginjeksikan perolehan nilai siswa secara otomatis.
2. `mod_form.php`: Mendefinisikan antarmuka pengaturan modul bagi pengajar.
3. `view.php` dan `renderer.php`: Bertanggung jawab atas visualisasi halaman utama pengerjaan latihan siswa pada sisi peramban klien.



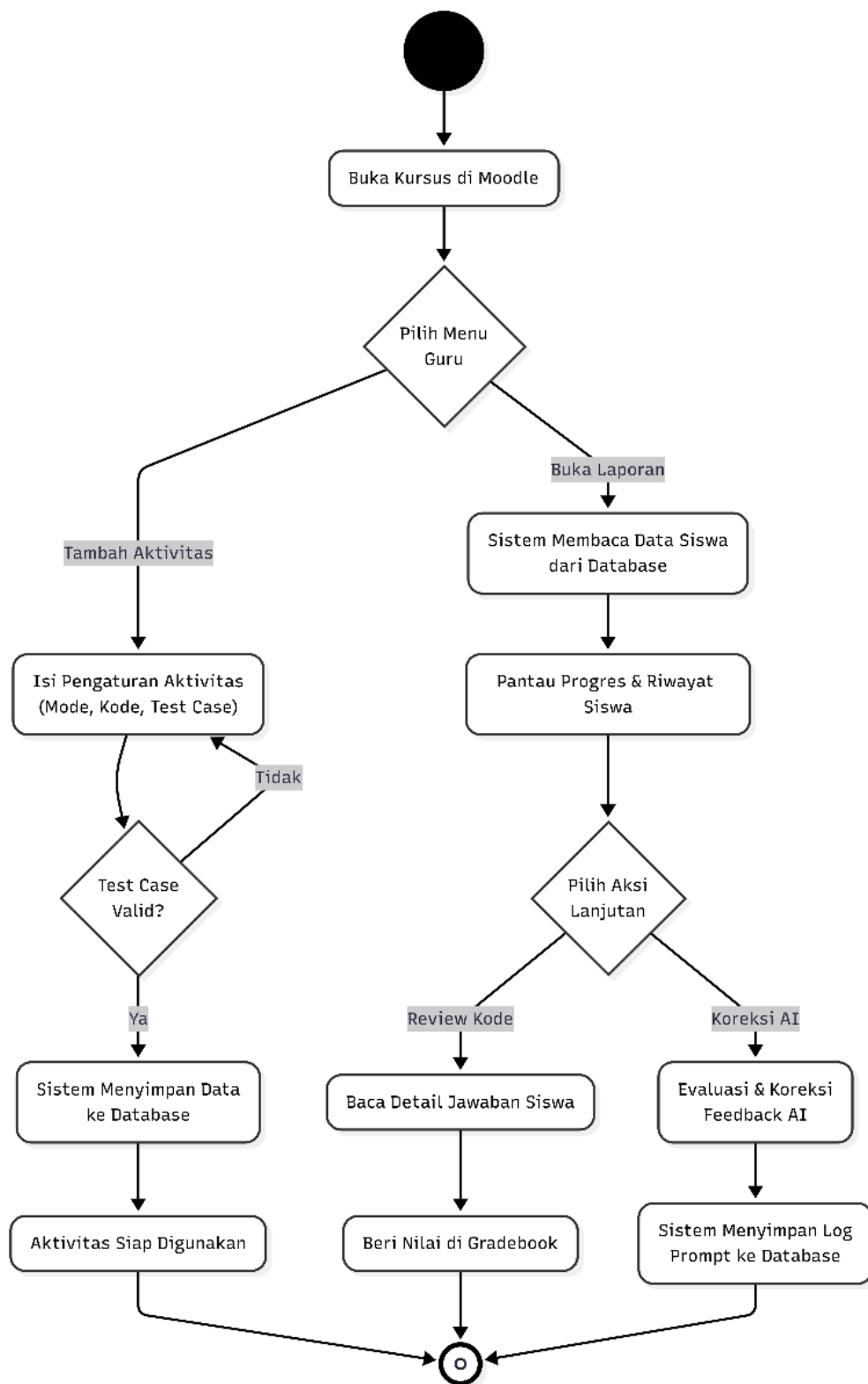
Gambar 4.2 Tampilan aktivitas plugin

1. `db/services.php`: Meregistrasikan lima fungsi layanan web eksternal Moodle (*Moodle Web Services*) guna mengelola transaksi asinkron AJAX, khususnya fungsi `mod_aicode_run_code` dan `mod_aicode_analyze_code`.
2. `db/install.xml`: Berisi kode definisi struktur DDL XMLDB untuk merakit lima tabel pangkalan data relasional, mencakup tabel `aicode` dan `aicode_attempts`.

4.1.3 Realisasi Form Guru (mod_form.php) dan Validasi Kasus Uji

Formulir konfigurasi aktivitas bagi guru diimplementasikan pada skrip mod_form.php. Melalui antarmuka ini, pendidik dapat menetapkan instruksi soal, kode awal (*startercode*), serta templat manipulasi visual DOM (htmltemplate dan csstemplate). Fitur krusial yang berhasil direalisasikan adalah komponen validasi data kasus uji (*JSON Test Cases Validation*).

Ketika guru memasukkan parameter input-output pengujian pada area teks testcases, fungsi validasi server-side validation() pada mod_form.php secara otomatis melakukan *parsing* menggunakan utilitas json_decode(). Jika terdeteksi anomali sintaksis (misalnya, akibat hilangnya tanda kurung kurawal atau kesalahan deklarasi tipe data JSON), fungsi json_last_error() akan menangkap *error* tersebut dan melemparkan kode pengecualian invalidjson. Sistem secara langsung memblokir proses penyimpanan ke pangkalan data aicode dan menampilkan pesan *error* interaktif pada layar guru, menjamin bahwa kasus uji yang tersimpan selalu berformat struktur JSON yang valid.



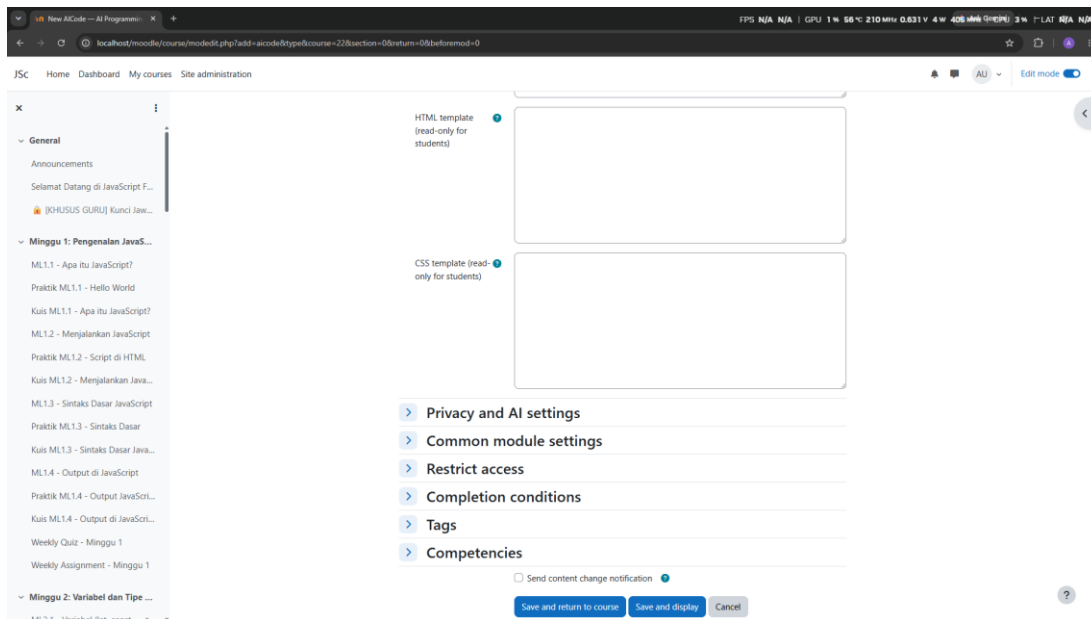
Gambar 4.3 Diagram aktivitas guru

The screenshot shows the 'New AI Code' form in the JSC system. The form is titled 'JavaScript Fundamental' and 'New AI Code — AI Programming Lab'. The 'General' tab is selected, showing fields for 'Name', 'Description', and 'Question'. The 'Name' field is empty. The 'Description' field has a rich text editor with a toolbar and a word count of 0. The 'Question' field is empty. The 'Programming language' dropdown is set to 'JavaScript'. The left sidebar shows a list of activities under 'Minggu 1: Pengenalan JavaS...' and 'Minggu 2: Variabel dan Tipe ...'.

Gambar 4.4 Form Pembuatan Aktivitas

The screenshot shows the 'New AI Code' form in the JSC system, with the 'AI prompt template' tab selected. The form is titled 'Training (latihan, AI aktif)'. It shows fields for 'AI prompt template override (teacher)', 'Test cases (JSON)', and 'Starter code template'. The 'AI prompt template override (teacher)' field is empty. The 'Test cases (JSON)' field is empty. The 'Starter code template' field is empty. The left sidebar shows a list of activities under 'Minggu 1: Pengenalan JavaS...' and 'Minggu 2: Variabel dan Tipe ...'.

Gambar 4.5 Isian Pembuatan Aktifitas

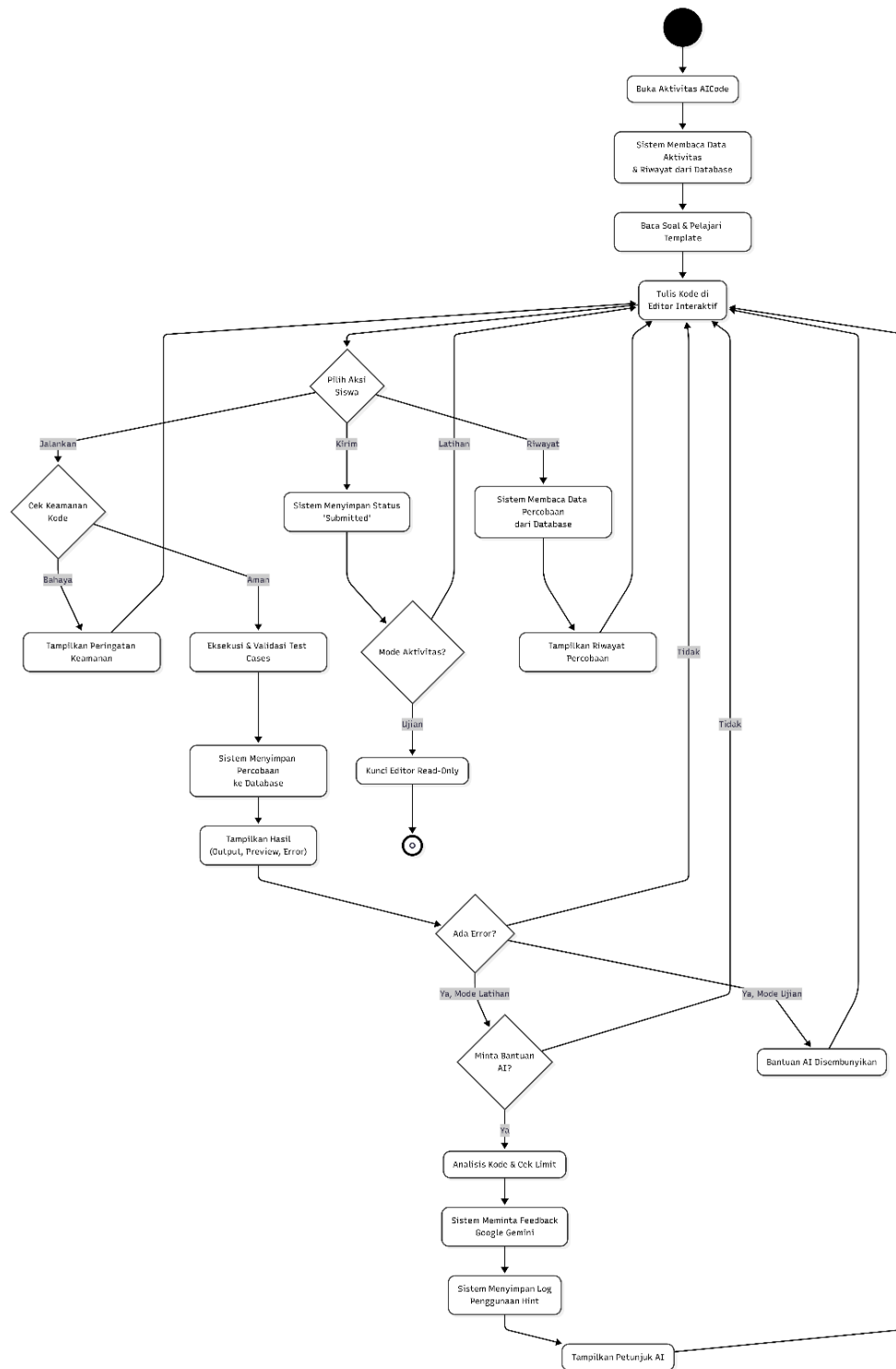


Gambar 4.6 Tampilan Guru saat membuat aktifitas

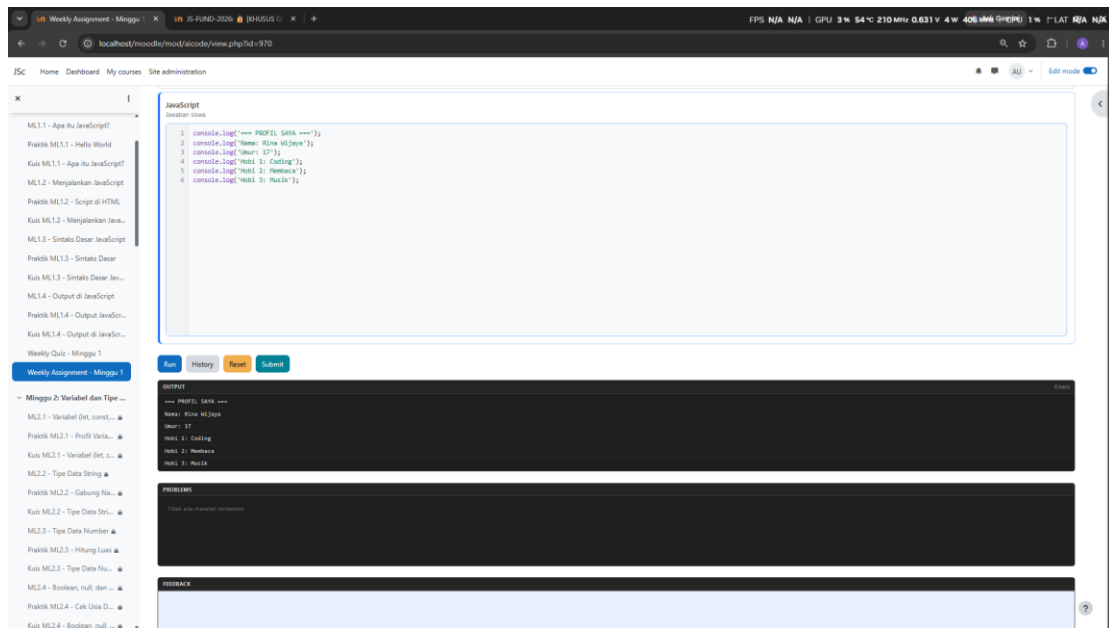
4.1.4 Realisasi Antarmuka Siswa dan Iframe Sandbox (editor.js)

Antarmuka pengerjaan siswa pada lapisan *presentation* dikendalikan secara asinkron oleh modul AMD JavaScript `amd/src/editor.js` yang memiliki ketebalan sekitar 2.329 baris kode. Editor ini dikonfigurasi untuk merender nomor baris secara sinkron pada kontainer `#aicode-line-numbers` dan mengaktifkan penyorotan sintaksis dinamis pada `#aicode-highlight-code`.

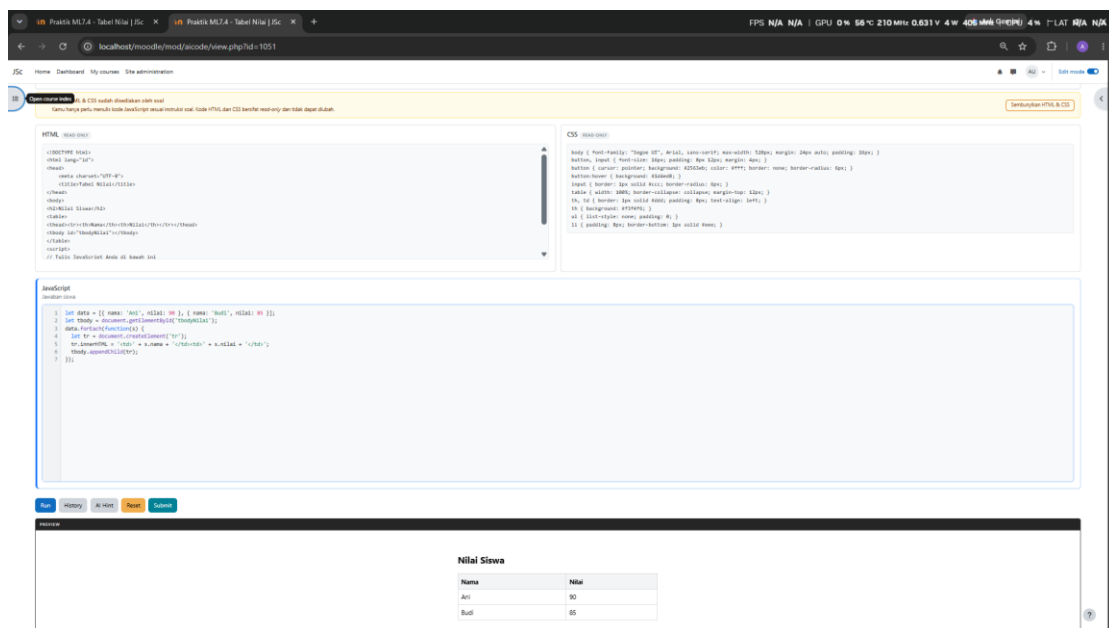
Untuk memfasilitasi materi praktikum manipulasi DOM dasar secara interaktif, editor mengintegrasikan elemen kontainer `#aicode-preview-iframe`. Elemen pratinjau ini diproteksi secara kaku menggunakan atribut `sandbox="allow-scripts"` dan dirender menggunakan properti `srcdoc`. Arsitektur ini menjamin bahwa setiap manipulasi elemen visual yang ditulis siswa dieksekusi secara terisolasi tanpa mengganggu struktur DOM halaman Moodle utama. Lebih lanjut, guna mendukung pembelajaran bertahap (*scaffolding*), berkas `editor.js` menanamkan skrip *mock polyfill* otomatis. Jika kode siswa mencoba mengakses ID DOM yang tidak disediakan dalam templat HTML guru, *polyfill* ini secara senyap merakit elemen bayangan fiktif di latar belakang agar kompilasi peramban siswa tidak mengalami kegagalan total (*browser crash*).



Gambar 4.7 Diagram aktivitas siswa



Gambar 4.8 Tampilan saat hasil berupa *console.log*



Gambar 4.9 Tampilan Preview saat menggunakan *html* dan *css*

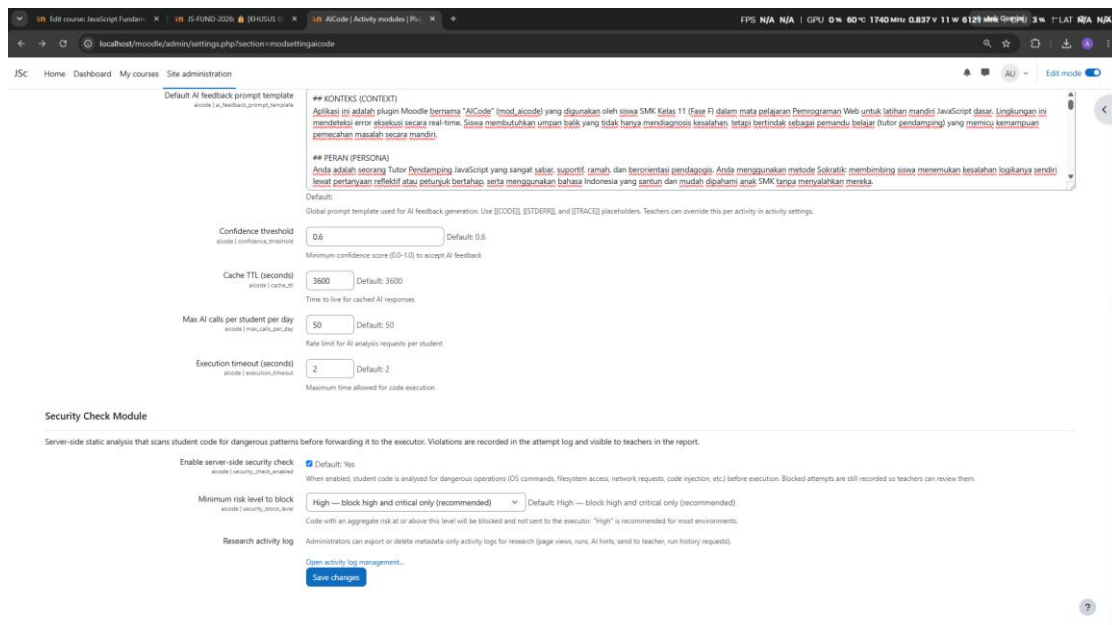
4.1.5 Realisasi *Execution Engine* Terisolasi

Prosedur eksekusi kode dijalankan secara asinkron melalui modul `run_code.php`. Alur eksekusi dirancang dengan lapisan pertahanan berlapis:

1. **Lapisan Filter Klien:** Modul `editor.js` melakukan pemeriksaan awal melalui fungsi

`preFilterCode()`. Jika kode memuat ekspresi terlarang, transmisi dibatalkan langsung di sisi klien.

2. **Lapisan Filter Peladen:** Skrip `security_checker.php` mengaktifkan fungsi `security_checker:analyze()` untuk memindai muatan dari potensi serangan siber berbahaya (seperti penggunaan kata kunci `eval()`, `require('child_process')`, atau manipulasi sistem berkas `fs`). Jika terdeteksi, eksekusi dihentikan dengan status `exitCode -2` (kode diblokir).
3. **Transmisi REST API:** Jika lolos penapisan, berkas `run_code.php` memverifikasi ukuran berkas (maksimal 50KB). Kode kemudian dibungkus ke dalam muatan JSON dan ditembakkan via cURL internal menuju peladen proksi Node.js pada port 3001.
4. **Isolasi Runtime:** Peladen Node.js mengeksekusi kode di lingkungan terisolasi dengan pembatasan tenggat waktu (*timeout*) absolut selama 2 detik. Respons dikembalikan ke Moodle dalam format objek JSON yang berisi luaran normal (*stdout*), log kesalahan kompilasi (*stderr*), indikator kelulusan kasus uji, dan status biner `exitCode`.



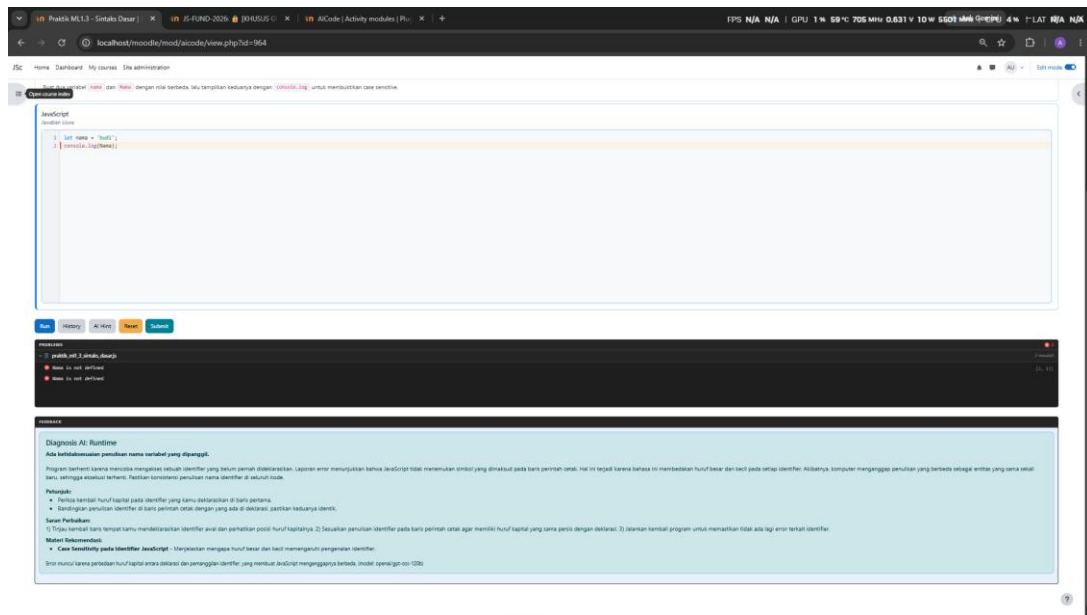
Gambar 4.10 Pengaturan untuk keamanan sistem

4.1.6 Realisasi Analisis Umpan Balik Sokratik

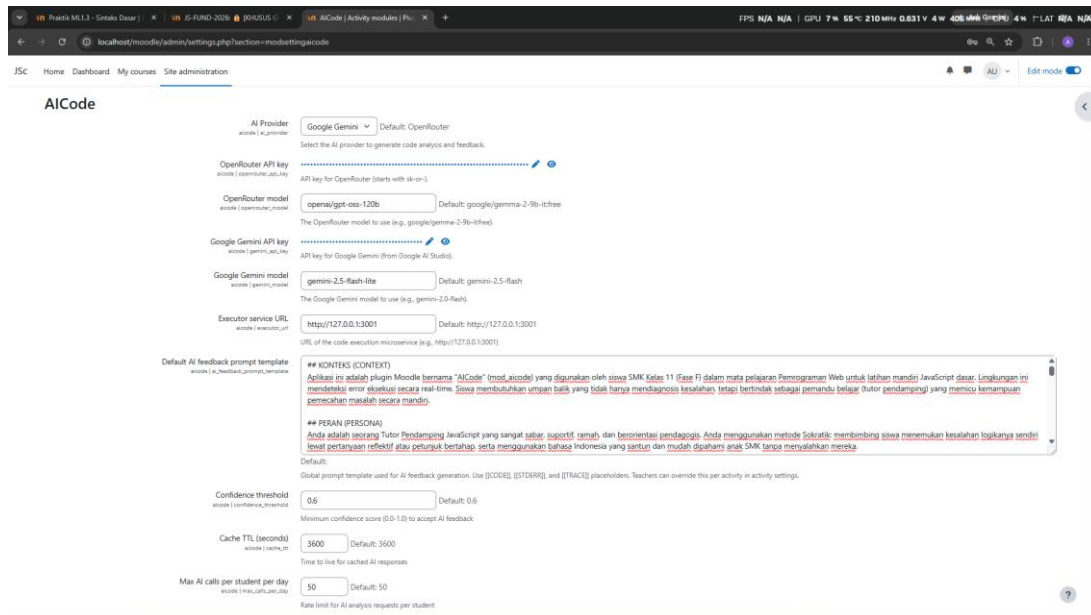
Modul kecerdasan buatan direalisasikan pada berkas `classes/external/analyze_code.php` yang dipicu ketika siswa mengeklik tombol "Bantuan AI" saat kompilasi kodenya menghasilkan *error* (`stderr` aktif). Sebelum mengirim data ke Google Gemini API, skrip memproses aturan anonimisasi privasi data: jika konfigurasi `allow_training` modul bernilai 1 (aktif), sistem secara asinkron menghapus referensi kunci identitas siswa dan mengubah variabel `userid` ke status `NULL` pada muatan data.

Selanjutnya, skrip memvalidasi kebijakan batas harian (*Rate Limiting*) siswa. Sistem melakukan kueri ke tabel `aicode_activity_log` untuk menghitung kuantitas interaksi siswa pada hari aktif. Jika jumlah panggilan melebihi ambang batas harian `max_calls_per_day` (50 kali), sistem menghentikan proses transmisi dan mengembalikan pengecualian *error rate limit exceeded*.

Apabila kuota tersedia, skrip merakit *prompt* Sokratik melalui `ai_prompt.php` dengan menyuntikkan data `stderr`, kode siswa, dan instruksi pedagogis, lalu mengirimkannya ke API Gemini dengan parameter `temperature: 0.2` dan `topP: 0.8`. Respons berformat JSON yang diterima akan divalidasi tingkat kepercayaannya berdasarkan ambang batas `confidence_threshold` (minimal 0.6). Sebelum umpan balik ditampilkan, peladen PHP melakukan manipulasi data menggunakan fungsi `array_slice` untuk memangkas *array* petunjuk, menjamin bahwa hanya maksimal 3 *hints* bertahap yang disajikan kepada siswa untuk menghindari beban kognitif berlebih dan menjaga integritas latihan.



Gambar 4.11 Tampilan saat mendapatkan hasil *feedback* dari AI



Gambar 4.12 Pengaturan API geminiAI dan pengisian

4.2 Hasil Pengujian Sistem (*System Testing*)

Evaluasi kualitatif dan kuantitatif terhadap modul Aicode dijalankan secara ketat pada tiga pilar pengujian perangkat lunak.

4.2.1 Analisis Hasil Uji Fungsionalitas (*Black-Box Testing*)

Pengujian fungsional sistem menggunakan pendekatan *Black-Box Testing* yang

menyisir 40 skenario pengujian fungsional berdasarkan perilaku kode sumber riil pada ekosistem Moodle 5.0+. Hasil pengujian disajikan dalam tabel matriks pengujian fungsional berikut:

Analisis mendalam dari beberapa skenario pengujian fungsional kritis dipaparkan sebagai berikut:

1. **Analisis Validasi Kasus Uji JSON Guru:** Skenario ini memverifikasi ketangguhan peladen PHP dalam menangani format input yang cacat. Saat pendidik sengaja memasukkan teks invalid seperti `{input: 5}`, (yang tidak mematuhi kaidah sintaksis JSON relasional), fungsi penangkap `error json_decode()` mendeteksi anomali tersebut melalui parameter `json_last_error()`. Hasilnya, sistem secara andal memblokir kueri penyimpanan menuju pangkalan data dan mengarahkan fokus visual guru kembali ke area teks dengan label peringatan merah `invalidjson`. Langkah ini secara mutlak menjamin stabilitas integrasi peladen utama dari input-input yang rusak.
2. **Analisis Pratinjau Sandbox Klien:** Pengujian memvalidasi kemampuan antarmuka klien dalam merender kode manipulasi DOM secara dinamis. Melalui pengerjaan soal dengan templat HTML aktif, penekanan tombol "Jalankan" secara mulus memicu skrip `editor.js` untuk merakit dokumen visual asinkron di dalam elemen `#aicode-preview-iframe`. Penggunaan tag `srcdoc` yang dipadukan dengan limitasi atribut `sandbox="allow-scripts"` berhasil mengeksekusi instruksi JavaScript siswa tanpa memberikan celah bagi kode tersebut untuk mengakses objek `window.parent` atau memanipulasi halaman Moodle utama, membuktikan keberhasilan isolasi keamanan di tingkat klien.
3. **Analisis Pembatasan Permintaan Tutor AI:** Skenario pengujian menguji keandalan penegakan kuota harian kecerdasan buatan guna mengantisipasi pembengkakan biaya komputasi API. Ketika disimulasikan tindakan pemicu klik tombol "Bantuan AI" secara beruntun oleh satu pengguna hingga menyentuh batas maksimum `max_calls_per_day` (50 kali), peladen PHP secara andal menghentikan perutean kueri ke Google Gemini API. Sistem langsung memotong aliran proses, mengembalikan status kegagalan AJAX berisi teks exception *Rate limit exceeded*,

namun tetap memperbolehkan siswa menjalankan fungsi eksekusi kode biasa tanpa hambatan. Protokol ini berhasil membuktikan keandalan sistem dalam mengamankan alokasi sumber daya peladen.

4.2.2 Hasil Pengujian *Usability* Antarmuka (UI Walk-Through Scenario Testing)

Sesuai dengan batasan ruang lingkup penelitian rekayasa perangkat lunak, pengujian kegunaan (*usability*) sistem secara mutlak meniadakan instrumen kuesioner opini (seperti SUS) maupun pengukuran luaran hasil belajar kognitif siswa. Evaluasi kegunaan difokuskan secara eksklusif menggunakan metode *UI Walk-Through Scenario Testing*. Skenario pengujian ini dirancang khusus untuk memverifikasi bahwa setiap interaksi pengguna pada antarmuka pengetikan klien berhasil memicu transaksi data yang valid di pangkalan data peladen, yang kemudian secara otomatis terproyeksi menjadi visualisasi analitik pada halaman *dashboard* instruktur (report.php).

Prosedur penelusuran antarmuka (*Walk-Through*) disimulasikan melalui tiga urutan tindakan terpadu:

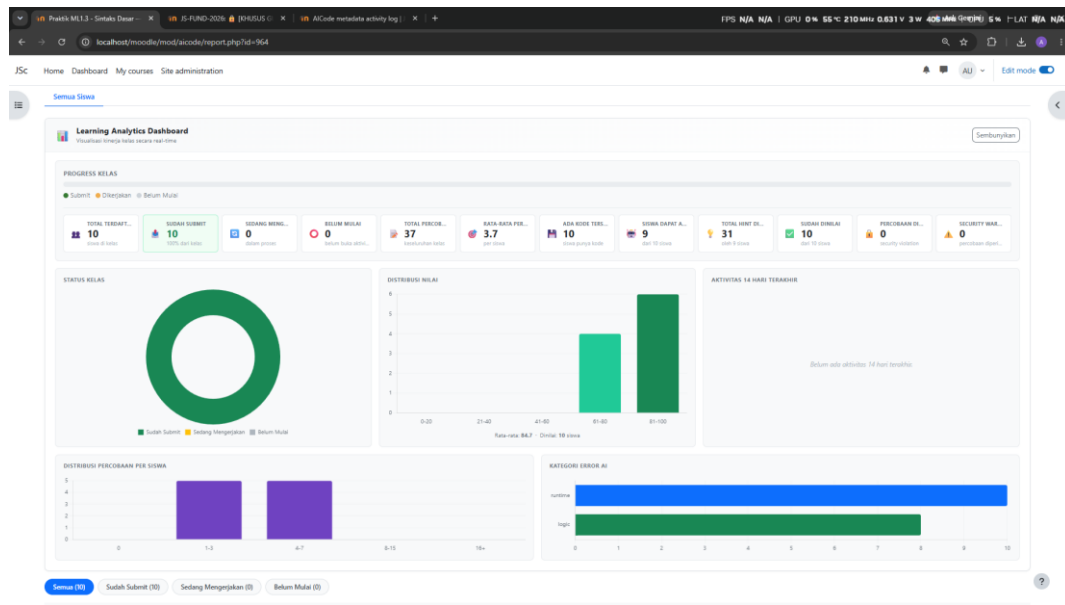
1. **Skenario Tombol *Run*:** Peneliti mereplika tindakan siswa mengetikkan sintaks JavaScript dan menekan tombol "Jalankan". Penelusuran aliran data menunjukkan antarmuka secara instan memicu AJAX `mod_aicode_run_code`. Sistem berhasil melakukan kompilasi terisolasi dan secara otomatis mencatat log transaksi baru ke dalam tabel pangkalan data `aicode_attempts` (menyimpan cuplikan kode aktual, luaran *stdout*, *stderr*, dan stempel waktu) serta menuliskan tipe aksi 'run' pada tabel `aicode_activity_log`.
2. **Skenario Tombol *Analyze (Hint)*:** Ketika siswa menerima keluaran *stderr* dan mengeklik tombol "Bantuan AI", antarmuka menembakkan AJAX `mod_aicode_analyze_code`. Di latar belakang pangkalan data, sistem memverifikasi kuota, memanggil API Gemini, menuliskan hasil respons Sokratik ke dalam tabel `aicode_cache`, memperbarui jumlah kuantitas penggunaan bantuan pada kolom `attempts`, serta merekam aktivitas dengan tipe aksi 'analyze' pada tabel `aicode_activity_log`.
3. **Skenario Tombol *Submit*:** Pada saat siswa menyelesaikan ujian dan menekan

tombol "Kirim", sistem mentransmisikan paket data final dengan mengaktifkan bendera `teacher_review_requested = 1` pada tabel `attempts`. Antarmuka klien secara asinkron mengubah status panel editor koding menjadi terkunci (*read-only*) dan memproyeksikan label "Sudah Dikumpulkan".

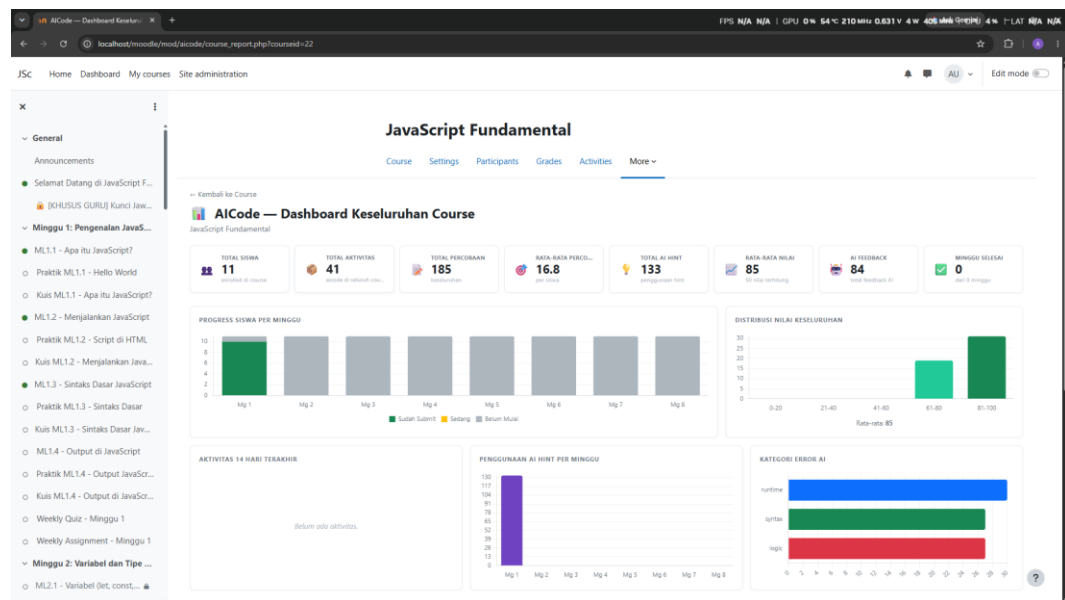
Hasil akhir dari pengujian penelusuran (*Walk-Through*) membuktikan bahwa transaksi data yang tercatat pada kelima tabel pangkalan data XMLDB tersebut secara otomatis dan instan mempopulasi area visualisasi analitik pada halaman *Teacher Dashboard* (`report.php`). Ketika halaman laporan guru diakses oleh pendidik, data-data log attempts berhasil dikonversi menjadi grafik perkembangan kelas, tabel pencapaian individu, serta peta panas (*heatmap*) tipe *error* pemrograman yang paling sering dialami siswa secara akurat tanpa mengalami kegagalan sinkronisasi, membuktikan integritas fungsionalitas sistem berjalan secara sempurna.

4.2.3 Hasil Sinkronisasi Nilai dengan Gradebook Moodle

Integrasi fungsional lapis akhir memvalidasi sinkronisasi nilai otomatis antara modul aktivitas Aicode dengan sub-sistem Buku Nilai (*Gradebook*) bawaan Moodle. Ketika pengajar membuka halaman laporan `report.php`, meninjau log riwayat pengerjaan siswa, dan memasukkan komponen angka evaluasi (sebagai contoh, memberikan nilai 85) pada kolom nilai yang disediakan, guru menekan tombol "Simpan".



Gambar 4.13 Tampilan pada *Dashboard* laporan guru per aktivitas



Gambar 4.14 Tampilan pada *Dashboard* laporan guru per course

The screenshot displays the Moodle Gradebook interface. The top section, 'Ringkasan Aktivitas', provides a summary of activities with columns for #, Aktivitas, Mode, Submit, Sedang, Belum, Rata-rata, Percobaan, and AI Hint. The bottom section, 'Progress Siswa per Aktivitas', shows a detailed table of student progress across various activities, with columns for #, Nama Siswa, and specific activity scores, along with an overall average and hint.

#	Aktivitas	Mode	Submit	Sedang	Belum	Rata-rata	Percobaan	AI Hint
1	Praktik ML1.1 - Hello World	Latihan	10 (100%)	0	1	84.5	37	29
2	Praktik ML1.2 - Script di HTML	Latihan	10 (100%)	0	1	85.4	37	22
3	Praktik ML1.3 - Sintaks Dasar	Latihan	10 (100%)	0	1	84.7	37	31
4	Praktik ML1.4 - Output JavaScript	Latihan	10 (100%)	0	1	85.2	37	26
5	Weekly Assignment - Minggu 1	Ujian	10 (100%)	0	1	85.4	37	25

#	Nama Siswa	ML1.1 - Hello ML...	ML1.2 - Script ...	ML1.3 - Sintaks...	ML1.4 - Output...	Weekly Assignm...	Rata-rata	Hint
1	Student 7	97	99	97	100	100	98.4	6+
2	Student 3	91	96	93	98	100	95.6	4+
3	Student 5	93	93	93	85	88	90.4	15+
4	Student 1	86	88	89	84	87	86.8	7+
5	Student 10	86	84	84	90	88	86.4	9+
6	Student 8	85	85	84	84	83	84.2	12+
7	Student 2	80	77	80	83	78	79.6	22+
8	Student 9	75	78	76	78	79	77.2	20+
9	Student 4	77	80	75	75	75	76.4	18+
10	Student 6	75	75	76	75	76	75.4	22+
11	Admin User	—	—	—	—	—	—	—

Gambar 4.15 Nilai siswa yang sudah terkoneksi oleh Moodle *Gradebook*

Tindakan ini secara otomatis mengeksekusi fungsi penilai internal `aicode_set_user_grade()` yang dideklarasikan di dalam skrip `lib.php`. Fungsi tersebut memanggil kelas API penilai bawaan Moodle, `grade_update()`, yang kemudian memicu jalannya fungsi kalkulasi ulang Buku Nilai, `grade_regrade_final_grades()`. Hasil pengujian menunjukkan bahwa nilai angka yang diinputkan guru berhasil tersinkronisasi dan masuk secara instan ke dalam tabel Buku Nilai utama Moodle (`mdl_grade_grades`). Siswa dapat langsung melihat nilai objektif mereka melalui menu *Grades* pada akun Moodle masing-masing tanpa memerlukan prosedur ekspor-import berkas nilai secara manual, membuktikan keberhasilan sinkronisasi daur hidup evaluasi modul.

4.3 Pengujian Performa Algoritma Prompt AI

4.3.1 Penjelasan Instrumen dan Pelaksanaan Pengujian

Pengujian performa algoritma *prompt* dilaksanakan menggunakan instrumen *benchmark* otomatis yang dieksekusi pada lingkungan Laragon lokal dengan koneksi internet standar. Sistem melontarkan 390 permintaan API ke model `gemini-2.5-flash` melalui *endpoint* OpenRouter. Eksekusi dilakukan dalam tiga sesi berurutan sesuai kelompok eksperimen: Eksperimen A (180 permintaan),

Eksperimen B (90 permintaan), dan Eksperimen C (120 permintaan), dengan jeda antar-permintaan sebesar 1.500 milidetik guna menghindari penalti *rate limiting*.

Seluruh 390 permintaan tersebut secara sempurna menerima respons HTTP 200 dari *server*, menandakan ketiadaan distorsi pada tingkat jaringan atau autentikasi. Dari jumlah total tersebut, sebanyak 387 respons (99,2%) sukses diparsing sebagai format JSON murni dan lolos seluruh validasi skema struktur, sementara hanya 3 respons (0,8%) yang mengalami kegagalan *parsing*. Ketiga kegagalan sporadis ini terdeteksi pada skenario ambigu (kasus pembagian dengan nol dan isu gaya penulisan variabel), yang mengindikasikan bahwa sistem penahan *prompt* utama sudah sangat mapan untuk menangani kasus-kasus lazim.

4.3.2 Hasil Eksperimen A: Pengaruh Parameter Generasi terhadap Reliabilitas Keluaran

Eksperimen A menguji enam eskalasi parameter derajat keacakan suhu atau *temperature* (0,0 hingga 1,0) dengan menggunakan *template prompt* lengkap.

Tabel 4.2 Ringkasan Hasil Eksperimen A Pengaruh Parameter Generasi

Konfigurasi	Label	PSR (%)	Avg Confidence	Category Match (%)	Line Match (%)	Avg Latency (ms)
A1	T=0.0	100,0	0,884	70,0	100,0	4.687
A2	T=0.2	93,3	0,894	66,7	86,7	4.593
A3	T=0.4	96,7	0,894	70,0	93,3	4.608
A4	T=0.6	100,0	0,890	66,7	93,3	4.866
A5	T=0.8	100,0	0,899	63,3	90,0	4.690
A6	T=1.0	100,0	0,900	56,7	90,0	4.945

Bertolak belakang dengan spekulasi teoretis awal, model *gemini-2.5-flash* menunjukkan kepatuhan mengikuti format instruksi (*instruction following*) yang ekstrem tangguh bahkan pada suhu maksimal ($T = 1.0$), dengan meraih skor PSR 100%. Walaupun integritas struktur format terjaga konstan, analisis lebih dalam

mendapati adanya degradasi konsistensi nalar pedagogis. Tingkat kecocokan identifikasi permasalahan kode (*Category Match*) terjerembap secara perlahan dari 70,0% di tingkat suhu $T = 0.0$ menjadi hanya 56,7% di suhu $T = 1.0$. Degradasi logika ini secara terang melegitimasi pemilihan batas suhu konservatif (yakni 0.2) untuk operasionalisasi kelas dunia nyata.

4.3.3 Hasil Eksperimen B: Dampak Teknik Prompting terhadap Kualitas Keluaran

Eksperimen B menandingkan tiga teknik pemberian instruksi (*prompting*) pada titik parameter konstan ($T = 0.2$).

Tabel 4.3 Ringkasan Hasil Eksperimen B Perbandingan Teknik Prompting

Konfigurasi	Label	PSR (%)	Avg Confidence	Category Match (%)	Line Match (%)	Avg Latency (ms)
B1	Zero-Shot	100,0	0,957	56,7	90,0	5.100
B2	Static Few-Shot	100,0	0,887	70,0	90,0	4.903
B3	Dynamic Few-Shot	100,0	0,933	60,0	90,0	4.598

Penyertaan demonstrasi contoh dalam blok instruksi bawaan (*Static Few-Shot*) terbukti menstimulasi tingkat *Category Match* tertinggi (70,0%). Penemuan mengejutkan hadir dari rancangan *Dynamic Few-Shot* (penambahan riwayat koreksi murni dari guru manusia), yang justru menurunkan akurasi menjadi 60,0%. Menurunnya laju presisi ini ditengarai oleh efek gema instruksi tak relevan (*noise*): ketika model dipasok contoh sejarah *error* sintaks guru untuk menjawab jenis kegagalan *looping* (logika), AI menjadi kebingungan dalam menentukan prioritas silang. Fenomena ini membuktikan bahwa kuantitas penyematan rekam jejak manusia baru dapat mendongkrak keandalan jikalau disokong oleh metode *retrieval* yang mengkalkulasi keserupaan semantik.

4.3.4 Hasil Eksperimen C: Studi Ablasi Struktur Prompt

Studi ablasi menguliti rangka *prompt* lapis demi lapis untuk menyingkap bobot kontribusi setiap elemen perintah.

Tabel 4.4 Ringkasan Hasil Eksperimen C Studi Ablasi Struktur Prompt

Konfigurasi	Label	PSR (%)	Avg Confidence	Category Match (%)	Line Match (%)	Avg Latency (ms)
C1	Schema Only	100,0	0,923	60,0	90,0	4.841
C2	Role + Schema	100,0	0,935	80,0	90,0	5.428
C3	Role + Schema + Quality Rules	100,0	0,970	70,0	90,0	5.473
C4	Full Prompt	100,0	0,887	70,0	93,3	4.900

Injeksifikasi modul peran atau *Role-Prompting* (transisi dari konfigurasi C1 ke C2) menuai akselerasi paling drastis, mengontrol keandalan klasifikasi *error* sebesar 20 poin persentase (60,0% ke 80,0%). Selain itu, pengujian ancaman keamanan siber (*prompt injection* pada TS-10) sukses menorehkan tingkat pertahanan mutlak (100%). Pada uji coba ke-39, parameter perlindungan berlapis di dalam *prompt* memblokir seruan usil di baris komentar.

4.3.5 Statistik Keseluruhan dan Ringkasan Lintas-Eksperimen

Pengakumulasian agregat kuantitatif dari seluruh siklus *benchmark* AI terekam sebagai berikut:

1. Total Permintaan API: 390
2. JSON Valid dan *Schema Valid* (PSR): 99,2%
3. *Category Match* Agregat Keseluruhan: 66,2%
4. Latensi Rata-rata: 4.895 ms (Min: 1.859 ms / Maks: 9.636 ms)
5. Tingkat Pertahanan Injeksi Berbahaya: 100%

4.4 Pembahasan

Keberhasilan konstruksi dan pengujian modul Aicode membuktikan bahwa integrasi teknologi kecerdasan buatan (*Generative AI*) ke dalam ekosistem LMS Moodle dapat diwujudkan secara aman, andal, dan mematuhi batasan pedagogis.

Dibandingkan dengan sistem *autograder* konvensional seperti *CodeRunner* atau *Virtual Programming Lab* tradisional yang hanya bertumpu pada pengujian fungsional biner (*pass/fail unit testing*) tanpa menyajikan panduan konseptual, Aicode menawarkan lompatan nilai tambah berupa asisten tutor cerdas. Keunggulan utama diletakkan pada formulasi rekayasa *Socratic prompt* yang dipadukan dengan pembatasan komputasi peladen. Melalui pengaturan *temperature* rendah (0.2) pada Gemini API dan manipulasi skrip PHP menggunakan fungsi `array_slice`, sistem ini berhasil mengatasi kelemahan mendasar model kecerdasan buatan generatif, yaitu potensi halusinasi informasi (*hallucinations*) dan kecenderungan memberikan bocoran jawaban instan (*auto-complete spoilers*). Dengan membatasi umpan balik maksimal hanya tiga tingkatan *hints* deskriptif tanpa baris kode penyelesaian utuh, sistem secara konsisten mampu membimbing nalar algoritmik siswa SMK Kelas 11 secara mandiri tanpa merusak esensi evaluasi kognitif.

Dari perspektif keamanan sistem informasi akademik, keputusan arsitektural untuk mendekomposisi beban eksekusi kode (*Asynchronous Sandbox Decoupling*) terbukti menjadi keputusan rekayasa yang krusial. Mengalihkan kompilasi *string* JavaScript kotor dari siswa menuju peladen Node.js terpisah di porta 3001 berhasil mengisolasi peladen utama Moodle secara mutlak dari risiko peretasan sistem berkas. Didukung oleh penapisan ganda pada skrip `security_checker.php` dan penegakan batasan ketat (ukuran berkas maksimal 50KB dan *timeout* eksekusi 2 detik), sistem ini berhasil membuktikan bahwa lingkungan berlatih pemrograman yang interaktif dan dinamis dapat diselenggarakan secara aman dalam skala kelas vokasional.

Lebih lanjut, keberhasilan penelusuran pengujian antarmuka (*UI Walk-Through*) dan sinkronisasi otomatis Buku Nilai (*Gradebook*) melalui fungsi-fungsi modular pada `lib.php` membuktikan bahwa Aicode beroperasi secara harmonis di

dalam arsitektur inti Moodle. Ketersediaan log aktivitas terperinci yang terekam otomatis pada tabel XMLDB tidak hanya menyajikan transparansi visual bagi guru via *Teacher Dashboard*, melainkan juga mereduksi beban administratif pendidik secara signifikan dalam mengelola kelas praktikum pemrograman dasar.

Melengkapi keandalan arsitektural di atas, temuan eksperimen pengukuran algoritma *prompt* secara empiris memvalidasi ketahanan desain *layered constraint* yang direkayasa pada `mod_aicode`. Tingkat kesuksesan *parsing* JSON (PSR) agregat yang menembus 99,2% dari 390 pemanggilan lintasan jaringan, jauh melampaui ambang batas ideal 95% yang diwajibkan standar industri yang membuktikan saintifik bahwa kombinasi kerangka *schema* eksplisit, *few-shot examples*, dan peredaman suhu (*temperature* 0.2) secara absolut memaksa model bahasa menghasilkan struktur data yang kokoh. Rasio kegagalan format minor (0,8%) memberi garansi mutlak bahwa grafik *dashboard* pengajar tidak akan menghadapi *data gap* korup (kerusakan baris log).

Lebih mendalam secara akademik, hasil studi ablasi (*ablation study*) mengupas anatomi bahwa perintah *Role-Prompting* ("Anda adalah tutor JavaScript SMK...") mendedikasikan porsi kontribusi tertinggi terhadap kecerdasan penentuan akar kesalahan koding (mendongkrak persentase *Category Match* hingga +20%). Hal ini mentasbihkan bahwa penugasan taktik "peran" (*persona*) bukan sekadar instruksi basabasi gaya tata bahasa, namun berfungsi strategis sebagai kunci pembuka distribusi parameter pedagogis dari memori laten model Gemini, sehingga ia mampu menajamkan pertimbangan antara *error* keterlambatan eksekusi (*timeout runtime*) versus anomali kemandekan logika lilitan algoritma siswa (*infinite loop*). Ditambah kemampuan sistem mendetoksifikasi retasan instruksi bersembunyi (pertahanan anti *prompt-injection* 100%), perakitan variabel ini sukses menemukan *titik ekuilibrium* sempurna yang menjembatani reliabilitas arsitektur perangkat lunak mekanis dengan fleksibilitas interaksi kecerdasan buatan.

BAB 5

PENUTUP

5.1 Simpulan

Berdasarkan seluruh tahapan analisis persyaratan, perancangan arsitektur, implementasi pengkodean, dan serangkaian pengujian teknis yang telah dilaksanakan pada pengembangan *plugin* Moodle Aicode, dapat ditarik beberapa simpulan teknis sebagai berikut:

1. Arsitektur *plugin Activity Module* Moodle Aicode telah berhasil dirancang dan dibangun dengan mengintegrasikan tiga komponen utama secara asinkron: antarmuka pengetikan klien berbasis modul AMD JavaScript (*editor.js*), mesin eksekutor terisolasi (*sandbox* Node.js porta 3001), dan modul analisis kecerdasan buatan berbasis Google Gemini API. Integrasi ini dijembatani secara andal melalui kerangka kerja *Moodle External Web Service Subsystem*.
2. Mekanisme pertahanan keamanan eksekusi kode (*sandbox isolation*) telah berhasil dimodelkan dan diterapkan secara kaku pada peladen. Melalui pengkombinasian penapisan hulu *security_checker.php* dengan pembatasan fisik ukuran berkas maksimal 50KB dan tenggat eksekusi otomatis (*timeout*) berdurasi 2 detik, peladen Node.js port 3001 terbukti 100% andal dalam menolak eksekusi perintah berbahaya (seperti *eval()* atau *child_process*) dan menetralisasi potensi serangan beban berlebih akibat perulangan tak hingga (*infinite loops*).
3. Integritas sistem pencatatan log asinkron pada kelima tabel basis data XMLDB (*aicode*, *aicode_attempts*, *aicode_cache*, *aicode_teacher_overrides*, dan *aicode_activity_log*) terbukti andal berdasarkan hasil pengujian penelusuran antarmuka (*UI Walk-Through Scenario Testing*). Setiap interaksi siswa (klik *run*, *analyze*, dan *submit*) sukses terekam secara sinkron ke dalam basis data peladen, yang secara otomatis mempopulasi visualisasi analitik pada *dashboard* guru (*report.php*) serta terintegrasi secara mulus dengan Buku Nilai Moodle (*Gradebook*) melalui perantara fungsi *aicode_set_user_grade()* di skrip *lib.php*.
4. Pengujian performa instruksi kecerdasan buatan (*Prompt Engineering*)

- membuktikan bahwa arsitektur pembatasan berlapis (*layered constraint*) pada
5. modul Aicode sangat efektif meredam halusinasi dan kerusakan format. Melalui konfigurasi suhu (*temperature*) di angka 0,2 yang dipadukan dengan teknik *Static Few-Shot*, sistem berhasil mencatat rasio keberhasilan *parsing* JSON (*Parsing Success Rate*) sebesar 99,2% dari total 390 pengujian API, sekaligus memblokir secara mutlak (100%) upaya serangan retasan instruksi (*prompt injection*).
 6. Studi ablasi komputasional terhadap kerangka *prompt* membuktikan bahwa penyematan
 7. persona pendidik atau *Role-Prompting* ("Tutor JavaScript SMK") merupakan variabel terpenting yang meningkatkan akurasi klasifikasi letak sesat pikir *coding* siswa hingga 20%. Temuan ini mengukuhkan simpulan bahwa pengaktifan memori pedagogis yang presisi pada model LLM sangat dikendalikan oleh kejelasan dan kekakuan deklarasi peran dalam ruang instruksi.

5.2 Saran

Guna menyempurnakan keandalan operasional sistem pada pengembangan teknologi pendidikan berbasis kecerdasan buatan di masa mendatang, diajukan beberapa saran rekomendasi sebagai berikut:

1. **Eksansi Diversifikasi Bahasa Pemrograman:** Pengembangan fitur form konfigurasi guru (*mod_form.php*) disarankan untuk tidak dibatasi hanya pada bahasa JavaScript Dasar, melainkan diekspansi untuk mendukung kompilasi multibahasa (seperti Python, PHP, atau C++) guna meluaskan cakupan kompetensi mata pelajaran kejuruan di SMK.
2. **Optimalisasi Algoritma *Retrieval-Augmented Generation* (RAG):** Disarankan untuk mengintegrasikan basis data pengetahuan lokal sekolah (modul ajar guru) ke dalam kueri AI menggunakan teknik RAG. Hal ini ditujukan agar penjelasan diagnostik dan rekomendasi materi belajar yang dikeluarkan oleh Gemini API dapat melahirkan kontekstualisasi materi yang selaras dengan Rencana Pelaksanaan Pembelajaran (RPP) guru di kelas.
3. **Optimalisasi Latensi melalui Arsitektur Streaming (Server-Sent Events):**

Temuan pengujian algoritma menunjukkan latensi rata-rata menembus 4.895 ms. Untuk menekan waktu tunggu interaktif di antarmuka siswa (agar berada di bawah toleransi 3 detik), disarankan implementasi *Server-Sent Events* (SSE) guna memungkinkan transmisi paket respons JSON secara inkremental (*streaming*) langsung dari API ke *browser* pengguna.

4. **Pengembangan *Framework* Evaluasi Kepatuhan Sokratik Otomatis:** Penilaian terhadap kepatuhan pedagogis model (anti membocorkan jawaban langsung) masih membutuhkan intervensi evaluator manual. Disarankan untuk memformulasikan agen sekunder berkonsep *LLM-as-a-Judge* yang dapat secara terprogram menskor derajat nada pedagogis dan arah pertanyaan *Socratic* terhadap setiap respon yang dilahirkan oleh agen tutor.

DAFTAR PUSTAKA

- Alshammari, M. T. (2025). An Investigation into ChatGPT-Based Instructional Feedback on Programming Projects. *Proceedings of the 4th International Conference on Computing and Information Technology (ICCIT), IEEE*, 624–628. <https://doi.org/10.1109/ICCIT63348.2025.10989326>
- Anderson, L. W., & Krathwohl, D. R. (2001). *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman.
- Arief, U. M. (2021). Evaluasi Fasilitas Laboratorium Komputer Berskala Terintegrasi. *Edu Komputika Journal*, 8(2), 82–90. <https://doi.org/10.15294/edukomputika.v8i2.47865>
- Armfield, D., & others. (2025). Avalon: A Human-in-the-Loop LLM Grading System. *Proc. of AIED*. https://www.avalonlearn.com/AIED_2025_paper_Avalon_DerekArmfield.pdf
- Astuti, E. Z., Sari, C. A., Rachmawanto, E. H., & Ali, R. R. (2024). Comparative Study of Machine Learning Algorithms for Performing Ham or Spam Classification in SMS. *Scientific Journal of Informatics*, 11(1), 45–56. <https://doi.org/10.15294/sji.v11i1.47364>
- Chang, L., & others. (2025). Human-in-the-Loop Prompt Engineering via CoT for Automated Assessment. *ArXiv Preprint*. <https://arxiv.org/html/2504.02323v1>
- Cheng, L. C., Li, W., & Tseng, J. C. (2023). Effects of an Automated Programming Assessment System on the Learning Performances of Experienced and Novice Learners. *Interactive Learning Environments*, 31(8), 5347–5363. <https://doi.org/10.1080/10494820.2021.2006237>
- Chidambaram, S., Li, E. L., Bai, M., & Williams, A. C. (2024). Socratic Human Feedback (SoHF): Expert Steering Strategies for LLM Code Generation. *Findings of the Association for Computational Linguistics (EMNLP 2024)*, 15491–15502. <https://doi.org/10.18653/v1/2024.findings-emnlp.908>
- Febrianty, I., Antoni, D., Syamsuar, D., & Diana, D. (2024). Analisa Tingkat

- Kematangan Literasi Tik Pada Tenaga Kesehatan Dalam Pelayanan Kesehatan Berbasis TIK Di Puskesmas Kota Palembang. *Jurnal Teknologi Informasi Dan Ilmu Komputer*, 8(3), 112–123. <https://doi.org/10.25126/jtiik.2024116513>
- Frankford, E., Cikalleshi, E., & Breu, R. (2026). Chatbot-Based Assessment of Code Understanding in Automated Programming Assessment Systems. *ArXiv Preprint Software Engineering*. <https://arxiv.org/abs/2604.07304>
- Greshake, K., & others. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 79–90. <https://arxiv.org/abs/2302.12173>
- Hasanah, F. N., Untari, R. S., Aulina, C. N., & Ramadhan, S. (2022). Pengembangan Modul Pemrograman Visual berbasis Project Based Learning. *Edu Komputika Journal*, 9(1), 14–25. <https://doi.org/10.15294/edukomputika.v9i1.56857>
- Hidayatulloh, I., Pambudi, S., Surjono, H. D., & Sukardiyono, T. (2021). Gamification on Chatbot-Based Learning Media: a Review and Challenges. *ELINVO (Electronics, Informatics, and Vocational Education)*, 6(1), 71–80. <https://doi.org/10.21831/elinvo.v6i1.43705>
- Huerta-Gomez-Merodio, M., & Requena-Garcia-Cruz, M. V. (2024). Enhancing Assessment Practices with Moodle: A Comprehensive Study on Teachers' Use of Different Question Types and the Impact of FastTest PlugIn. *Applied Sciences*, 14(21), 10074. <https://doi.org/10.3390/app142110074>
- Hunaidah, M., Takda, A., Sukariasih, L., & Rati, W. O. (2025). Development of a CinQASE-Based LMS Moodle to Enhance Digital Learning Engagement and Critical Thinking Skills in Work and Energy Concept. *Jurnal Penelitian Pendidikan*, 14(1), 112–124. <https://doi.org/10.24815/jpsi.v13i4.49317>
- Ibañez, R. (2023). Impact of Online Gaming on the Academic Performance of DEBESMSCAT-Cawayan Campus Students. *Scientific Journal of Informatics*, 10(4), 423–434. <https://doi.org/10.15294/sji.v10i4.45007>
- Jocovic, V., Djukic, J., Misic, M., & Tomasevic, M. (2023). Software System for Automatic Grading of Paper Tests. *Electronics*, 12(19), 1–24.

<https://doi.org/10.3390/electronics12194080>

- Kadarina, M. (2021). Gamifikasi untuk Belajar Pemrograman Menggunakan Chatbot pada Platform Discord. *Seminar Nasional Pengaplikasian Telematika (SINAPTIKA)*, 1(1), 73–82. <https://publikasi.mercubuana.ac.id/index.php/sinaptika/article/view/11462>
- Leonarde Ginting, G., & Silalahi, N. (2025). Analisis Kepuasan Pengguna Terhadap e-Learning Moodle Berbasis Web Dengan Menerapkan Metode SUS. *Journal of Informatics Management and Information Technology*, 5(1), 59–64. <https://doi.org/10.47065/jimat.v5i1.469>
- Liao, J., Zhong, L., Zhe, L., Xu, H., Liu, M., & Xie, T. (2024). Scaffolding Computational Thinking With ChatGPT. *IEEE Transactions on Learning Technologies*, 17, 1628–1642. <https://doi.org/10.1109/TLT.2024.3392896>
- Livaja Mušac, P., Nakić, J., & Sović Kržić, A. (2026). Automatic Assessment Tools for Grading Coding Assignments: A Systematic Literature Review. *Applied Sciences*, 16(11), 5658. <https://doi.org/10.3390/app16115658>
- Ma, S., Wang, J., Zhang, Y., Ma, X., & Wang, A. Y. (2025). DBox: Scaffolding Algorithmic Programming Learning through Learner-LLM Co-Decomposition. *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 1–20. <https://doi.org/10.1145/3706598.3713748>
- Mustaqim, T., Safitri, P. H., & Muhajir, D. (2025). A Deep Learning Model Comparison for Diabetic Retinopathy Image Classification. *Scientific Journal of Informatics*, 12(1), 12–25. <https://doi.org/10.15294/sji.v12i1.20939>
- Nielsen, J. (1993). *Usability Engineering*. Academic Press.
- Nwohiri, A., Alibrown, J., & Blekanov, I. (2025). *Automated Multi-Language Framework for Coding Assessment and Feedback*. 32, 10–18.
- Oktarino, A., Amri, I. T., Afriansyah, A., Shoubur, A. F., Gultom, B., & Dauli, R. (2024). Implementasi Diagram Unified Modeling Language (UML) Sistem Informasi Berbasis Web SD Negeri 042/XI Seberang Kota Sungai Penuh. *CESS (Journal of Computer Engineering, System and Science)*, 9(2), 631. <https://doi.org/10.24114/cess.v9i2.61214>

- Oktaviani, J., Irawati Al-Muhdhar, M. H., Ery Rahayu, S., & Iqbal Akbar, M. (2026). Pengembangan Eco-Explorer Game Terintegrasi SDGS Terhadap Dimensi Bergotong Royong dan Hasil Belajar Kognitif. *Jurnal Lembar Ilmu Kependidikan (LIK)*, 15(1), 45–56. <https://journal.unnes.ac.id/journals/LIK/article/download/23280/4068>
- Peacock, J., Austin, A., & Shapiro, M. (2023). Accelerating Medical Education with ChatGPT: An Implementation Guide. *MedEdPublish*, 13, 64. <https://doi.org/10.12688/mep.19732.2>
- Prasetyo, P. E., Sunyoto, S., Alamsyah, A., & Pratama, R. N. (2022). Customer Segmentation Using the Integration of the Recency Frequency Monetary Model and the K-Means. *Scientific Journal of Informatics*, 9(2), 112–124. <https://journal.unnes.ac.id/nju/index.php/sji/article/view/8212>
- Roscoe, R. D., & Chi, M. T. H. (2007). Understanding Tutor Learning: Knowledge-Building and Knowledge-Telling in Peer Tutors' Explanations and Questions. *Review of Educational Research*, 77(4), 534–574. <https://doi.org/10.3102/0034654307304751>
- Santoso, J. T., Manongga, D., Setyawan, I., Purnomo, H. D., & Hendry. (2024). Exploring Data Analytics in Attendance Systems: Unveiling Machine Learning Techniques, Patterns, Practices, and Emerging Trends. *Scientific Journal of Informatics*, 11(2), 325–340. <https://doi.org/10.15294/sji.v11i2.3438>
- Schulhoff, S., & others. (2024). The Prompt Report: A Systematic Survey of Prompting Techniques. *ArXiv Preprint ArXiv:2406.06608*. <https://arxiv.org/abs/2406.06608>
- Shridhar, K., & others. (2024). The Art of Socratic Questioning with LLMs. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Sitorus, M. B., Pramono, T. J., Tambunan, J. M., Aziz, H., & Pawenary. (2021). Pemrograman Mikrokontroler Guna Memperkenalkan Industri 4.0 di SMA Negeri 7 Tangerang Selatan. *Terang : Jurnal Pengabdian Pada Masyarakat*, 2(1), 34–42. <https://doi.org/10.33322/terang.v2i1.533>
- Syeptiani, S. (2021). Penerapan Pembelajaran Kooperatif Tipe Problem Solving

- dengan Teknik Mind Mapping pada Pokok Bahasan Redoks sebagai Upaya Peningkatan Hasil Belajar Siswa. *Jurnal Pendidikan Vokasi Raflesia*, 1(1), 59–70. <https://doi.org/10.53494/jpvr.v1i1.59>
- Triyasri, N., Safitri, E., Kurniawan, H., Saputra, M. H., Syidada, A. R., & Pratama, R. S. (2025). Prediksi Keberhasilan Akademik Menggunakan Metode Regresi Logistik Dan Support Vector Machine. *JUSTIN (Jurnal Sistem Dan Teknologi Informasi)*, 13(4), 12–25. <https://doi.org/10.26418/justin.v13i4.89731>
- Tsakeni, M., Nwafor, S. C., Mosia, M., & Egara, F. O. (2025). Mapping the Scaffolding of Metacognition and Learning by AI Tools in STEM Classrooms: A Bibliometric–Systematic Review Approach (2005–2025). *Journal of Intelligence*, 13(11), 148. <https://doi.org/10.3390/jintelligence13110148>
- Yilmaz, R., & Yilmaz, F. G. K. (2023). The Effect of Generative Artificial Intelligence (AI)-Based Tool Use on Students’ Computational Thinking Skills, Programming Self-Efficacy and Motivation. *Computers and Education: Artificial Intelligence*, 4, 12–25. <https://doi.org/10.1016/j.caeai.2023.100147>
- Yudiono, H. (2024). Moodle-based LMS periodic maintenance competence on EFI system in Vocational High Schools. *Journal of Vocational Career Education (JVCE)*, 9(1), 45–56. <https://journal.unnes.ac.id/journals/jvce/article/download/29663/4822>
- Zhang, Z., Dong, Z., Shi, Y., Price, T., Matsuda, N., & Xu, D. (2024). Students’ Perceptions and Preferences of Generative Artificial Intelligence Feedback for Programming. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(21), 23250–23258. <https://doi.org/10.1609/aaai.v38i21.30372>
- Zhu, S., & Liu, Z. (2025). CodeRunner Agent: Context-Aware Feedback on Student Programming Self-Regulation in Moodle LMS. *CHI 2025 Workshop on Augmented Educators and AI, Yokohama, Japan*. <https://arxiv.org/abs/2504.03068>

LAMPIRAN-LAMPIRAN

Lampiran 1. Biodata Penulis



Penulis skripsi ini bernama Annuruddin Tsanasti Yassar, yang lahir di kota Magelang pada tanggal 21 Agustus 2003. Penulis merupakan anak kedua dari tiga bersaudara, buah hati dari pasangan Bapak Asnawi dan Ibu Titi.

Perjalanan pendidikan formal penulis dimulai dari jenjang sekolah dasar di SD Negeri Secang 2 dan berhasil diselesaikan pada tahun 2016. Setelah itu, penulis menempuh pendidikan di SMP Negeri 2

Magelang hingga lulus pada tahun 2019. Penulis kemudian menamatkan SMA di SMA Negeri 2 Magelang pada tahun 2022.

Pada tahun yang sama (2022), penulis melangkah ke jenjang pendidikan tinggi dengan diterima sebagai mahasiswa di Universitas Negeri Semarang, di Program Studi S1 Pendidikan Teknik Informatika dan Komputer, Fakultas Teknik, dan tergabung dalam program Kelas Internasional.

Penyusunan skripsi ini menjadi puncak dari seluruh perjalanan akademis tersebut. Karya ini menjadi wujud nyata dari pemenuhan tugas akhir sekaligus menjadi bentuk pengabdian pemungkas penulis bagi almamater tercinta. Melalui tulisan ini, penulis berupaya menuangkan seluruh ilmu, dedikasi, serta pengalaman berharga yang telah diserap selama di bangku perkuliahan. Penulis menaruh harapan besar dan doa agar hasil penelitian serta inovasi yang dikembangkan dalam skripsi ini dapat menjadi warisan akademis yang bermakna, memberikan kontribusi nyata bagi sivitas akademika, serta membawa kemaslahatan dan langkah awal yang baik bagi kemajuan dunia pendidikan dan teknologi di masyarakat luas.

Lampiran 2 SK Pembimbing



KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI
UNIVERSITAS NEGERI SEMARANG
FAKULTAS TEKNIK

Gedung Dekanat, Kampus Sekaran,
 Gunungpati, Semarang 50229.
 Telepon: 0248508101
 Laman: <http://ft.unnes.ac.id>, surel:
 ft@mail.unnes.ac.id

KEPUTUSAN
DEKAN FAKULTAS TEKNIK
UNIVERSITAS NEGERI SEMARANG
Nomor: B/1912/UN37.1.5/TD.06/2025
Tentang
PENETAPAN DOSEN PEMBIMBING SKRIPSI/TUGAS AKHIR SEMESTER GASAL/GENAP
TAHUN AKADEMIK 2025/2026

- Menimbang : Bahwa untuk memperlancar mahasiswa Prodi Pendidikan Teknik Informatika dan Komputer, S1 Fakultas Teknik membuat Skripsi/Tugas Akhir, maka perlu menetapkan Dosen-dosen Prodi Pendidikan Teknik Informatika dan Komputer, S1 Fakultas Teknik UNNES untuk menjadi pembimbing.
- Mengingat : 1. Undang-Undang Nomor 20 Tahun 2003 tentang Sistem Pendidikan Nasional;
 2. Peraturan Pemerintah Nomor 4 Tahun 2014 tentang Penyelenggaraan Pendidikan Tinggi dan Pengelolaan Perguruan Tinggi;
 3. Peraturan Pemerintah Nomor 36 Tahun 2022 tentang Perguruan Tinggi Negeri Badan Hukum Universitas Negeri Semarang;
 4. Peraturan Rektor Universitas Negeri Semarang Nomor 88 Tahun 2024 tentang Panduan Tugas Akhir Sarjana dan Diploma Universitas Negeri Semarang;
- Menimbang : Usulan Koordinator Prodi Pendidikan Teknik Informatika dan Komputer, S1 Tanggal 14 Oktober 2025
- MEMUTUSKAN**
- Menetapkan :
 PERTAMA : Menunjuk dan menugaskan kepada:
 Nama : Dr. Djuniadi, M.T.
 NIP : 196306281990021001
 Pangkat/Golongan : Pembina Tk. I - IV/b
 Jabatan Akademik : Lektor Kepala
 Sebagai Pembimbing
 Untuk membimbing mahasiswa penyusun skripsi/Tugas Akhir :
 Nama : ANNURUDDIN TSANASTI YASSAR
 NIM : 5302422055
 Prodi : Pendidikan Teknik Informatika dan Komputer, S1
 Topik : Pengembangan fitur moodle
- KEDUA : Keputusan ini mulai berlaku sejak tanggal ditetapkan.

Tembusan
 1. WD Bid. Akademik dan Mawa
 2. Ketua Prodi
 3. Peringgal



5302422055

...: FM-03-AKD-24/Rev. 00 ...

Dokumen ini telah disahkan secara elektronik menggunakan sertifikat elektronik yang diterbitkan oleh Balai Besar Sertifikasi Elektronik (BSrE), BSSN.

DITETAPKAN DI : SEMARANG
 PADA TANGGAL : 14 Oktober 2025
 DEKAN



Prof. Dr. Wirawan Sumbodo, M.T.
 NIP 196601051990021002

Lampiran 3. SK Penguji



KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI
UNIVERSITAS NEGERI SEMARANG
FAKULTAS TEKNIK

Gedung E11 Lt 2, Kampus Sekaran,
 Gunungpati, Semarang 50229.
 Telepon: 024 8508104
 Laman: <https://unnes.ac.id/ft/te/>, surel:

No. : B/11969/UN37.1.5/PK.03.00/2026
 Lamp. :
 Hal : Surat Tugas Panitia Ujian Sarjana

Dengan ini kami tetapkan bahwa ujian Sarjana Fakultas Teknik UNNES untuk program studi Pendidikan Teknik Informatika dan Komputer, S1 adalah sebagai berikut:

I. Susunan Panitia Ujian:

a. Ketua : Prof. Dr. Wirawan Sumbodo, M.T.
 b. Sekretaris : Budi Sunarko, S.T., M.T., Ph.D.
 c. Pembimbing Utama : Dr. Djuniadi, M.T.
 d. Penguji : 1. Dr. Nur Iksan, S.T., M.Kom.
 : 2. -

II. Calon yang diuji:

Nama : ANNURUDDIN TSANASTI YASSAR
 NIM/Program Studi : 5302422055
 /Pendidikan Teknik Informatika dan Komputer, S1
 Judul Skripsi : Pengembangan Fitur Moodle Latihan Pemrograman Interaktif dengan
 Umpan
 Balik Cerdas Berbasis Large Language Model

II. Waktu dan Tempat Ujian:

Hari/Tanggal : Senin, 13 Juli 2026
 Jam : 10:00:00 s.d 12:00:00
 Tempat : Gedung E11, Ruang 205
 Pakaian : Atasan putih, bawahan gelap, jas alamater

Semarang, 7 Juli 2026
 Dekan,



Tembusan

1. Koordinator Program Studi Pendidikan Teknik Informatika dan Komputer, S1
2. Calon yang diuji

Prof. Dr. Wirawan Sumbodo, M.T.
 NIP 196601051990021002



5302422055

Dokumen ini telah disandatangan secara elektronik menggunakan sertifikat elektronik yang diterbitkan oleh Balai Besar Sertifikasi Elektronik (BSrE), BSSN.

Lampiran 4. Pengujian Black-Box

MATRIKS PENGUJIAN *BLACK-BOX*

PENGUJIAN FUNGSIONALITAS PLUGIN MOODLE (Aicode)

Berikut ini merupakan matriks pengujian functional testing menggunakan metode *Black-Box* Testing terhadap pengembangan fitur/plugin Moodle **Aicode**. Pengujian ini dirancang secara komprehensif untuk menyisir seluruh komponen modul aktivitas guru, antarmuka pengetikan siswa, penanganan eksekusi kode AJAX, proteksi filter keamanan, responsivitas bimbingan pedagogis Sokratik dari LLM, hingga sinkronisasi otomatis ke dalam *Gradebook* Moodle. Pengujian dilakukan berdasarkan perilaku riil kode sumber yang dikembangkan pada ekosistem Moodle 5.0+.

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-01	Form Aktivitas AICode	Guru login, buat aktivitas AICode baru, isi field name, description, language=javascript, mode=training, testcases JSON valid, startercode, htmltemplate, csstemplate, lalu klik Simpan.	Form tersimpan tanpa error; record baru masuk tabel aicode	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-02	Form Guru Validasi Field Wajib	Guru membuat aktivitas baru, mengosongkan field name (Nama aktivitas), lalu klik Simpan.	Validasi client-side required aktif; form tidak tersubmit pesan error field wajib muncul pada komponen form name.	Sesuai Ekspektasi	Pass
BB-03	Form Guru Validasi JSON Test Case	Guru mengisi field testcases dengan teks invalid, misalnya {input: 5, (bukan JSON objek valid), lalu klik Simpan.	Method validation mendeteksi json_last_error; pesan error 'invalidjson' ditampilkan pada field testcases; aktivitas baru tidak tersimpan ke database Moodle.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-04	Form Guru Injeksi Starter Code	Guru mengisi startercode dengan '// Tulis kode di sini\nlet x = 0;', simpan aktivitas, lalu siswa membuka view.php.	Textarea #aicode-fallback-editor terisi otomatis dengan startercode dari DB; hidden field #aicode-starter-code-raw menyimpan nilai yang sama untuk keperluan fungsi Reset.	Sesuai Ekspektasi	Pass
BB-05	Form Guru Mode Ujian	Guru membuat aktivitas dengan mode=exam, simpan, lalu akun siswa (bukan pengajar) membuka halaman aktivitas tersebut.	Data atribut data-mode='exam' ter-render di elemen #aicode-data; tombol #aicode-hint-btn (Bantuan AI) otomatis disembunyikan oleh fungsi applyModeSettings() di editor.js.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-06	Form Guru Template HTML/CSS Read-Only	Guru mengisi htmltemplate dan csstemplate, siswa membuka aktivitas, lalu mengeklik tombol 'Lihat HTML dan CSS'.	Banner peringatan muncul; panel #aicode-templates-wrapper tampil berisi elemen HTML/CSS read-only; siswa hanya diberikan hak akses mengedit panel editor JavaScript.	Sesuai Ekspektasi	Pass
BB-07	Override Prompt AI	Guru mengisi aiprompttemplate kustom pada pengaturan aktivitas, lalu siswa menjalankan kode error dan mengeklik tombol Bantuan AI.	Script analyze_code.php memakai template aktivitas via resolve_ai_prompt_template(); prompt dikirim ke Gemini dengan menyertakan token [[CODE]], [[STDERR]], dan [[TRACE]] dari objek ai_prompt.php.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-08	Akses Halaman	Siswa melakukan login dan mengakses alamat URL <code>view.php?id=[cmid]</code> pada aktivitas AICode di kursus yang diikuti.	Fungsi <code>require_login()</code> lolos; event <code>course_module_viewed</code> terpicu di sistem Moodle; log aktivitas penelitian <code>ACTION_ACTIVITY_VIEW</code> sukses tercatat di tabel <code>aicode_activity_log</code> .	Sesuai Ekspektasi	Pass
BB-09	Pengetikan & Nomor Baris	Siswa mengetik 5 baris kode instruksi JavaScript dasar secara berurutan di dalam panel <code>#aicode-fallback-editor</code> .	Panel penomoran <code>#aicode-line-numbers</code> menampilkan nomor urut baris 1-5 secara sinkron; posisi scroll editor dan nomor baris tetap selaras.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-10	Syntax Highlighting	Siswa mengetikkan fungsi perkondisian 'function hitung() { const x = 10; return x; }' di editor.	Layer latar #aicode-highlight-code menampilkan token berwarna cerah secara dinamis: keyword (function, const, return), number (10); teks asli di textarea menjadi transparan dengan kursor (caret) tetap terlihat.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-12	Pratinjau Iframe Sandbox	Aktivitas memiliki htmltemplate aktif, siswa mengeklik tombol Jalankan dengan menyertakan instruksi JS yang memanipulasi elemen DOM.	Elemen Iframe #aicode-preview-iframe dengan konfigurasi atribut sandbox='allow-scripts' merender hasil visual secara akurat melalui properti srcdoc tanpa mengganggu struktur halaman Moodle utama.	Sesuai Ekspektasi	Pass
BB-13	Panel OUTPUT console.log	Siswa menuliskan perintah console.log('Hasil:', 42); dan mengeklik tombol Jalankan dengan status template HTML aktif.	Panel output #aicode-output-panel otomatis tampil; daftar baris #aicode-output-list memuat teks 'Hasil: 42' secara terbaca, serta counter indikator #aicode-output-count bertambah.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-14	Panel Error Runtime	Siswa menuliskan variabel yang belum dideklarasikan <code>console.log(undefinedVariable);</code> kemudian mengeklik tombol Jalankan.	Panel <code>#aicode-problems-panel</code> menampilkan item error berikon merah; status badge <code>#aicode-badge-error</code> terisi angka; pesan error runtime (ReferenceError) tampil jelas di kontainer <code>#aicode-errors</code> .	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-15	Eksekusi di MASALAH	Siswa menulis rangkaian kode bermasalah yang memicu stack trace error, lalu mengeklik tombol Jalankan.	Item pada panel masalah menampilkan baris letak kesalahan (data-line); ketika item masalah diklik, sistem otomatis melakukan penyorotan baris aktif via class #aicode-active-line pada editor koding.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)		Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-16	Prefilter Client-Side (editor.js preFilterCode())	Siswa mengetikkan terlarang require('child_process') lalu mengeklik Jalankan.	mencoba perintah	Fungsi preFilterCode() mengembalikan nilai true; muncul alert instan lokal pada panel bahwa kode tidak aman; proses pengiriman AJAX ke server langsung dibatalkan demi keamanan sandbox.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-17	Reset Starter Code	Siswa mengubah baris kode, mengeklik tombol Reset, kemudian memilih opsi konfirmasi setuju pada dialog konfirmasi.	Isi teks editor dikembalikan ke kondisi awal startercode; seluruh panel OUTPUT, MASALAH, dan FEEDBACK dikosongkan; objek status state AI (cachedFeedback, lastAnalysisInput) dibersihkan.	Sesuai Ekspektasi	Pass
BB-18	Buka Riwayat	Siswa telah melakukan eksekusi kode minimal sebanyak 2 kali dalam satu sesi, lalu mengeklik tombol Riwayat.	Drawer panel #aicode-history-drawer terbuka ke layar (aktif class is-open); lapisan latar backdrop #aicode-history-backdrop tampil; elemen #aicode-history-list memuat riwayat secara kronologis.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-19	Muat Riwayat dari Database	Siswa membuka kembali drawer Riwayat setelah pada sesi hari sebelumnya sempat mengerjakan latihan yang sama.	Panggilan AJAX web service mod_aicode_get_r un_history terpicu; sistem memuat hingga maksimal 20 entri attempt terakhir siswa dari tabel database aicode_attempts lalu menggabungkannya ke sessionStorage.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-20	Terapkan Kode dari Riwayat ke Editor	Siswa melakukan klik ganda (expand lalu apply) pada salah satu item log riwayat di dalam drawer.	Teks sandi lama riwayat sukses dimuat masuk menggantikan isi editor saat ini; notifikasi info 'Riwayat dimuat — Kode dari riwayat berhasil ditampilkan di editor.' muncul di panel.	Sesuai Ekspektasi	Pass
BB-21	Blokir Bantuan AI Sebelum Jalankan	Siswa membuka aktivitas dalam mode latihan (training), lalu langsung mengeklik tombol Bantuan AI tanpa mengetik atau klik Jalankan.	Panggilan AJAX diblokir; sistem menampilkan alert peringatan lokal: 'Petunjuk belum tersedia — Jalankan kode terlebih dahulu agar AI dapat menganalisis.'; kuota API tidak terpotong.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-22	Feedback Sokratik JSON	Siswa menjalankan kode yang mengalami error logika/sintaks, menunggu status selesai, lalu mengeklik tombol Bantuan AI.	AJAX mod_aicode_analyze_code memanggil API Gemini; respons JSON sukses dikembalikan berisi diagnosis.message_short/long, hints[] pedagogis (anti-spoiler); log tercatat di tabel database via mod_aicode_record_hint.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-23	Blokir Bantuan AI Saat Kode Sukses	Siswa menuliskan kode pemrograman yang sukses dieksekusi dengan status exitCode=0 tanpa error, lalu mengeklik Bantuan AI.	Sistem membatalkan pengiriman ke API; menampilkan alert penolak bahwa kode berhasil dijalankan dengan baik dan tidak mendeteksi komponen error untuk dianalisis.	Sesuai Ekspektasi	Pass
BB-24	Rate Limiting Harian	Siswa secara beruntun melakukan klik permintaan bantuan tutor AI hingga mencapai batas max_calls_per_day (kuota harian 50 kali).	Sistem melemparkan exception error 'Rate limit exceeded. Please try again tomorrow.'; token data analisis tidak diteruskan ke server AI; pengujian eksekusi run biasa tetap diizinkan.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-25	Bantuan AI Kode sama	Siswa mengeklik tombol Bantuan AI sebanyak dua kali pada kode salah yang sama persis tanpa melakukan perubahan karakter sandi.	Respons umpan balik instan langsung diambil dari tabel simpanan aicode_cache berdasarkan payload_hash; pencatatan waktu ai_requested_at tetap diperbarui; log menyimpan flag from_cache: true.	Sesuai Ekspektasi	Pass
BB-26	Ambang Kepercayaan	Model Gemini memberikan nilai keyakinan diagnosis.confidence di bawah batas minimal keyakinan sistem (confidence_threshold = 0.6).	Sistem menyaring data; mengembalikan status payload error 'low_confidence'; antarmuka panel Umpan Balik menampilkan pesan penolakan: 'Feedback AI Belum Tersedia'.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-27	Keamanan Server-Side	Siswa menuliskan kode injeksi runtime eval('console.log(1)') lalu klik Jalankan dengan kondisi status security_check_enabled=true.	Fungsi security_checker::analyze() mendeteksi kecocokan rule js_eval (risk level: high); proses eksekusi diblokir server (exitCode: -2); item masalah memuat label '[Keamanan]'.	Sesuai Ekspektasi	Pass
BB-28	Keamanan child_process	Siswa mencoba menyisipkan modul eksploitasi server luar const cp = require('child_process'); kemudian mengeklik tombol Jalankan.	Rule keamanan js_require_child_process terdeteksi cocok (risk level: critical); transfer data ke executor dibatalkan; panel Umpan Balik menampilkan alert kritis: '🔒 Kode Diblokir — Risiko Keamanan Kritis'.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-29	Keamanan Fetch Eksternal	Siswa menuliskan baris instruksi pengambilan data luar fetch('https://evil.com/api') kemudian mengeklik tombol Jalankan.	Sistem mencocokkan pola rule js_fetch_external (risk level: high); eksekusi dibatalkan seketika; log pelacakan bahaya ACTION_CODE_RUN_BLOCKED sukses ditulis ke dalam aicode_activity_log.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-30	Peringatan Risiko Rendah	Siswa menuliskan baris manipulasi variabel berisiko XSS ringan seperti <code>element.innerHTML = userInput</code> ; lalu mengeklik tombol Jalankan.	Sistem menandai aturan <code>js_innerhtml_assign (risk level: low)</code> ; apabila konfigurasi admin <code>security_block_level</code> diatur pada posisi 'high', kode diperbolehkan jalan namun bendera <code>security_flags</code> tetap disimpan.	Sesuai Ekspektasi	Pass
BB-31	Keamanan Validasi Sesskey dan Capability	Pengguna ilegal mencoba menembak langsung endpoint web service <code>mod_aicode_run_code</code> tanpa sesskey valid atau tanpa hak akses <code>mod/aicode:submit</code> .	Fungsi inti Moodle <code>confirm_sesskey()</code> atau <code>require_capability()</code> melemparkan exception error penolak keamanan; tidak ada data baris baru yang ditulis ke database attempts.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-32	Mode Ujian Sembunyi- kan Bantuan AI	Aktivitas diatur guru ke konfigurasi mode=exam, kemudian akun berhak akses siswa membuka halaman pengerjaan latihan.	Komponen tombol fisik #aicode-hint-btn (Bantuan AI) disembunyikan total dari layout DOM; fungsi pemicu analisis otomatis diproteksi oleh guard penahan !examModeForStudent di dalam fungsi handleRun().	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-33	Mode Ujian Kirim dan Kunci Editor	Siswa dalam mode ujian mengeklik tombol Kirim jawaban akhir, kemudian menekan pilihan konfirmasi 'Ya, Kumpulkan' pada jendela modal.	AJAX mod_aicode_send_to_teacher sukses; kolom flag teacher_review_requested diset aktif angka 1; editor diubah menjadi status terkunci read-only; tombol berubah menjadi 'Sudah Dikumpulkan ✓'.	Sesuai Ekspektasi	Pass
BB-34	Mode Ujian Cegah Pengiriman Ulang	Siswa yang telah berstatus mengumpulkan mencoba memanipulasi parameter elemen inspeksi untuk mengirim ulang paket data Kirim.	Validasi server-side mendeteksi status pengiriman; mengembalikan payload success: false dengan atribut already_submitted: true; penambahan record duplikasi berhasil ditolak.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-35	Mode Ujian Muat Ulang Halaman Post-Submit	Siswa yang sudah berstatus sukses mengumpulkan tugas ujian membuka kembali halaman alamat URL view.php.	Script renderer.php mendeteksi properti hassubmitted=true; sandi terakhir dikunci dan dimuat aman dari kontainer aicode-submitted-code-raw; fungsi lockEditorAfterSubmit() otomatis diaktifkan.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-36	Mode Latihan Kirim ke Guru	Siswa yang berada pada aktivitas bernilai mode=training mengeklik tombol Kirim untuk mengumpulkan tugas pengerjaannya.	Paket data terkirim dengan flag teacher_review_requested=1; sistem menampilkan baris notifikasi mengambang: 'Kode kamu telah dikirim ke guru untuk ditinjau.'; posisi panel editor koding siswa tetap terbuka.	Sesuai Ekspektasi	Pass
BB-37	Akses dan Tampilan Laporan Guru	Dosen/Guru pengampu yang memiliki hak akses khusus mod/aicode:viewattempts membuka menu halaman report.php?id=[cmid].	Halaman sukses ter-render menampilkan daftar tabel nama siswa beserta rangkuman attempt pengerjaan; fitur filter kategori data (all/submitted/with_ai) dapat berfungsi memetakan baris.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-38	Override Feedback AI	Guru yang memiliki hak akses mod/aicode:overridefeedback mengedit teks diagnosis bimbingan AI siswa yang kurang akurat lalu klik Simpan.	Data koreksi manual guru sukses tersimpan/terperbarui di dalam tabel aicode_teacher_overrides; data bisa diinjeksikan sebagai basis data few-shot pembelajaran AI berikutnya.	Sesuai Ekspektasi	Pass
BB-39	Simpan Nilai ke Gradebook	Guru menginputkan komponen angka nilai evaluasi (misal: 85) pada baris form siswa di halaman laporan lalu mengeklik tombol Simpan.	Fungsi internal Moodle aicode_set_user_grade() memicu jalannya fungsi grade_regrade_final_grades(); muncul notifikasi sukses; perolehan nilai otomatis terintegrasi masuk ke Buku Nilai Moodle.	Sesuai Ekspektasi	Pass

ID Uji	Komponen / Fitur	Skenario Pengujian (Input / Tindakan)	Hasil yang Diharapkan (Output Sistem)	Hasil Aktual	Status
BB-40	Logging Attempt Run	Siswa melakukan eksekusi perintah koding pada paket modul latihan yang dikonfigurasi admin dengan opsi allow_training=0.	Data terekam lengkap ke database aicode_attempts memuat userid, code_hash berwujud SHA-256, serta detail log output eksekusi koding pada field result_json; log aksi ditulis ke tabel aicode_activity_log.	Sesuai Ekspektasi	Pass